

JCOP 4 P71

User manual for JCOP 4 P71

Rev. 3.7 – 20190531

NXP doc. no. 469537

User Guidance and Administrator Manual

COMPANY CONFIDENTIAL

Document Information

Info	Content
Keywords	AGD_OPE, AGD_PRE, User manual for JCOP 4 P71, JCOP 4 P71
Abstract	This is the JCOP 4 P71 v4.7 R1.00.4 User Guidance and Administrator manual. It gives all relevant information for applet developers and administrators of JCOP 4 P71 on SmartMX3 P71.

Rev	Date	Description
3.7	20190531	Updated Section 4.2.8 - Limitations and REQ_CRYPT_SHA in Section 6.4 - Cryptographic requirements . Added note to Section 4.2.3 - Sending response data . Updated Table 2.1.
3.6	20190412	Updated Table 8.1. Added note to Section 8.3.4 - FIPS support . Updated Table 3.2.
3.5	20190325	Corrected errors thrown by <code>closeProcessDataSession()</code> in Section 5.4.4.2 - Class MemoryAccessX . Updated Section 5.4.5.1 - Class MemoryAccessServiceInterface . Added document references to Section 1 - Introduction . Added note to INSTALL[for registry update] in Section 4.5.2 - APDUs .
3.4	20190308	Removed duplicate sections from 8 - Product configuration and updated 4.7 - WTX configurations . Added Table 5.45. Tidied up Table 5.4.9.6. Added note to 8.3.1.3 - TCL_ATS_IF . Fixed bibliography issue.
3.3	20181213	Updates after user feedback: Added REQ_SCP02 to Section 6.2. Updated Tables 5.32, 8.4, and Section 4.5.9.
3.2	20181116	Updates after extensive architect review. Added Table 4.3, Section 8.3.3.6 and Section 4.2.7. Updated Table 10.1, Table 2.1, Section 5.4.8, Section 5.4.4, Section 5.4.5, Section 5.4.6, Section 5.4.10.6, Section 6.4, Section 4.2.5.2. Various other tweaks and improvements.
3.1	20180907	Updated Section 2.
3.0	20180905	Final release
2.4	20180727	Updated after review.
2.3	20180622	Updated in line with evaluator feedback.
2.2	20180417	Updated after architect review.
2.1	20180413	Updated after architect review.
2.0	20180309	Reviewed and approved.
1.3	20180308	Updated after architect review.
1.2	20180227	Updated after architect review.
1.1	20180221	Update after architect review. Most sections are affected, in particular: 5 - Proprietary features and platform-dependent behavior, 6 - Product security and 7 - Pre-personalization (OS initialization).
1.0	20180213	Initial release.

1 Introduction

NXP Semiconductors offers a Java Card Operating System (OS) called JCOP. It is based on independent, third-party specifications by Oracle Corporation, the GlobalPlatform consortium, the International Organization for Standardization (ISO), EMVCo, and others. This document describes the features implemented in JCOP 4 P71.

The documentation for NXP's Java Card OS consists of the following documents:

- JCOP 4 P71 User Guidance and Administrator Manual, doc. no. 469537 [28] (this document).
- SmartMX3 P71 Family P71D321 Overview, Pinning and Electrical Characteristics Product Short Data Sheet, doc. no. 412530 [27].
- SmartMX3 P71 JCOP Delivery Forms and Electrical Characteristics, doc. no. 458010 [26].
- NXP Secure Smart Card Controller Antenna Design Guide Application Note, doc. no. 497630 [25].

This document outlines JCOP 4 P71 features, the OS initialization process and its specific commands, features available through the Java Card Application Programming Interface (API), and additional functionality on the Java layer. It also describes how to order JCOP.

1.1 Audience

The intended audience for this document are JCOP card administrators and providers of applications which execute on a JCOP card. All users of this manual should be familiar with the GlobalPlatform Card Specification 2.3 and Java Card Classic 3.0.5 as a minimum.

Refer to the bibliography for other specifications and documents referenced in this document.

1.2 Roles

This document refers to different roles:

- The end user is the final recipient of the card.
- A card administrator is someone who adjusts the product at some point before it is issued to end users. Alternative terms for the card administrator are:
 - Customer: Entity that buys JCOP 4 P71 from NXP.
 - Card manufacturer: Entity that manufactures smart cards.
 - Card issuer: “Entity that owns the card and is ultimately responsible for the behavior of the card” (GlobalPlatform card specification [15]).

These terms are used instead of card administrator for clarity. They are defined in specifications or commonly used and well understood in the industry.

- Application provider: “Entity that owns an application and is responsible for the application’s behavior” [15]. In this document, that definition includes applet developers.

1.3 Reading this document

All readers benefit from reading the initial chapters of this document:

- To find out how to identify a product, see Section 2 - [Product identification](#).
- For a general description of JCOP 4 P71, see Section 3 - [Product description](#). It lists application options, supported communication interfaces, cryptographic algorithms and other features.
- For details of the standard features that JCOP supports with specific focus on Java Card, GlobalPlatform and communications, see Section 4 - [Standard features](#).

For information on JCOP proprietary features, security and how to configure the product:

- For list of proprietary features, including proprietary Java Card APIs, see Section 5 - [Proprietary features and platform-dependent behavior](#).
- For a description of the product security features, see Section 6 - [Product security](#).
- For a discussion on pre-personalization, see Section 7 - [Pre-personalization \(OS initialization\)](#).
- For a list of available configuration options, see Section 8 - [Product configuration](#).

For information on how to order NXP products:

- Before ordering NXP products, see Section 9 - [Ordering and delivery](#).
- To find out how to use the Order Entry Form to order customized products, see Section 9.2 - [Customer type submission using Order Entry Form](#).
- For information about product delivery, see Section 9.4 - [Product delivery](#).

2 Product identification

Products implementing JCOP 4 P71 can be identified using the IDENTIFY command (see Section [5.1.1.1 - GET DATA IDENTIFY](#)).

When it is delivered, JCOP 4 P71 is in transport state with limited functionality. Only the Card Manager is selectable (and typically also default selected) or a bulk update can be performed. There is only a limited set of APDUs allowed in this state. Within this transport state, JCOP can be identified by sending the IDENTIFY command to the ISD (see Section [7.4 - Config Module](#) for details).

Table [2.1](#) lists the results which are obtained for a JCOP 4 P71 product after delivery.

Tab. 2.1: Product identification

ID	Value
Patch ID (tag 02)	0000000000000001
ROM ID (tag 08)	2E5AD88409C9BADB
Platform ID	JCOP 4 P71 v4.7 R1.00.4: 4A335233353130314641394530343030DD0984593B0048EF JCOP 4 P71 v4.7 R1.01.4: 4A335233353130323336333130343030DCE5C19CFE6D0DCF

Further information of the card configuration is available in the Order Entry Form (see Section [9.2 - Customer type submission using Order Entry Form](#)). The Order Entry Form can be identified as part of the FLASH-ID which is returned by the IDENTIFY command.

After Issuer Security Domain (ISD) authentication the card is switched to operational state where the complete functionality of JCOP 4 P71 becomes available.

3 Product description

The architecture of JCOP 4 P71 is based on several independent third-party specifications: the Java Card specification from Oracle, the GlobalPlatform specification from the GlobalPlatform consortium, the specifications from International Organization for Standards (ISO), EMV (Europay, MasterCard and VISA), and others.

These industry standards together ensure application interoperability for card issuers as well as application providers. By adhering not just to the standards themselves, but also to their spirit (as evidenced in numerous heritage applications), JCOP 4 P71 ensures large interoperability with third-party applets as well as all existing Smart Card infrastructures. With JCOP 4 P71 the promise of multi-sourcing any component in smart card solutions becomes true. Even in existing infrastructures, JCOP 4 P71 equipped with proper applications can substitute any existing smart card.

Within its targeted segments, the new JCOP 4 P71 platform on SmartMX3 is the most advanced solution available. It combines standard interfaces as defined in Java Card 3.0.5 Classic (see [23], [21], [22]), GlobalPlatform 2.3 (see [15]), and powerful cryptographic capabilities by using coprocessors for public and secret key encryption. The secret key encryption supports Rivest Shamir Adleman asymmetric algorithm (RSA), Elliptic Curve Cryptography (ECC), Advanced Encryption Standard (AES), and Data Encryption Standard with 3 keys (3DES). JCOP does all this within a high security, ultra low power, performance optimized design concept. The platform supports voltage classes “C”, “B”, “extended B”, and “A” (1.62 - 5.5 V) as required by application standards such as the credit/debit card standard (EMV).

For further details on general JCOP 4 P71 platform features, see Section [4.1 - JCOP 4 P71 product family features](#).

3.1 Application options

JCOP 4 P71 is a conversion platform that supports EMV and SECID applications. It can be used for the following applications:

- EMVCo payment card
- Electronic passport (ePP) providing BAC, EACv1 and SAC/PACE support
- European Citizen Card (EN 15480)
- European Health Insurance Card (CDA15974-2009 E)
- Fingerprint Match on Card (ISO 19794) — MINEX III compliant
- International Driving License BAP and EAP (ISO 18013)
- SECID applications such as ePKI, eVR and eRP

- MasterCard, Visa, American Express, Discover and other payment applications

JCOP 4 P71 is highly configurable. The base configuration includes all required functionality for Europay, MasterCard and Visa (EMV) payment cards. The Secure Identification (SECID) configuration adds additional functionality typically required in passport, ID, or similar applications.

Some features are available as add-ons. Add-ons have to be selected when ordering (see Section 9 - [Ordering and delivery](#)). Some are included in the main configurations by default, others are selectable only for certain configurations.

Table 3.1 shows which features are included in which default configuration (EMV or SECID) and which are selectable.

Tab. 3.1: JCOP product family features

Add-on	Feature	EMV	SECID
OS features¹			
	ISO/IEC 7816	Y	Y
	ISO/IEC 14443 (2016)	Y	Y
	Java Card 3.0.5 Classic	Y	Y
	GlobalPlatform ID Configuration v1.0	Y	Y
	GlobalPlatform Common Implementation Configuration v2.0	Y	Y
	GlobalPlatform Card Financial Configuration	Y	Y
	GlobalPlatform Mapping Guidelines	Y	Y
	EMV 4.3	Y	Y
	EMV CL 2.6	Y	Y
	Support for PUF	Y	Y
	CL EAL6+, EMV Open Platform, BCTC OS	Y	Y
Cryptographic features			
	Data Encryption Standard (DES) and dual/triple key DES via coprocessor	Y	Y
	AES via coprocessor (128, 192 and 256 bit)	Y	Y
	RSA via coprocessor: up to 4096 bit up to 2048 bit	Y Y	Y Y
	Other cryptographic support such as SHA-1, SHA-224, SHA-256, SHA-384, SHA-512 and CRC 16 & 32 according to ISO 3309	Y	Y
RSA key gen add-on	RSA key generation	-	Selectable

Continued on the next page.

Tab. 3.1 – Continued from the previous page.

Add-on	Feature	EMV	SECID
ECC add-on	- ECC via coprocessor (up to 521 bit)	-	Y
	- ECC key gen	-	Y
Korean Seed add-on	- Secure Korean Seed algorithm (KTFC certified)	Selectable	-
Interfaces			
	Contact interface with T=0 and T=1 protocols according to ISO/IEC 7816	Y	Y
	Contactless interface with T=CL protocol according to ISO/IEC 14443 Type A (2016)	Y	Y
Java Card and GlobalPlatform features			
	Java Card 3.0.5 Classic	Y	Y
	Inter-applet PIN sharing	Y	Y
	Loader Service ready	Y	Y
	Secure in-field applet download	Y	Y
	DRG.3 compliant pseudo-random number generator (see 4.1 - JCOP 4 P71 product family features)	Y	Y
SECID add-on	- Support for ISO compliance and ICAO 9303-1 - Machine Readable Travel Documents Part 1 - SAC and PACE support - Secure Messaging Accelerator (eGovAccelerator) - Additional JCOP APIs	-	Y
Biometric features			
MoC ID3 add-on ²	Biometric APIs	Selectable	Selectable
MoC Neurotechnology add-on ²	Biometric APIs	Selectable	Selectable
Other features			
FIPS add-on	- FIPS 140-2 SL3 - ISO/IEC 1979 - Support for PIV Opacity	-	Selectable
Secure Box	Secure native execution of 3 rd party libraries	Y	Y
Config Module	Used for OS initialization	Y	Y

¹ For more details on supported specifications (see Section 11 - Supported specifications).

² Only one MoC library can be enabled at a time (see Section 3.3 - Integrated MoC).

3.2 Communication interfaces

JCOP 4 P71 provides an interface for the ISO/IEC 7816 [17] communication. If the JCOP hardware supports a contactless communication interface, then ISO/IEC 14443 Type A [19] based communication is also supported.

If the hardware implements two communication interfaces, the current card session uses the interface that received the first clock signal.

The active communication interface can be changed by resetting the card and sending data on the other communication interface.

The rest of this document does not further distinguish between products with one or two hardware communication interfaces. Descriptions for the contactless interface apply only to products with such hardware interface.

JCOP 4 P71 supports the following communication protocols:

- ISO/IEC 7816-3 T=1 direct convention (default),
- ISO/IEC 7816-3 T=0 direct convention,
- ISO/IEC 7816-3 T=1 inverse convention,
- ISO/IEC 7816-3 T=0 inverse convention
- ISO/IEC 14443 Type A T=CL.

For further details see Section [4.6 - Communications](#).

3.3 Integrated MoC

The biometrics algorithm is provided by the JCOP platform as an add-on using Secure Box. Only one MoC library can be active at a time. Decide which library, if any, should be enabled prior to ordering the product (see see Section [9 - Ordering and delivery](#)). For information on how to use the activated library, and details on the exact features available, refer to its user manual.

- Minutiae-based fingerprint comparison algorithm (ISO/IEC 19794-2 compact card format).
- PIV-compliant, certified by NIST in MINEX III program.
- Configurable rotation tolerance up to 180°.
- Maximum number of minutiae: up to 128 (depending on allocated memory).

3.4 Cryptographic algorithms and key sizes

JCOP 4 P71 products support 3DES and AES in Cipher Block Chaining (CBC) and Electronic codebook (ECB) mode, RSA, ECC, and the Korean SEED algorithm. JCOP can generate all RSA and ECC keys on the card for maximum security and supports the hashing methods SHA-1 and SHA-2.

For certified usage JCOP supports the following length of cryptographic keys and Secure Hash Algorithm (SHA) algorithms as defined in the Java Card API [\[21\]](#):

- 3DES: 3DES with 2 keys and 3DES with 3 keys
- AES: 128 bits, 192 bits, and 256 bits
- RSA: 512 bits up to 2048 bits in 8-bit steps; optionally (via Order Entry Form) up to 4096 bits in 8-bit steps.

Note: JCOP supports RSA keys starting at 512 bits. Select a sufficient strength for the intended application.

- ECC: 224 bits up to 521 bits
- SHA: SHA_224, SHA_256, SHA_384, SHA_512

Table 3.1 shows which features are included in which standard configuration, as well as which features are available through add-ons.

JCOP supports additional cryptographic algorithms which are not part of the Java Card API. The use of these algorithms is certified, for details see Section 5.4.10 - JCOPX Security.

3.5 Product customization options

NXP provides a technology process to create transparent blends between JCOP 4 P71 and any set of applets. Standard applications of a particular card issuer as well as native libraries for Secure Box¹ can be put into the Non-Volatile Memory (NVM) during the chip production.

For details of available memory configurations in JCOP products, see Section 3.6 - Available memory.

Note: The memory footprint of an applet installed as part of the customization process may vary from the numbers reported by CAP File Converter. The difference is due to how data structures are handled.

The customization process does not impact the JCOP OS platform's functionality, performance, or security. This process can be used to pre-load applets and Secure Box native libraries. An example memory configuration is shown in figure 3.1.

The following product behavior can be observed for customized products:

- The customized card contains only the Executable Load File (ELF) of the applet.
- The ELF of the applet is always registered to the ISD.
- The ELF of the applet can be physically deleted. The space is freed up again when the GlobalPlatform (GP) DELETE command is finished. To allow resources to be recovered when deleting a Secure Box library, only the Card Manager should be selected. No other applet should be selected on another logical channel. If another applet is selected, an unhandled exception can occur. Triggering an unhandled exception multiple times leads to an unresponsive card.

¹The Secure Box feature allows third-party libraries to be securely loaded and accessed. It is described in a separate manual.

The customization process also has no impact on a certificate which may have been issued for a particular version of JCOP.

Fig. 3.1: Examples of customization for JCOP.

3.6 Available memory

The amount of free memory for applets and data as returned by the command GET DATA (Available Memory) (5.1.3.1 - [Get available memory](#)) is listed in Table 3.2. The given values need to be read in combination with the required features on the platform. The basic OS leaves space for the technical modules that may be required in certain configurations.

The available PHEAP space can be increased according to Table 3.2 by deleting any preloaded item.

Note: The Config Module requires space until it is deleted.

For details on the Config Module see 7.4 - [Config Module](#).

Note: The amount of free memory depends on which add-ons (made up by one or more modules) and applets are installed on the card. To estimate the size available for a configuration, start with the card memory size and subtract module and applet sizes. For module sizes, see Table 3.2. For application sizes, refer to the user manual for the individual applet.

Tab. 3.2: Memory consumption by module

Module	PHEAP	Deletable/memory reclaimable ¹
Config module (<code>config_module</code>)	5 216	Yes
ECC (<code>ecc</code>)	33 208	Yes
eGov (<code>egovaccelerators</code>)	8 304	Yes
FIPS (<code>fips</code>)	8 288	Yes
Korean SEED (<code>koreanseed</code>)	8 652	Yes
Opacity (<code>piv</code>)	2 084	Yes
PACE IM (<code>paceim</code>)	2 412	Yes
RSA key generation (<code>rsakeygen</code>)	4 972	Yes
Secure Box (<code>securebox</code>)	9 808	Yes

¹ Modules may be switched to persistent, depending on configuration. Persistent modules cannot be deleted and the memory cannot be reclaimed. Attempts to delete a persistent module result in an error message.

See Section 5.2 - OS modules for information about individual modules and which add-ons they are part of.

3.7 Designed-in support

NXP provides the following support:

- Development environment
 - JCOP Eclipse Generic Plugin
 - JCOP Eclipse Target Pack
 - Secure Box Development Framework
 - JCShell Standalone
 - SCCommUI (Smart Card Communication User Interface):
 - * A graphical user interface for smart card operating systems
- NXP Semiconductors Customer Application Support (CAS)
- JCOP 4 P71 sample modules or cards

4 Standard features

This chapter lists features which are available to all JCOP 4 P71 products.

4.1 JCOP 4 P71 product family features

- Java Card 3.0.5 Classic
- GlobalPlatform 2.3 ([15]) (CI v2.0, see [14])
- GlobalPlatform Mapping Guidelines (see [11])
- GlobalPlatform Secure Channel Protocol 01, 02 and 03 (see GlobalPlatform 2.3 [15])
- GlobalPlatform Delegated Management, DAP and Authorized Management
- Data Encryption Standard (DES) and dual/triple key DES via coprocessor
- AES via coprocessor
- RSA via coprocessor
- ECC via coprocessor
- Other cryptographic support such as SHA-1, SHA-224, SHA-256, SHA-384, SHA-512 and CRC support
- Contact interface with T=0 and T=1 protocols according to ISO/IEC 7816-3
- Contactless interface with T=CL protocol according to ISO/IEC 14443 Type A
- FIPS certified (see ISO/IEC 19790 [4])
- Additional JCOP 4 P71 APIs (see Section 5.4 - [JCOPX Java Card API extension](#)).
- DRG.3 compliant pseudo-random number generator according to AIS 20 [9]. Random numbers requested by a function call to the `RandomData` class are acquired from this random number generator. This includes the random challenge values used in the SCP protocols as defined by GlobalPlatform [15]. DRG.3 is the default RNG setting.

4.2 Java Card 3.0.5 Classic

JCOP 4 P71 implements Java Card 3.0.5 Classic (API [21], Runtime Environment [22] and Virtual Machine [23]).

All mandatory Java Card APIs defined can be invoked by an applet. However, some of these APIs provide restricted functionality or no functionality at all.

4.2.1 API packages

The tables in this section list the supported extension packages of the Java Card API. APIs not listed below have no functionality in JCOP 4 P71 or throw an exception.

- `javacardx.apdu` (see Table 4.1).
- `javacardx.crypto` (see Table 4.2).

Tab. 4.1: Supported APIs of package `javacardx.apdu`

Interface	Method	Comment
ExtendedLength		See Section 4.2.4 - Extended length APDU support

Tab. 4.2: Supported APIs of package `javacardx.crypto`

Interface	Method	Comment
Cipher	<code>doFinal()</code> <code>getAlgorithm()</code> <code>getInstance()</code> <code>init()</code> <code>update()</code>	See Section 4.2.5 - Cryptography

4.2.2 Protocols

The method `APDU.getProtocol()` returns information about the currently active communication protocol. The return value is split in the transport protocol type in the low nibble and a transport media field in the high nibble, conforming to Java Card API [21].

JCOP supports the following constants (defined in the class `javacard.framework.APDU`) for the transport protocol type:

PROTOCOL_T0

```
public static final byte PROTOCOL_T0 = (byte) 0x00;
```

Transport protocol T=0.

PROTOCOL_T1

```
public static final byte PROTOCOL_T1 = (byte) 0x01;
```

Transport protocol T=1.

JCOP supports the following constants (defined in the class `javacard.framework.APDU`) for the transport media:

PROTOCOL_MEDIA_DEFAULT

```
public static final byte PROTOCOL_MEDIA_DEFAULT = (byte) 0x00;
```

Default protocol media.

PROTOCOL_MEDIA_CONTACTLESS_TYPE_A

```
public static final byte PROTOCOL_MEDIA_CONTACTLESS_TYPE_A = (byte) 0x80;
```

Protocol media contactless type A.

Note: To find out whether contactless or contact (wired) communication is used, check bit b8 of the transport media. If this bit is set to 0 the active communication interface is based on a contact (wired) communication link. If this bit is set to 1 the active communication interface is based on a contactless communication link.

The protocol T=CL is coded as PROTOCOL_T1 and PROTOCOL_MEDIA_CONTACTLESS_TYPE_A. The protocols T=0 and T=1 on the contact based ISO/IEC 7816 interface are coded as PROTOCOL_MEDIA_DEFAULT, and PROTOCOL_T0 or PROTOCOL_T1 respectively.

4.2.3 Sending response data

Java Card applets can send response data via the methods `APDU.sendBytes()` and `APDU.sendBytesLong()`. JCOP 4 P71 supports both methods.

For `sendBytes` or `sendBytesLong` the data is set immediately to the Interface Device (Interface Device (IFD)) unless it contains the last block of outgoing data. The OS determines the last block by the applets call to `APDU.setOutgoingLength()`. The last block will be delayed in order to include the status word. If there is not space in the Application Protocol Data Unit as defined in ISO/IEC 7816 (APDU) buffer to append a status word at the end of the block then an additional frame is sent immediately.

In T=0, multiple calls to `sendBytes` or `sendBytesLong` results in the following T=0 protocol chaining sequence:

1. `sendBytes(length m)` (first `sendBytes`):
 - (a) Send m-1 bytes sent to IFD with 0x6101 status word.

2. `sendBytes(length n):`
 - (a) Send last byte from previous `sendBytes` with 61xx status word where xx is length n-1.
 - (b) Send n-1 bytes sent to IFD with 0x6101 status word.
3. `sendBytes(length p) (last block)`
 - (a) Send last byte from previous `sendBytes` with 61xx status word where xx is length p.

The last block is sent to the IFD with status word on exit from the applet process method.

Note: For general security the APDU buffer should not be used for storing confidential data. If it is used then the confidential data should be overwritten with randoms before sending the response APDU.

4.2.4 Extended length APDU support

JCOP 4 P71 supports APDUs with extended length for protocols T=1 and T=CL. For T=0 only short APDUs are supported.

Incoming APDUs which fit entirely into the APDU buffer will be retrieved by the lower software layer in one chunk. The applet's process method is then called with the full APDU available.

In case the incoming APDU is longer than the APDU buffer size, the chaining mode of the underlying protocol (T=1 or T=CL) will be used to stop the card reader from transmitting more bytes than would fit into the APDU buffer. The applet's process method is called with the maximum number of data blocks which fit into the APDU buffer. It is then the responsibility of the applet to ask for more bytes being retrieved from the card reader.

In case the response data of an applet does not fit entirely into the APDU buffer, the chaining mode of the underlying protocol will be used. The program flow will return to the applet when all response data has been sent to the card reader.

Note: The full APDU buffer size is also available after receiving a short APDU. Because it is not allowed to respond with a long APDU on an incoming short APDU, the response length is limited to 256 byte in this case.

4.2.5 Cryptography

This section describes the cryptographic primitives supported by JCOP 4 P71. Refer also to the Java Card API description [21] for more details on the individual primitives.

4.2.5.1 Checksum algorithms

The following checksum algorithms are supported:

- ALG_ISO3309_CRC16
- ALG_ISO3309_CRC32

4.2.5.2 Cipher algorithms

The following cipher algorithms are supported:

- ALG_AES_CBC_ISO9797_M1
- ALG_AES_CBC_ISO9797_M2
- ALG_AES_CBC_PKCS5
- ALG_AES_BLOCK_128_CBC_NOPAD
- ALG_AES_BLOCK_128_ECB_NOPAD
- ALG_AES_ECB_ISO9797_M1
- ALG_AES_ECB_ISO9797_M2
- ALG_AES_ECB_PKCS5
- ALG_DES_CBC_NOPAD
- ALG_DES_CBC_ISO9797_M1
- ALG_DES_CBC_ISO9797_M2
- ALG_DES_CBC_PKCS5
- ALG_DES_ECB_NOPAD
- ALG_DES_ECB_ISO9797_M1
- ALG_DES_ECB_ISO9797_M2
- ALG_DES_ECB_PKCS5
- ALG_KOREAN_SEED_CBC_NOPAD¹
- ALG_KOREAN_SEED_ECB_NOPAD²
- ALG_RSA_NOPAD
- ALG_RSA_PKCS1
- ALG_RSA_PKCS1_OAEP
- CIPHER_AES_CBC

¹Requires the Korean Seed add-on (see Section 3 - [Product description](#)).

²Requires the Korean Seed add-on (see Section 3 - [Product description](#)).

- CIPHER_AES_ECB
- CIPHER_DES_CBC
- CIPHER_DES_ECB
- CIPHER_KOREAN_SEED_CBC³
- CIPHER_KOREAN_SEED_ECB⁴
- CIPHER_RSA
- MODE_DECRYPT
- MODE_ENCRYPT
- PAD_ISO9796
- PAD_ISO9797_1_M2_ALG3
- PAD_ISO9797_M1
- PAD_ISO9797_M2
- PAD_NOPAD
- PAD_NULL
- PAD_PKCS1
- PAD_PKCS1_OAEP
- PAD_PKCS1_PSS

Note: Not all combinations of CIPHER and PAD algorithms are possible and supported.

Note: If the input buffer and output buffer partially overlap, the output of the crypto operation may be incorrect. This is especially the case when input data is modified with a previous block operation. The JCOP 4 simulator can behave differently in this case, as the input data is copied before the usage leading to a correct output.

³Requires the Korean Seed add-on (see Section 3 - [Product description](#)).

⁴Requires the Korean Seed add-on (see Section 3 - [Product description](#)).

4.2.5.3 Signature algorithms

The following signature algorithms are supported:

- ALG_AES_MAC_128_NOPAD
- ALG_AES_CMAC_128
- ALG_AES_CMAC8
- ALG_DES_MAC8_NOPAD
- ALG_DES_MAC8_ISO9797_1_M2_ALG3
- ALG_DES_MAC4_ISO9797_1_M2_ALG3
- ALG_DES_MAC8_ISO9797_M1
- ALG_DES_MAC4_ISO9797_M1
- ALG_DES_MAC8_ISO9797_M2
- ALG_DES_MAC4_ISO9797_M2
- ALG_DES_MAC8_ISO9797_1_M1_ALG3
- ALG_DES_MAC4_ISO9797_1_M1_ALG3
- ALG_DES_MAC8_PKCS5
- ALG_DES_MAC4_PKCS5
- ALG_KOREAN_SEED_MAC_NOPAD
- ALG_ECDSA_SHA⁵
- ALG_ECDSA_SHA_224⁶
- ALG_ECDSA_SHA_256⁷
- ALG_ECDSA_SHA_384⁸
- ALG_ECDSA_SHA_512⁹
- ALG_RSA_SHA_224_PKCS1_PSS

⁵Requires the ECC add-on (see Section 3 - [Product description](#)).

⁶Requires the ECC add-on (see Section 3 - [Product description](#)).

⁷Requires the ECC add-on (see Section 3 - [Product description](#)).

⁸Requires the ECC add-on (see Section 3 - [Product description](#)).

⁹Requires the ECC add-on (see Section 3 - [Product description](#)).

- ALG_RSA_SHA_224_PKCS1
- ALG_RSA_SHA_256_PKCS1
- ALG_RSA_SHA_256_PKCS1_PSS
- ALG_RSA_SHA_256_ISO9796
- ALG_RSA_SHA_384_PKCS1
- ALG_RSA_SHA_384_PKCS1_PSS
- ALG_RSA_SHA_512_PKCS1
- ALG_RSA_SHA_512_PKCS1_PSS
- ALG_RSA_SHA_ISO9796
- ALG_RSA_SHA_PKCS1
- ALG_RSA_SHA_PKCS1_PSS
- ALG_RSA_SHA_ISO9796_MR
- ALG_KOREAN_SEED_MAC_NOPAD
- MODE_SIGN
- MODE_VERIFY
- SIG_CIPHER_AES_MAC128
- SIG_CIPHER_AES_CMAC128
- SIG_CIPHER_DES_MAC4
- SIG_CIPHER_DES_MAC8
- SIG_CIPHER_ECDSA¹⁰
- SIG_CIPHER_RSA
- SIG_CIPHER_KOREAN_SEED_MAC¹¹

¹⁰Requires the ECC add-on (see Section 3 - [Product description](#)).

¹¹Requires the Korean Seed add-on (see Section 3 - [Product description](#)).

Tab. 4.3: RSA signature hash/padding combinations

	ISO9796	ISO9796_MR	PKCS1	PSS
MD5	-	-	-	-
SHA1	Yes	Yes	Yes	Yes
SHA224	-	-	Yes	Yes
SHA256	Yes	-	Yes	Yes
SHA384	-	-	Yes	Yes
SHA512	-	-	Yes	Yes

4.2.5.4 Key agreement algorithms

The following key agreement algorithms are supported¹²:

- ALG_EC_SVDP_DH
- ALG_EC_SVDP_DH_KDF
- ALG_EC_SVDP_DHC
- ALG_EC_SVDP_DHC_KDF
- ALG_EC_SVDP_DH_PLAIN
- ALG_EC_SVDP_DHC_PLAIN
- ALG_EC_PACE_GM¹³

4.2.5.5 Keys

Conforming to the the Java Card specification, the input array is not cleared after invoking a method to set a key or key parameter. If confidentiality of the key or the key parameter is required, it is recommended to overwrite the input array with a random number.

JCOP does not support the key encryption mechanism in the `KeyBuilder.buildKey()` method. Therefore the parameter `keyEncryption` shall be set to `false`, otherwise, JCOP will throw a `CryptoException.NO_SUCH_ALGORITHM`.

¹²They all require the ECC add-on (see Section 3 - [Product description](#)).

¹³Requires the eGov add-on (see Section 3 - [Product description](#)).

4.2.5.5.1 Algorithmic key types

The supported algorithmic key types are listed below.

- ALG_TYPE_AES (TRANSIENT_DESELECT and TRANSIENT_RESET)
- ALG_TYPE_DES (TRANSIENT_DESELECT and TRANSIENT_RESET)
- ALG_TYPE_EC_FP_PRIVATE (TRANSIENT_DESELECT and TRANSIENT_RESET)
- ALG_TYPE_EC_FP_PUBLIC
- ALG_TYPE_EC_FP_PARAMETERS
- ALG_TYPE_RSA_CRT_PRIVATE (TRANSIENT_DESELECT and TRANSIENT_RESET)
- ALG_TYPE_RSA_PRIVATE (TRANSIENT_DESELECT and TRANSIENT_RESET)
- ALG_TYPE_RSA_PUBLIC
- ALG_TYPE_KOREAN_SEED (TRANSIENT_DESELECT and TRANSIENT_RESET)

4.2.5.5.2 Key lengths

The supported key lengths are listed below for the certified cryptographic key length. For RSA algorithm any key length between 512 bits and 2048 bits is supported by default. A maximum length for RSA keys of 4096 bits is optionally supported and can be ordered via Order Entry Form. For ECC algorithm any key length between 128 bits and 521 bits is supported. Either the constants given below or the value itself can be used.

- LENGTH_AES_128
- LENGTH_AES_192
- LENGTH_AES_256
- LENGTH_DES
- LENGTH_DES3_2KEY
- LENGTH_DES3_3KEY
- LENGTH_EC_FP_128
- LENGTH_EC_FP_160
- LENGTH_EC_FP_192
- LENGTH_EC_FP_224

- LENGTH_EC_FP_256
- LENGTH_EC_FP_320
- LENGTH_EC_FP_384
- LENGTH_EC_FP_521¹⁴
- LENGTH_RSA_512
- LENGTH_RSA_736
- LENGTH_RSA_768
- LENGTH_RSA_896
- LENGTH_RSA_1024
- LENGTH_RSA_1280
- LENGTH_RSA_1536
- LENGTH_RSA_1984
- LENGTH_RSA_2048
- LENGTH_RSA_4096
- LENGTH_KOREAN_SEED_128

4.2.5.5.3 Key memory types

The supported key memory types are given below:

- TYPE_AES
- TYPE_AES_TRANSIENT_DESELECT
- TYPE_AES_TRANSIENT_RESET
- TYPE_DES
- TYPE_DES_TRANSIENT_DESELECT
- TYPE_DES_TRANSIENT_RESET
- TYPE_EC_FP_PRIVATE

¹⁴The maximum supported length for the EC FP algorithm is 521 bit.

- TYPE_EC_FP_PRIVATE_TRANSIENT_DESELECT
- TYPE_EC_FP_PRIVATE_TRANSIENT_RESET
- TYPE_EC_FP_PUBLIC
- TYPE_RSA_CRT_PRIVATE
- TYPE_RSA_CRT_PRIVATE_TRANSIENT_DESELECT
- TYPE_RSA_CRT_PRIVATE_TRANSIENT_RESET
- TYPE_RSA_PRIVATE
- TYPE_RSA_PRIVATE_TRANSIENT_DESELECT
- TYPE_RSA_PRIVATE_TRANSIENT_RESET
- TYPE_RSA_PUBLIC
- TYPE_KOREAN_SEED
- TYPE_KOREAN_SEED_TRANSIENT_DESELECT
- TYPE_KOREAN_SEED_TRANSIENT_RESET

4.2.5.6 Key pairs

The following key pairs are supported:

- ALG_RSA
- ALG_RSA_CRT
- ALG_EC_FP

4.2.5.7 Message digests

The following message digest algorithms with the corresponding lengths are supported for the certified hashing algorithms.

- ALG_NULL
- ALG_SHA
- ALG_SHA_224
- ALG_SHA_256
- ALG_SHA_384

- ALG_SHA_512
- LENGTH_SHA
- LENGTH_SHA_224
- LENGTH_SHA_256
- LENGTH_SHA_384
- LENGTH_SHA_512

4.2.5.8 Random numbers

The JCOP OS provides random numbers from a certified deterministic random number generator. This generator is automatically seeded at boot up of JCOP and is used to provide all random numbers requested via `RandomData.generateData()`. Only the algorithm types listed below can be used to request an instance of `RandomData` during `RandomData.getInstance()`. The algorithms result in the same source of random numbers. It is proposed to use `ALG_PSEUDO_RANDOM`. It is not needed and not recommended to call `RandomData.setSeed()`. `RandomData.setSeed()` performs no operation when called.

The following algorithms are available to request an instance of `RandomData`:

- ALG_PSEUDO_RANDOM
- ALG_SECURE_RANDOM
- ALG_FAST
- ALG_KEYGENERATION
- ALG_PRESEEDED_DRBG
- ALG_TRNG

Note: All random number algorithms are mapped to the same RNG source.

4.2.6 Exception handling

The following exception classes are implemented.

- `APDUException`
- `ArithmeticException`
- `ArrayIndexOutOfBoundsException`

- `ArrayStoreException`
- `BioException`
- `Bio1toNException`

Note: The `BioException` and `Bio1toNException` exception classes are supported if integrated MoC support has been enabled (see Section [3.3 - Integrated MoC](#)).

- `CardException`
- `CardRuntimeException`
- `ClassCastException`
- `CryptoException`
- `Exception`
- `ExternalException`
- `IndexOutOfBoundsException`
- `IOException`
- `ISOException`
- `NegativeArraySizeException`
- `NullPointerException`
- `PINException`
- `RuntimeException`
- `SecurityException`
- `SystemException`
- `Throwable`
- `TransactionException`
- `UserException`

4.2.7 ECDSA signature length

ECDSA signatures consist of two integers: r , s where each number has the same length as the curve order. For the standard curves used in ICAO ePassport the order size is the same as the prime size (or key size), for example 256 or 384 bits. So the raw signature size is two times the key size, for example:

- 2×256 bits = 64 Bytes
- 2×384 bits = 96 Bytes

However, the Java Card API [21], states that ECDSA signatures are returned with the following ASN structure:

"The signature is encoded as an ASN.1 sequence of two INTEGER values, r and s , in that order:

SEQUENCE ::= r INTEGER, s INTEGER "

Furthermore, according to ASN rules, INTEGER is interpreted as signed integer, whereas the ECDSA signature components r, s are interpreted as unsigned integers. Therefore, if the most significant bit of numbers r, s is 1, then a leading zero byte must be added to the ASN encoding, so that the number remains positive. Below are three examples of ECDSA signatures using a Brainpool 384 bit curve (note the leading zero bytes):

```
<SEQUENCE> (0x30) [0x64]
{
<INTEGER r> (0x02) [0x30]
{ 29DF9061BEA2347234B707A453F0D10F87226B55DBBE671720AF88ACFE6AD53CC1B5F6E852E012FF7762FACAABA64123
}
<INTEGER s> (0x02) [0x30]
{ 7D5DD08428746D3DD6CD8507A179BFCC241724E7C043277A0DF74CA894C209864C029FA0C9B280AF18E74CEA0E010366
}
// sig1 byte length = 0x64+2 = 102 decimal
<SEQUENCE> (0x30) [0x65]
{
<INTEGER r> (0x02) [0x31]
{ 00823DC91827F6DCA1C20E2D1082EED9D2F8C2CBCD9D9EC0552475519068ABA27A6E993A64CCE07C268C8FDB22869015AB
}
<INTEGER s> (0x02) [0x30]
{ 42321C49800CB42E3A278B8A924E0700D9BB9800CCEED09CCDF557CA542901983EE4EBF2D151C5A8D3367711147A4B43
}
// sig2 byte length = 0x65+2 = 103 decimal
<SEQUENCE> (0x30) [0x66]
{
```

```
<INTEGER r> (0x02) [0x31]
{ 00849FBA754F38BD0CD824CAEBDDD8B55A6326831484849C433C73C18B175C894F930CC00E2ADC2A5D708EFC4BFFFC54F6
}
<INTEGER s> (0x02) [0x31]
{ 00C3D299A7D75DE7519963CB5076217A02489948154C6D37EAC2E1BEA7390DDED8CDE8CE1856F60737BF8A321638A4939D
}
// sig3 byte length = 0x66+2 = 104 decimal
```

As seen above, the length of a 384 bit signature can be 102, 103 or 104 Bytes, depending on the signature generated. Therefore the correct formula for the `signature.getLength` method should return the maximum signature length:

$\text{maxByteLength} = (384/8)*2 + 8 = 104 \text{ Bytes}$

This number can be used to allocate a buffer that can hold any signature on this curve (although in most cases the buffer will have one or two unused bytes). **Note:** Some curves have an order size that is larger than the prime

size. For those curves `r,s` is 1 Byte longer, so the signature length formula is:

$\text{maxByteLength} = 8 + (\text{Key.getSize}()/8+1)*2.$

4.2.8 Limitations

The following limitations apply to the Java Card API implemented in JCOP 4 P71.

- JCOP 4 P71 supports RSA public exponent lengths up to 4 bytes. RSA public exponent values of 0 and 1 are not allowed. It should also be smaller than `n` and less than `min(p-1, q-1)`.
- JCOP 4 P71 supports the value 1 for the cofactor `K` of an `ECKKey`. If the method `ECKKey.setK()` is invoked with values different from 1 then the key will become unusable. Using such unusable keys in `Signature.init(Key, byte)` for example throws a `CryptoException.ILLEGAL_VALUE`.
- The constructor `public KeyPair(byte algorithm, short keyLength)` is supported for the following key pairs:
 - RSA
 - RSA algorithm in its Chinese Remainder Theorem form (RSA-CRT),
 - EC key pair for EC operations over large prime fields (EC-FP).

Other key algorithms throw a `CryptoException.NO_SUCH_ALGORITHM`.

- For strict compliance to SP800-67 [5], the usage guidance specified in section 3.4 of [5] shall be applied:
“The security of TDEA is affected by the number of blocks processed with one key bundle. One key bundle shall not be used to apply cryptographic protection (e.g., encrypt) more than 220 64-bit data blocks. Note that this limitation applies to a key bundle with three unique keys (i.e., 3TDEA); the use of TDEA with only two unique keys (i.e., 2TDEA) shall not be used to apply cryptographic protection (see Section 3.1).”
When the FIPS mode of JCOP is activated see Section 8.3.4 - FIPS support, this restriction is enforced by JCOP.
- javacardx.biometry API is only supported if integrated Match-on-Card (MoC) support has been enabled (see Section 3.3 - Integrated MoC).
- The following optional Java Card API packages [21] are not supported:
 - java.rmi
 - javacardx.apdu.util
 - javacard.framework.service
 - javacardx.annotations
 - javacardx.external
 - javacardx.framework.math
 - javacardx.framework.string
 - javacardx.framework.tlv
 - javacardx.framework.util
 - javacardx.framework.util.intx
 - javacardx.security

4.3 Java Card Virtual Machine

The Virtual Machine conforms to [23]. All byte codes specified by Java Card are supported, including integer byte codes.

4.3.1 CAP file restrictions

4.3.1.1 CAP file loading

The Java Card Virtual Machine [23] and GlobalPlatform [15] specifications define a data format and loading process for Java applets contained in a Converted Applet format (CAP) file. JCOP 4 P71 supports only the standardized CAP file components and throws an `IOException.WRONG_DATA` if new or custom components are contained in a CAP file.

4.3.1.2 Applet AIDs

JCOP 4 P71 does not allow installation of applets whose Application Identifier (AID) is shorter than 5 bytes or longer than 16 Bytes.

4.4 Runtime Environment

4.4.1 Multiple logical channels

Logical channels can be used to provide more than one virtual communication path on one physical interface. On each logical channel a different applet can be selected at the same time or one applet can be selected multiple times on different logical channels (if the `MultiSelectable` interface is implemented). JCOP 4 P71 supports 4 logical channels (0, 1, 2, and 3) per physical interface.

4.4.2 Garbage collection

Garbage collection is an optional feature in the Java Card Virtual Machine [23] where it is referred to as object deletion. It can be used to free up persistent and transient memory from objects that are not referenced any more by any application. JCOP 4 P71 supports garbage collection. Garbage collection is triggered by

- invocation of `JCSystem.requestObjectDeletion()`,
- after the last LOAD command of package loading, or
- at the end of a successful DELETE (package or instance) command.

4.4.3 Transaction support

JCOP 4 P71 supports transactions as defined in the Java Card Runtime Environment specification [22].

4.4.3.1 Transaction abort

According to the RE specification ([22], chapter 7.6.3) JCOP 4 P71 locks the card session in case an ongoing transaction is aborted after objects have been created within this transaction. Transactions should only be programmatically aborted in rare circumstances, for example when the application's state is in danger of being compromised. The base system then ensures complete safety and security properties by forcing a tear/reset.

4.4.3.2 Transaction buffer size

The total size of the transaction buffer can be requested by invoking `JCSystem.getMaxCommitCapacity()`, the currently remaining free size can be requested by invoking `JCSystem.getUnusedCommitCapacity()`.

Note: Due to internal data structures the effective usable size depends on the operations within a transaction.

4.4.4 Card termination behavior

A call to the GlobalPlatform API method `GPSystem.terminateCard()` sets the card life cycle to *TERMINATED* if the calling applet has the Card Terminate privilege. The processing of the applet is continued. Any further operation which is not allowed in *TERMINATED* state will fail.

The card life cycle can also be set to *TERMINATED* using the GlobalPlatform SET STATUS command or implicitly due to detected security attacks to the product.

4.4.5 Limitations

The following limitations apply to the runtime environment:

- JCOP 4 P71 dynamically allocates transient memory for the management of open logical channels. The basic channel '00' is opened implicitly when the card is powered on. Memory for additional logical channels is allocated each time an additional logical channel is opened.

A new logical channel can be opened either explicitly by a MANAGE CHANNEL command or implicitly by a SELECT (by name) command on a closed logical channel. If a new logical channel shall be opened and there is not enough free transient memory for the logical channel management then the error status word 6A84h (FILE_FULL) is returned. In such situations the logical channel will not be opened.

Transient memory which is allocated for a logical channel remains allocated for this particular logical channel for the rest of the card life time, regardless if the channel was closed in between or the card was reset.

4.5 GlobalPlatform 2.3

JCOP can be configured to comply with the following GlobalPlatform configurations:

- Mapping Guidelines [11]
- GlobalPlatform ID Configuration [12]
- GlobalPlatform Card Common Implementation Configuration [14]
- GlobalPlatform Financial Configuration [16]

The current configuration can be set via a STORE DATA command to the ISD provided the Config Module has not been deleted. In addition, configuration is performed as outlined in Section 8.3.5.

JCOP supports the secure channel protocols SCP01, SCP02 and SCP03 as defined in GlobalPlatform 2.3 [15]. For limitations of JCOP 4 P71 see Section 4.5.9 - Limitations.

4.5.1 Framework

JCOP supports the following features of the GlobalPlatform Card Specification 2.3:

- Confidential card content management
- Issuer Security Domain
- Supplementary Security Domain
- Creation, deletion and blocking of Supplementary Security Domain (SSD)s
- Loading and installing applets
- Applet extradition to SSDs
- Applet selection
- All applet and Card Life Cycles and transitions between them
- Personalization through STORE DATA and PUT KEY
- Authorized management
- Delegated Management
 - Delegated Management Token verification with RSA scheme (Variant 1) specified in chapter C.4 of [15]. The verification key must be a 1024 bit RSA public key.
 - Delegated Management Token verification with RSA scheme (Variant 2) specified in chapter C.4 of [15]. The verification key must be a 2048 bit RSA public key.
 - Delegated Management Token verification with AES scheme (128, 192 and 256 bit keys) as specified in chapter C.4 of [15].
- Delegated Management Token verification with ECC specified in chapter 4.1 of [15] The verification key must be a 256 bit ECC public key.
- Data Authentication Pattern (DAP) and Mandated DAP
 - DAP and Mandated DAP verification with RSA scheme (Variant 1) specified in chapter C.6.1 of [15]. The verification key must be a 1024 bit RSA public key.
 - DAP and Mandated DAP verification with RSA scheme (Variant 2) specified in chapter C.6.1 of [15]. The verification key must be a 2048 bit RSA public key.
 - DAP and Mandated DAP verification with AES scheme (128, 192 and 256 bit keys) as specified in chapter C.4 of [15].

- DAP and Mandated DAP verification with ECC specified in chapter 4.1 of [15]. The token verification key must be a 256 bit ECC public key.
- Load File Data Block Hash computation using SHA-256 or SHA-1 algorithm.

4.5.2 APDUs

The GlobalPlatform APDU commands supported by JCOP are listed below. Refer to the GlobalPlatform standard [15] for more details and see Section 4.5.9 - Limitations for limitations on these commands.

- DELETE
- GET STATUS
- GET DATA
- INSTALL[for load]
- INSTALL[for install]
- INSTALL[for make selectable]
- INSTALL[for install and make selectable]
- INSTALL[for personalization]
- INSTALL[for extradition]
- INSTALL[for registry update]

Note: This command supports some options for the restrict parameter, tag 'D9'. For possible values for xx, see Table 4.4.

Tab. 4.4: Supported values for tag 'D9'

Bit 7	6	5	4	3	2	1	0	Definition
					1			Install. As per GP 2.3 [15], Table 11-53.
						1		Load. As per GP 2.3.
							1	Delete. As per GP 2.3.
Unused bits are set to '0'.								

- LOAD
- MANAGE CHANNEL
- PUT KEY

Note: A PUT KEY command to update the default key set for an ISD or SSD will impact performance when either the key size or algorithm is changed. This is a one-time effect.

- SELECT
- SET STATUS
- STORE DATA
- INITIALIZE UPDATE
- EXTERNAL AUTHENTICATE

4.5.3 Command processing

4.5.3.1 Destination application

All commands are sent to a destination application in the card. This destination application can be either the ISD, a Security Domain (SD) or an applet. The destination application must be selected before a command can be sent to it. The ISD is available after the Card Manager has been created in the OS initialization phase.

4.5.3.2 Default selected application

In JCOP 4 P71 the ISD initially has the Default Selected privilege. Therefore any command is sent after a reset by default to the ISD unless another application is selected.

4.5.3.3 Command chaining

JCOP supports a maximum of two commands in the chaining sequence. The APDU buffer must be large enough to contain the non-segmented command (see Section [8.3.3.5 - APDU_BUFFER_SIZE](#) for configuring the APDU buffer size). After command chaining consistency has been validated, APDU processing is performed on the reconstructed (non-segmented) command.

4.5.4 API packages

The following table lists the supported methods of the GlobalPlatform API 2.3. APIs not listed below have no functionality in JCOP 4 P71 or throw an exception.

Tab. 4.5: Supported APIs of package org.globalplatform

Class/interface	Method	Comment
Application	processData()	
CVM	isActive() isSubmitted() isVerified() isBlocked() getTriesRemaining() update() resetState() blockState() resetAndUnblockState() setTryLimit() verify()	
GPRegistryEntry	getAID() getPrivileges() getState() isAssociated() isPrivileged() setState()	
GPSystem	getCardContentState() getCardState() getCVM() getSecureChannel() lockCard() getPlatformId() getRegistryEntry() setATRHistBytes() setCardContentState() terminateCard()	
Personalization	processData()	
SecureChannel	decryptData() getSecurityLevel() processSecurity() resetSecurity() unwrap() wrap()	
SecureChannelx	setSecurityLevel()	

The functionality of the following interfaces from [15] is not implemented by any class:

- Authority
- HTTPAdministration
- HTTPReportListener
- SecureChannelx2

The deprecated OpenPlatform API package (`visa.openplatform`) is not supported by JCOP 4 P71.

4.5.5 Issuer Security Domain (ISD)

The ISD has the following default AID:

A000000151000000h

This AID can be changed with a STORE DATA command (see [5.1.2.2 - Change ISD AID](#)).

4.5.6 Supplementary Security Domain (SSD)

JCOP 4 P71 is delivered with a Security Domain package:

- AID of Executable Load File is:
A0000001515350h
- AID of Executable Module is:
A000000151535041h

The Executable Module can be used to instantiate a SSD.

Note: The GlobalPlatform Common Implementation Configuration document [14], Section 3.3.1.3 “Security Domain Install Parameters”, states that: *“tag ‘81’ may be omitted only if the card supports only a single Secure Channel Protocol (SCP) (that is, an obvious default value)”*

GlobalPlatform does not enforce a specific policy when the tag ‘81’ is not present. On JCOP tag ‘81’ may be omitted only if the card supports a single Secure Channel Protocol (SCP) or a specific policy has been defined by the Card Issuer. This latter case is in the hands of the Card Issuer and out of scope of this document.

4.5.7 Secure channel protocols

JCOP 4 P71 supports the SCP protocols SCP01, SCP02 and SCP03 with different protocol options as listed hereafter. The active SCP protocols can be configured during OS initialization, see [8.3.5.1 - SCP_ENABLE](#) for further details.

BEGIN RMAC SESSION and END RMAC SESSION APDUs are not supported.

- Secure Channel Protocol '01' (SCP01) with option i='05': Initiation mode explicit, C-MAC on modified APDU, ICV set to zero, no ICV encryption, 3 Secure Channel Keys.
- Secure Channel Protocol '02' (SCP02) with option i='75', i='55', i='45', i='35', i='15' or i='05':

Tab. 4.6: Values of Parameter "i" for SCP02

Bit 7	6	5	4	3	2	1	0	Definition
-	-	-	-	-	-	-	1	3 Secure channel keys
-	-	-	-	-	-	-	0	1 Secure channel base key
-	-	-	-	-	-	1	-	C-MAC on unmodified APDU
-	-	-	-	-	-	0	-	C-MAC on modified APDU
-	-	-	-	-	1	-	-	Initiation mode explicit
-	-	-	-	-	0	-	-	Initiation mode implicit
-	-	-	-	1	-	-	-	Initial Chaining Vector (ICV) set to MAC over AID
-	-	-	-	0	-	-	-	ICV set to zero
-	-	-	1	-	-	-	-	ICV encryption for C-MAC session
-	-	-	0	-	-	-	-	No ICV encryption
-	-	1	-	-	-	-	-	R-MAC support
-	-	0	-	-	-	-	-	No R-MAC support
-	1	-	-	-	-	-	-	Well-known pseudo-random algorithm (card challenge)
-	0	-	-	-	-	-	-	Unspecified card challenge generation method

Unused bits are set to '0'.

- Secure Channel Protocol '03' (SCP03) with option i='70', i='60', i='20', i='10' or i='00'. For a more detailed description see GlobalPlatform Card Specification [15].

Tab. 4.7: Values of Parameter "i" for SCP03

Bit 7	6	5	4	3	2	1	0	Definition
-	-	-	-	-	-	-	-	RFU
-	-	-	0	-	-	-	-	Random card challenge
-	-	-	1	-	-	-	-	Pseudo-random card challenge
-	0	0	-	-	-	-	-	No R-MAC/R-ENCRYPTION support
-	0	1	-	-	-	-	-	R-MAC support/no R-ENCRYPTION support
-	1	1	-	-	-	-	-	R-MAC and R-ENCRYPTION support
-	-	-	-	-	-	-	-	Reserved

Unused bits are set to '0'.

4.5.7.1 Key diversification data

During explicit initiation of a Secure Channel between the card and the host, an INITIALIZE UPDATE command is used. The first 10 bytes of the INITIALIZE UPDATE response message contain key diversification data. JCOP uses 'last two bytes of the SD SAID': '8 Byte serial number' as the initial default key derivation data for any SSD and the ISD. The serial number is chip specific and is assigned by NXP to each chip during manufacture.

The derivation data can be modified by sending the STORE DATA command with DGI tag '00CF' to the SD.

4.5.7.2 SD with support for multiple protocols

When a SD (including the ISD) is configured to support several secure channel protocols, the KVN is tied to a specific Secure Channel Protocol version. When only one protocol is allowed, the KVN is allowed to be in the range '01' ... '6F'. On multiple SCP protocol configurations, the following KVN mapping needs to be respected:

Tab. 4.8: KVN to SCP mappings

KVN range	SCP protocol
'10' ... '17'	SCP01
'20' ... '2F'	SCP02
'30' ... '3F'	SCP03

For GlobalPlatform details on SCP to KVN mapping, refer to Table 4.1 "Usage of Key Version Numbers for Secure Channel Protocols", chapter 4, GlobalPlatform CIC [14].

4.5.8 Configuration

GlobalPlatform feature support is configured using a STORE DATA command to the Config Module. The configuration is defined on 2 bytes encapsulated inside a Tag Length Value (TLV) with tag 0x200E (80E2880008DF2B05200E02xxxx). The detail of the 2 configuration bytes is outlined in Table 4.5.8.

Tab. 4.9: GlobalPlatform STORE DATA bit table

Bit 15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Definition
-	-	-	-	-	-	-	-	-	-	-	-	-	-	0	0	No SSD support
-	-	-	-	-	-	-	-	-	-	-	-	-	-	0	1	SSD supported but no DAP
-	-	-	-	-	-	-	-	-	-	-	-	-	-	1	0	SSDs but single SSD with mandated DAP
-	-	-	-	-	-	-	-	-	-	-	-	-	-	1	1	SSDs multiple SSD with mandated DAP

Continued on the next page.

Tab. 4.9 – Continued from the previous page.

Bit 15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Definition
-	-	-	-	-	-	-	-	-	-	-	-	-	0	-	-	No authorized management for SSDs
-	-	-	-	-	-	-	-	-	-	-	-	-	1	-	-	Authorized Management for SSDs
-	-	-	-	-	-	-	-	-	-	-	-	0	-	-	-	Delegated management not supported
-	-	-	-	-	-	-	-	-	-	-	-	1	-	-	-	Delegated management supported
-	-	-	-	-	-	-	-	-	-	-	0	-	-	-	-	LFDBH SHA-1
-	-	-	-	-	-	-	-	-	-	-	1	-	-	-	-	LFDBH SHA-256
-	-	-	-	-	-	-	-	-	0	0	-	-	-	-	-	Ciphered data block not supported
-	-	-	-	-	-	-	-	-	0	1	-	-	-	-	-	DES ciphered data block not supported
-	-	-	-	-	-	-	-	-	1	0	-	-	-	-	-	AES ciphered data block supported
-	-	-	-	-	-	-	-	-	1	1	-	-	-	-	-	DES and AES data block supported
-	-	-	-	-	-	-	-	0	-	-	-	-	-	-	-	DM and DAP key replacement not supported
-	-	-	-	-	-	-	-	1	-	-	-	-	-	-	-	DM and DAP key replacement supported
-	-	-	-	-	-	-	0	-	-	-	-	-	-	-	-	Strict mapping guidelines disabled
-	-	-	-	-	-	-	1	-	-	-	-	-	-	-	-	Strict mapping guidelines enabled ¹
-	-	-	-	-	-	0	-	-	-	-	-	-	-	-	-	ID config app-specific parameter filter disabled
-	-	-	-	-	-	1	-	-	-	-	-	-	-	-	-	ID config app-specific parameter filter enabled ²
-	-	-	-	0	0	-	-	-	-	-	-	-	-	-	-	Card recognition data: ID configuration
-	-	-	-	0	1	-	-	-	-	-	-	-	-	-	-	Card recognition data: CI configuration
-	-	-	-	1	0	-	-	-	-	-	-	-	-	-	-	Card recognition data: MG configuration

¹ This option is enabled when the following apply:

- SSD application specification parameters C90145 is accepted.
- Bits 4 and 5 of the STORE DATA P1 has not been verified (that is, they are restricted to 00 or 01).
- Final application privilege will remain with the ISD.

² If this option is enabled then if the value specified for the application-specific install parameters

tags '82', '83' or '87' is not compliant with the GP Config Spec [12]

tags '82', '83', then error status word 0x6A80 is returned.

Unused bits are set to '0'.

4.5.9 Limitations

JCOP has the following limitations:

- General limitations
 - An ongoing applet loading process (started with an INSTALL[for load] command followed by subsequent LOAD commands) will get terminated with 6985h (CONDITIONS_NOT_SATISFIED) in case another command is sent on the same logical channel before this process is finished.

- GET STATUS does not support Tag Lists (that is, GET STATUS tag 'C5' is not supported).
 - JCOP 4 P71 does not automatically delete temporary objects that may be created during applet installation. The applet developer can request that temporary objects are removed by calling the `JCSystem.requestObjectDeletion` method.
- Communication
 - JCOP does not support Implicit Secure Channel initiation.
 - The interfaces `Authority`, `HTTPAdministration`, `HTTPReportListener`, `SecureChannelx` and `SecureChannelx2` of GlobalPlatform Card Specification [15] are not implemented.
 - Card management
 - The combination of the [for load], [for install] and [for make selectable] options of an INSTALL command is not supported.
 - The behavior of the STORE DATA command is defined in GlobalPlatform Card Specification [15], chapter 11.11. JCOP 4 P71 always uses the option of Data Grouping Identifier (DGI) in order to increase the personalization efficiency (P1 is ignored except b8). Certain tests may try to exercise simple TLV updates with the STORE DATA command. Such tests would fail. They need to be omitted or recoded using DGIs.
 - The GlobalPlatform specification gives no indication on how to handle commands which contain the same tag more than once. JCOP 4 P71 handles such cases as follows:
 - * The DELETE command throws an exception 6A80h (WRONG_DATA) in case Tag '4F' (AID) is present more than once.
 - * All other commands use the value of the last occurrence of the Tag in the command data field.
 - A "Transaction Buffer Full" event during INSTALL for INSTALL AND MAKE SELECTABLE or INSTALL for INSTALL will mute the card and not return with a status word.
 - If an applet creates an object from a module class, it can only delete the module after setting the object reference to `null` and running a garbage collection.
 - DELETE [key] needs the same privileges as DELETE [card content], that is AM privilege or valid DM token.
 - Global services
 - JCOP 4 P71 does not support Global Services.
 - * The methods `registerService` and `deregisterService` of `GRegistryEntry` throw 6A81h (FUNC_NOT_SUPPORTED).
 - * The method `getService` of `GPSystem` returns `null`.

- Logical channels

The API methods `APDU.getCLAChannel` and `JCSystem.getAssignedChannel` interpret the information contained in the CLA byte located in the APDU buffer. If the content of the APDU buffer is modified before calling these methods then the method may return the wrong channel number.

4.6 Communications

JCOP 4 P71 supports the ISO/IEC 7816-3 T=0 and T=1 in both direct and inverse convention and the ISO/IEC 14443 Type A protocol.

The configuration of the Answer to Reset ISO-7816 (ATR), the Answer to Select ISO-14443 (ATS) and protocol details can be modified during the OS initialization phase (see Section 7 - [Pre-personalization \(OS initialization\)](#)).

4.6.1 Contactless interface

For contactless communication the parameters Answer To Request, Type A (ATQA), Select Acknowledge (SAK), Unique Identifier (UID) and ATS (interface bytes and historical characters) can be changed by a Java Card API and during OS initialization (see Section 8.3.1 - [Contactless communication](#)).

The contactless interface is available on all dual interface Smart Card controller ICs with JCOP 4 P71. The following communication speeds are supported:

- 106 kbit/s (default)
- 212 kbit/s
- 424 kbit/s
- 848 kbit/s

Very High Baud Rate (VHBR) is supported for PICC to PCD up to 3.2MBit/sec. (It is controlled by a configuration item, see Section 8.3.3.7 - [VHBR_ENABLED](#).)

4.6.2 Contact-based interface

The ATR can be configured during the OS initialization for both, the cold and warm reset. The ATR after a cold reset is sent by the card once on the first start after power is supplied, the warm ATR is sent by the card on all following resets, as long as the card stays powered.

For configuration details see Section 8.3.2 - [Contact-based communication](#).

JCOP further implements a Java Card API to modify the historical characters.

Tab. 4.10: Typical supported configurations and corresponding bit rates

ATR Parameters		CLK and I/O interface			Remark
TA1 (Fi,Di)	Fi/Di	CLKs/etu	CLK [MHz]	Bit rate [bits/s]	
'01' or '11'	372/1	372	3.5712	9 600	Default setting
'02' or '12'	372/2	186	3.5712	19 200	
'03' or '13'	372/4	93	3.5712	38 400	
'08' or '18'	372/12	31	3.5712	115 200	
'91'	512/1	512	4.9152	9 600	
'92'	512/2	256	4.9152	19 200	
'93'	512/4	128	4.9152	38 400	
'94'	512/8	64	4.9152	76 800	
'95'	512/16	32	4.9152	115 600	
'96'	512/32	16	4.9152	307 200	

4.7 WTX configurations

The Waiting Time Extension (WTX) bytes indicate to the terminal that the card is still performing operations and the terminal shall wait until the ongoing operation has finished. JCOP 4 P71 automatically sends the WTX bytes after a configurable time, depending on the requirements of the customer.

Note: The API `APDU.waitExtension()` has no effect, sending the WTX is implicitly sent by the platform.

The interval in which the WTX is sent is configurable for each interface.

4.7.1 Protocol T=0

The waiting time for T=0 protocol is calculated as defined in ISO/IEC 7816-3 [17]:

$$WT = WI \cdot 960 \cdot \frac{F_i}{f}$$

Tab. 4.11: Formula symbols for protocol T=0

Symbol	Description	Default Value
WT	Waiting time [s]	
WI	Waiting time integer	$WI = 10$
	$1 \leq WI \leq 255$	

Continued on the next page.

Tab. 4.11 – Continued from the previous page.

Symbol	Description	Default Value
F_i	Clock rate conversion integer $372 \leq F_i \leq 2048$	$F_i = 372$
f	Frequency value of the clock signal [MHz] $1 \text{ MHz} \leq f \leq 8 \text{ MHz}$	$f = 3.57 \text{ MHz}$

The waiting time can be set during the OS initialization phase separately for the start after a cold and warm reset.

4.7.2 Protocol T=CL

The frame waiting time for T=CL protocol is calculated as defined in ISO/IEC 14443 [1]:

$$FWT = \frac{256 \cdot 16}{f_c} \cdot 2^{FWI}$$

Tab. 4.12: Formula symbols for protocol T=CL

Symbol	Description	Default Value
FWT	Frame waiting time [s]	
FWI	Frame waiting time integer $1 \leq FWI \leq 14$	$FWI = 7$
f_c	Frequency value of the clock signal [MHz] $f_{c_{min}} = 13.553 \text{ MHz} \leq f_c \leq 13.567 \text{ MHz} = f_{c_{max}}$	$f_c = 13.567 \text{ MHz}$

The FWI is defined in the upper nibble of the TB1 byte of the ATS, see also Section 8.3.1.3 - [TCL_ATS_IF](#) for details.

4.7.2.1 Limitations

The BWT is currently a hard coded value in the OS and can not be set with the ATR. The BWT is set to 1 second.

5 Proprietary features and platform-dependent behavior

This chapter describes features of JCOP 4 P71 which extend the functionality of the underlying Java Card and GlobalPlatform functionality.

5.1 Additional APDUs

JCOP 4 P71 supports APDUs that extend the functionality of the underlying GlobalPlatform standard. These APDUs shall be sent only to the ISD unless otherwise stated. In particular, the attack detection mechanism may be triggered if an unsupported command is sent during the OS initialization phase.

Please see Section 4.5.3 - [Command processing](#) for a description of the command processing within JCOP 4 P71.

The following table gives an overview of all additional commands and to which target application they shall be sent.

Group	Command	Target	Authentication
Identification	GET DATA IDENTIFY	ISD	Not required
	Get CPLC data	ISD	Not required
	Get platform ID	ISD	Not required
Administration	Set SCP default key version	SD	Required ¹
	Change ISD AID	ISD	Required
	Disable ISD	ISD	Required
	Reset Attack Counter	ISD	Required
Card information	Get available memory	ISD	Not required
	Get extended card resources	ISD	Not required
	Read attack counter log	ISD	Not required
	Set garbage collection behavior	ISD	Not required

¹ Authentication to the ISD or to the SD whose keys shall be changed.

For a list of commands supported by the Config Module, see Section 7.4.2 - [OS initialization command processing](#).

5.1.1 Card identification

5.1.1.1 GET DATA IDENTIFY

This command returns the card identification data. This data makes it possible to unambiguously identify the content in Read Only Memory (ROM), NVM and loaded patches (if any). The APDU can be sent without prior authentication.

The availability of the IDENTIFY command can be restricted during the OS initialization. If the command is disabled, 6985h (CONDITIONS_NOT_SATISFIED) is returned.

The command GET DATA (IDENTIFY) shall be formatted as follows:

Tab. 5.1: GET DATA (IDENTIFY) command format and parameter settings

Code	Value	Parameter settings
CLA	'80'/'00'	GlobalPlatform/ISO/IEC
INS	'CA'	GET DATA (IDENTIFY)
P1	'00'	High order tag value
P2	'FE'	Low order tag value - proprietary data
Lc	'02'	Length of data field
Data	'DF28'	Card identification data
Le	'00'	Length of response data

GET DATA (IDENTIFY) response data is formatted as follows:

Tab. 5.2: Response data of GET DATA (IDENTIFY)

Field	Length	Content	Remark
Tag value - proprietary data	'01'	'FE'	Only present if class byte is '80'
Length of following data	'01'	'49'	Only present if class byte is '80'
Tag card identification data	'02'	'DF28'	Only present if class byte is '80'
Length of card identification data	'01'	'46'	Only present if class byte is '80'
Tag configuration ID	'01'	'01'	Identifies the configuration content
Length configuration ID	'01'	'0C'	
Configuration ID	'0C'	var	
Tag patch ID	'01'	'02'	Identifies the patch level
Length patch ID	'01'	'08'	
Patch ID	'08'	var	
Tag platform build ID ¹	'01'	'03'	Identifies the JCOP platform
Length platform build ID	'01'	'18'	
Platform build ID	'18'	var	
Tag FIPS mode	'01'	'05 ² '	FIPS mode active
Length FIPS mode	'01'	'01'	'00' - FIPS mode not active, '01' - FIPS mode active
FIPS mode	'01'	var	
Tag modules enabled	'01'	'06 ³ '	Lists enabled and disabled modules
Length modules enabled	'01'	'02'	Big endian format
Bit mask of enabled modules	'02'	var	See Table 5.3
Tag pre-perso state	'01'	'07'	Lists pre-perso state
Length pre-perso state	'01'	'01'	bit0 = 1 = config module available, bit1 = 1 = transport state is active.
Bit mask of pre-perso state	'01'	var	

Continued on the next page.

Tab. 5.2 – Continued from the previous page.

Field	Length	Content	Remark
			Unused bits are set to '0'.
Tag ROM ID	'01'	'08'	Identifies the ROM content
Length ROM ID	'01'	'08'	
ROM ID	'08'	var	
Status Word (SW)	'02'	9000h	Normal ending

Tab. 5.3: Supported values for tag Modules enabled

Bit 15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Definition
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	1	eGov accelerator module
-	-	-	-	-	-	-	-	-	-	-	-	-	-	1	-	Secure Box module
-	-	-	-	-	-	-	-	-	-	-	-	-	1	-	-	RSA key generation module
-	-	-	-	-	-	-	-	-	-	-	-	1	-	-	-	Config Module
-	-	-	-	-	-	-	-	-	-	-	1	-	-	-	-	ECC module
-	-	-	-	-	-	-	-	-	1	-	-	-	-	-	-	FIPS module
-	-	-	-	-	-	-	-	-	1	-	-	-	-	-	-	Korean Seed module
-	-	-	-	-	-	-	-	1	-	-	-	-	-	-	-	Opacity module
-	-	-	-	-	1	-	-	-	-	-	-	-	-	-	-	PACE IM module

Unused bits are set to '0'.

For more information about the modules, see Section [5.2 - OS modules](#).

- Configuration ID

The configuration ID consists of the following data items:

- Byte 0: RFU,
- Bytes 1-3: Configuration ID determined by the Fab Key help desk,
- Bytes 4-11: NVM background identifier.

The NVM background identifier is pre-calculated over the following items outside the card:

- * Applets pre-loaded in NVM, this includes also the Config Module and any of its embedded keys.

¹The platform build ID is made up of the platform ID (0x10 Bytes) and the platform build fingerprint (0x08 Bytes).

²Tag '04' is intentionally omitted as it is not applicable to this product.

³Tag '06' is only included in the response if there has been a successful authentication with the card manager.

- * Default card configuration, see Section [8.3 - Configuration options](#).

The NVM background identifier will never change, even if any of the above mentioned items are modified during the OS initialization.

- Patch ID

The patch ID indicates the patch configuration/revision if one is installed. If no patch is installed in the card, then '00'... '00' is returned as patch ID.

- Platform build ID:

- Bytes 1-16: The ASCII representation of the platform build ID.
- Bytes 17-24: Fingerprint identifying the JCOP build (ROM and Flash, native, and Java).

The fingerprint will never change, even if any of the modules are deleted during OS (runtime) pre-personalization

- ROM ID

Fingerprint identifying ROM content.

The GET DATA (IDENTIFY) command may return the following error status word:

Tab. 5.4: Error status words returned by GET DATA (IDENTIFY)

SW	Meaning	Condition
6985h	CONDITIONS_NOT_SATISFIED	Availability of the command was restricted during OS initialization
6A80h	WRONG_DATA	Length of the data field not equal to 2 or the proprietary data tag is not recognized

5.1.1.2 Get CPLC data

This command returns the Card Production Life Cycle (CPLC) data. The APDU can be sent without prior authentication.

The availability of the Get CPLC data command can be restricted during the OS initialization see Section [8.3.5.3 - PROPRIETARY_GET_DATA_DISABLED](#). If the command is disabled, 6985h (CONDITIONS_NOT_SATISFIED) is returned.

The command GET DATA (CPLC data) shall be formatted as follows:

Tab. 5.5: GET DATA (CPLC data) command format and parameter settings

Code	Value	Parameter settings
CLA	'80'/'00'	GlobalPlatform/ISO/IEC
INS	'CA'	GET DATA (CPLC data)
P1	'9F'	High order tag value - CPLC data
P2	'7F'	Low order tag value - CPLC data
Le	'00'	Length of response data

GET DATA (CPLC data) response data is formatted as follows:

Tab. 5.6: Response data of GET DATA (CPLC data)

Field	Length	Content	Remark
Tag CPLC data	'02'	'9F7F'	Only present if the class byte is '80'
Length CPLC data	'01'	'2A'	Only present if the class byte is '80'
CPLC data	'2A'	CPLC data	For more information, see Section 8.1
SW	'02'	9000h	Normal ending

The GET DATA (CPLC data) command may return the following error status word:

Tab. 5.7: Error status words returned by GET DATA (CPLC data)

SW	Meaning	Condition
6985h	CONDITIONS_NOT_SATISFIED	Availability of the command was restricted during OS initialization

5.1.1.3 Get platform ID

This command returns the platform identification data. This data consists of parts of the CPLC data (see [Section 8.1 - Card Production Life Cycle \(CPLC\)](#)) and a random number which is unique for each card. For products which have not been produced by Trust Provisioning, the random number is replaced by '00'... '00'. The APDU can be sent without prior authentication.

The availability of this command can be restricted during the OS initialization see [Section 8.3.5.3 - PROPRIETARY_GET_DATA_DISABLED](#). If the command is disabled, 6985h (CONDITIONS_NOT_SATISFIED) is returned.

The command GET DATA (Platform ID) shall be formatted as follows:

Tab. 5.8: GET DATA (Platform ID) command format and parameter settings

Code	Value	Parameter settings
CLA	'80'/'00'	GlobalPlatform/ISO/IEC
INS	'CA'	GET DATA (Platform ID)
P1	'00'	High order tag value
P2	'FE'	Low order tag value - proprietary data
Lc	'02'	Length of data field
Data	'DF20'	Platform ID
Le	'00'	Length of response data

GET DATA (Platform ID) response data is formatted as follows:

Tab. 5.9: Response data of GET DATA (Platform ID)

Field	Length	Content	Remark
Tag value - proprietary data	'01'	'FE'	Only present if class byte is '80'
Length of following data	'01'	'1B'	Only present if class byte is '80'
Tag platform ID	'02'	'DF20'	Only present if class byte is '80'
Length platform ID	'01'	'18'	Only present if class byte is '80'
IC OS initializer	'02'	WX	See Section 8.1.
IC OS initialization date	'02'	YN	See Section 8.1.
IC OS initialization equipment ID	'04'	NNNN	See Section 8.1.
IC fabrication date	'02'	tt	See Section 8.1.
IC serial number	'04'	nnnb	See Section 8.1.
IC batch identifier	'02'	bb	See Section 8.1.
Random number or '00'... '00'	'08'	Random data or '00'... '00'	
SW	'02'	9000h	Normal ending

The GET DATA (Platform ID) command may return the following error status word:

Tab. 5.10: Error status words returned by GET DATA (Platform ID)

SW	Meaning	Condition
6985h	CONDITIONS_NOT_SATISFIED	Availability of the command was restricted during OS initialization

Continued on the next page.

Tab. 5.10 – Continued from the previous page.

SW	Meaning	Condition
6A80h	WRONG_DATA	Length of the data field not equal to 2 or the proprietary data tag is not recognized

5.1.2 ISD administration

5.1.2.1 Set SCP default key version

The default key version of the currently selected SD for the SCP protocol can be changed by sending the following STORE DATA command to a SD with SCP support. The default key version is used for the SCP protocol if P1 of the INITIALIZE UPDATE command indicates “first available key”.

The command STORE DATA (SCP default key version) shall be formatted as follows:

Tab. 5.11: STORE DATA (SCP default key version) command format and parameter settings

Code	Value	Parameter settings
CLA	'80'	GlobalPlatform
INS	'E2'	STORE DATA (SCP default key version)
P1	'88'	Last block, DGI format
P2	'00'	First block
Lc	'04'	Length of data field
Data	'7F0D' '01' kvn	Data field

with:

- kvn = new key version number.

The STORE DATA (SCP default key version) command may return the following error status word:

Tab. 5.12: Error status words returned by STORE DATA (SCP default key version)

SW	Meaning	Condition
6A80h	WRONG_DATA	Key version indicated by kvn does not exist

5.1.2.2 Change ISD AID

The AID of the ISD can be changed sending the following STORE DATA command to the ISD:

The command STORE DATA (Change ISD AID) shall be formatted as follows:

Tab. 5.13: STORE DATA (Change ISD AID) command format and parameter settings

Code	Value	Parameter settings
CLA	'80'	GlobalPlatform
INS	'E2'	STORE DATA (Change ISD AID)
P1	'88'	Last block, DGI format
P2	'00'	First block
Lc	'05' + AID length	Length of data field
Data	See below	Data field

STORE DATA (Change ISD AID) data field is formatted as follows:

Tab. 5.14: Data field of STORE DATA (Change ISD AID)

Field	Length	Content	Remark
DGI handle TLV tags	'02'	'0070'	
Length of following data	'01'	'02' + AID length	
Tag AID	'01'	'4F'	
Length of new ISD AID	'01'	AID length	
New ISD AID	AID length	new AID	

The STORE DATA (Change ISD AID) command may return the following error status word:

Tab. 5.15: Error status words returned by STORE DATA (Change ISD AID)

SW	Meaning	Condition
6A80h	WRONG_DATA	Length of the AID smaller than 5 bytes or longer than 16 bytes or new AID is already in use

5.1.2.3 Disable ISD

JCOP 4 P71 allows the ISD to be switched off to prevent future card administration. If the ISD is switched off and a Select command is sent to the card to select the ISD, the error status word 6A82h (FILE_NOT_FOUND) is returned.

The following preconditions shall be fulfilled before sending the switch off ISD command:

- The GlobalPlatform CARD_RESET privilege shall not be assigned to the ISD.
- The ISD shall be selected.

If the ISD is disabled, only following white listed commands will be accepted. For all other commands the status word 69FFh (ISD_LIMITED_ARC_EXPIRED) is returned.

- Get CPLC data (see Section 5.1.1.2 - [Get CPLC data](#)),
- Read attack counter log (see Section 5.1.3.3 - [Read attack counter log](#)).

The command STORE DATA (Disable ISD) shall be formatted as follows:

Tab. 5.16: STORE DATA (Disable ISD) command format and parameter settings

Code	Value	Parameter settings
CLA	'80'	GlobalPlatform
INS	'E2'	STORE DATA (Disable ISD)
P1	'88'	Last block, DGI format
P2	'00'	First block
Lc	'03'	Length of data field
Data	'DF5000'	Data field

The STORE DATA (Disable ISD) command may return the following error status word:

Tab. 5.17: Error status words returned by STORE DATA (Disable ISD)

SW	Meaning	Condition
6985h	CONDITIONS_NOT_SATISFIED	Privilege CARD_RESET still assigned to ISD or ISD not selected

5.1.2.4 Reset Attack Counter

JCOP 4 P71 allows the attack counter to be reset. Two mechanisms are implemented for this purpose

- Reset the Attack Counter after previous authentication to the ISD.
- Reset the Attack Counter using a signature based mechanism without previous authentication to the ISD.

Resetting the Attack Counter with previous authentication to the ISD only works once. Afterwards this functionality is disabled.

The command STORE DATA (Reset Attack counter with previous authentication to the ISD) shall be formatted as follows:

Tab. 5.18: STORE DATA (Reset Attack counter with previous authentication to the ISD) command format and parameter settings

Code	Value	Parameter settings
CLA	'80'	GlobalPlatform
INS	'E2'	STORE DATA (Reset Attack counter with previous authentication to the ISD)
P1	'88'	Last block, DGI format
P2	'00'	First block
Lc	'03'	Length of data field
Data	'DF2700'	Data field

Resetting the Attack Counter using signature-based authentication requires the direct interaction with NXP. The response of the following APDU needs to be provided to NXP for the unlock request.

The command STORE DATA (Reset Attack counter using signature) shall be formatted as follows:

Tab. 5.19: STORE DATA (Reset Attack counter using signature) command format and parameter settings

Code	Value	Parameter settings
CLA	'80'	GlobalPlatform
INS	'CA'	STORE DATA (Reset Attack counter using signature)
P1	'00'	
P2	'FE'	
Lc	'04'	Length of data field
Data	'DF270100'	Data field

5.1.3 Card information

5.1.3.1 Get available memory

The available transient and persistent memory can be retrieved with the following GET DATA APDU. The APDU can be sent without prior authentication to the ISD.

The availability of this command can be restricted during the OS initialization see Section 8.3.5.3 - [PROPRIETARY_GET_DATA_DISABLED](#).

The command GET DATA (Available memory) shall be formatted as follows:

Tab. 5.20: GET DATA (Available memory) command format and parameter settings

Code	Value	Parameter settings
CLA	'80'/'00'	GlobalPlatform/ISO/IEC
INS	'CA'	GET DATA (Available memory)
P1	'00'	High order tag value
P2	'FE'	Low order tag value - proprietary data
Lc	'02'	Length of data field
Data	'DF25'	Available memory
Le	'00'	Length of response data

GET DATA (Available memory) response data is formatted as follows:

Tab. 5.21: Response data of GET DATA (Available memory)

Field	Length	Content	Remark
Tag proprietary data	'01'	'FE'	Only present if class byte is '80'
Length proprietary data	'01'	'1B'	Only present if class byte is '80'
Tag available memory	'02'	'DF25'	Only present if class byte is '80'
Length available memory	'01'	'18'	Only present if class byte is '80'
Tag free transient memory (clear on deselect)	'01'	'02'	
Length of free transient memory (clear on deselect)	'01'	'04'	
Free transient memory (clear on deselect)	'04'	variable	
Tag free transient memory (clear on reset)	'01'	'01'	
Length of free transient memory (clear on reset)	'01'	'04'	
Free transient memory (clear on reset)	'04'	variable	
Tag free persistent memory	'01'	'00'	

Continued on the next page.

Tab. 5.21 – Continued from the previous page.

Field	Length	Content	Remark
Length of free persistent memory	'01'	'04'	
Free persistent memory	'04'	variable	
Tag number of free indices in the object table	'01'	'03'	
Length of number of free indices in the object table	'01'	'04'	
Number of free indices in the object table	'04'	variable	
SW	'02'	9000h	Normal ending

The reported size for free persistent memory consists of space available for object payload + free space for object headers (free indices). An index has the size of 16 Bytes. When no indices are left, the table will be increased by the OS implicitly.

Note: On JCOP 4 P71 v4.7 R1.01.4, the reported free transient memory is shown in 16 Byte granularity. The memory allocation is shown in Byte granularity.

Note: The instantiation of an object (for example *Signature*) might allocate implicitly several objects, which reduces the number of the free indices.

The GET DATA (Available memory) command may return the following error status word:

Tab. 5.22: Error status words returned by GET DATA (Available memory)

SW	Meaning	Condition
6E00h	CLA_NOT_SUPPORTED	CLA byte (excluding logical channel number) is not 00, 80, 84
6700h	WRONG_LENGTH	Length of the command data is not equal to 2
6A80h	WRONG_DATA	Length of the data field is less than 2 or the proprietary data tag is not recognized
6985h	CONDITIONS_NOT_SATISFIED	Availability of the command was restricted during OS initialization

5.1.3.2 Get extended card resources

Reads the extended card resources as defined in ETSI TS 102 226 [3].

The command GET DATA shall be formatted as follows:

Tab. 5.23: GET DATA command format and parameter settings

Code	Value	Parameter settings
CLA	'00'	ISO/IEC 7816

Continued on the next page.

Tab. 5.23 – Continued from the previous page.

Code	Value	Parameter settings
INS	'CA'	GET DATA
P1	'FF'	High order tag value - extended card resources
P2	'21'	Low order tag value - extended card resources
Le	'00'	Length of response data

GET DATA response data is formatted as follows:

Tab. 5.24: Response data of GET DATA

Field	Length	Content	Remark
Tag extended card resources	'02'	'FF21'	Only present if class byte is '80'
Length extended card resources	'01'	'10'	Only present if class byte is '80'
Tag number of installed applications	'01'	'81'	
Length of number of installed applications	'01'	'02'	
Number of installed applications	'02'	variable	
Tag free non volatile memory	'01'	'82'	
Length of free non volatile memory	'01'	'04'	
Free non volatile memory	'04'	variable	
Tag free volatile memory	'01'	'83'	
Length of free volatile memory	'01'	'04'	
Free volatile memory	'04'	variable	
SW	'02'	9000h	Normal ending

The GET DATA command may return the following error status word:

Tab. 5.25: Error status words returned by GET DATA

SW	Meaning	Condition
6E00h	CLA_NOT_SUPPORTED	CLA byte (excluding logical channel number) is not 0x80
6700h	WRONG_LENGTH	Length of the command data is not equal to 2
6A80h	WRONG_DATA	Length of the data field is less than 2 or the proprietary data tag is not recognized
6985h	CONDITIONS_NOT_SATISFIED	Availability of the command was restricted during OS initialization

5.1.3.3 Read attack counter log

Before reading the attack log with the GET DATA command below, the following SELECT command needs to be sent: '00A404000BD276000085304A434F800000'.

The response data contains version information used by NXP. Typical response data is '000C0C00000C0211020405009230000004F00083000680'.

The command returns the logging information of the Attack Counter (AC). For further information on the AC see Section 5.8 - [Attack detection](#). The AC logging information is encrypted by a key owned by NXP. The APDU can be sent at any time.

The command GET DATA shall be formatted as follows:

Tab. 5.26: GET DATA command format and parameter settings

Code	Value	Parameter settings
CLA	'80'	GlobalPlatform
INS	'CA'	GET DATA
P1	'00'	High order tag value
P2	'FE'	Low order tag value - proprietary data
Lc	'02'	Length of data field
Data	'DF26'	Attack counter log
Le	'00'	Length of response data

GET DATA response data is formatted as follows:

Tab. 5.27: Response data of GET DATA

Field	Length	Content	Remark
Tag proprietary data	'01'	'FE'	
Length proprietary data	'02'	'8194'	
Tag read AC log	'02'	'DF26'	
Length AC log	'02'	'8190'	
Encrypted logging data	'90'	variable	
SW	'02'	9000h	Normal ending

The GET DATA command may return the following error status word:

Tab. 5.28: Error status words returned by GET DATA

SW	Meaning	Condition
6E00h	CLA_NOT_SUPPORTED	CLA byte (excluding logical channel number) is not 0x00, 0x80, 0x84
6A80h	WRONG_DATA	Length of the data field not equal to 2 or the proprietary data tag is not recognized
6985h	CONDITIONS_NOT_SATISFIED	An error occurred when reading the AC log

5.1.3.4 Set garbage collection behavior

The garbage collection behavior can be altered during a card session as follows:

- Temporarily disable garbage collection for this card session.
- Re-enable garbage collection for this card session.
- Enforce a garbage collection regardless of whether it is temporarily disabled or not.

The availability of this command can be restricted during the OS initialization (see Section 7 - [Pre-personalization \(OS initialization\)](#)).

The command GET DATA (GC behavior) shall be formatted as follows:

Tab. 5.29: GET DATA (GC behavior) command format and parameter settings

Code	Value	Parameter settings
CLA	'80'	GlobalPlatform
INS	'CA'	GET DATA (GC behavior)
P1	'00'	High order tag value
P2	'FE'	Low order tag value - proprietary data
Lc	'04'	Length of data field
Data	'DF25'	Option Set GC behavior: Option 01=temporarily disable garbage for this card session. Option 02=re-enable garbage for this card session. Option 03=enforce a garbage collection regardless of whether it is temporarily disabled or not.
Le	'00'	Length of response data

GET DATA (GC behavior) response data is formatted as follows:

Tab. 5.30: Response data of GET DATA (GC behavior)

Field	Length	Content	Remark
Tag value - proprietary data	'01'	'FE'	
Length of following data	'01'	'03'	
Tag GET DATA (GC behavior)	'02'	'DF25'	
Length GC behavior	'01'	'00'	
SW	'02'	9000h	Normal ending

The GET DATA (GC behavior) command may return the following error status word:

Tab. 5.31: Error status words returned by GET DATA (GC behavior)

SW	Meaning	Condition
6E00h	CLA_NOT_SUPPORTED	CLA byte (excluding logical channel number) is not 0x80
6700h	WRONG_LENGTH	Length of the command data is not equal to 4
6A80h	WRONG_DATA	Length of the data field is less than 2 or the proprietary data tag is not recognized
6985h	CONDITIONS_NOT_SATISFIED	Availability of the command was restricted during OS initialization
6984h	DATA_INVALID	GC option is not valid

5.1.3.5 Verify package integrity

If a LFDBH is supplied when a package is loaded, the hash value is verified during the loading process but it also stored and can be checked at a later time using the following command.

The command GET DATA shall be formatted as follows:

Tab. 5.32: GET DATA command format and parameter settings

Code	Value	Parameter settings
CLA	'80'	GlobalPlatform
INS	'CA'	GET DATA
P1	'00'	High order tag value
P2	'FE'	Low order tag value - proprietary data
Lc	'X'	Length of data field
Data	'DFBA'xyyaaaaaaaaaazzhhhhhhhhhh	The data field is encoded described in Table 5.33

Tab. 5.33: Data field encoding

Length	Value	Description
xx	'01'	Length of the following data.
yy	'01'	Length of AID.
aaaaaaaa	Var.	AID of package/ELF to be checked.
zz	'01'	Length of LFDBH.
hhhhhhhhh	Var.	LFDBH.

The LFDBH supplied in this command is compared with the LFDBH that was stored when the corresponding package (i.e. package with AID supplied in the command data of this APDU) was loaded.

The GET DATA command may return the following error status word:

Tab. 5.34: Error status words returned by GET DATA

SW	Meaning	Condition
6A80h	WRONG_DATA	The LFDBHs do not match
6A88h		The supplied AID is not found or a LFDBH was not supplied when the package was loaded
9000h	NO_ERROR	Success

5.1.4 Retrieving the FIPS configuration and triggering self tests

If the FIPS module is present, the FIPS configuration can be retrieved with the following command. If FIPS mode is enabled (see Section 8.3.4 - [FIPS support](#)) then the self tests can be triggered on-demand by sending the following GET DATA APDU to the card manager.

The command GET DATA shall be formatted as follows:

Tab. 5.35: GET DATA command format and parameter settings

Code	Value	Parameter settings
CLA	'80'	GlobalPlatform
INS	'CA'	GET DATA
P1	'00'	High order tag value
P2	'FE'	Low order tag value - proprietary data
Lc	'04'	Length of data field
Data	'DF4B01XX'	Where xx can be:

Continued on the next page.

Tab. 5.35 – Continued from the previous page.

Code	Value	Parameter settings
		'10': Retrieve FIPS configuration. '20': Trigger FIPS self tests.

For the Retrieve FIPS configuration option, response data of 'FE 04 DF 4B 01 NN' is returned, where 'nn' is the value of the Federal Information Processing Standard (FIPS) configuration item (see Table 8.13 in Section 8.3.4).

For the Trigger FIPS self tests option, the self tests are executed upon receipt of this command. The card will mute if any self test fails. If all self tests pass then response data 'FE 04 DF 4B 01 20' is returned with status word 9000h (NO_ERROR). See Section 8.3.4 - FIPS support to configure which self tests are executed by this command. The GET DATA command may return the following error status word:

Tab. 5.36: Error status words returned by GET DATA

SW	Meaning	Condition
6984h	DATA_INVALID	FIPS mode is not enabled. Self tests cannot be executed.
9000h	NO_ERROR	Self test executed successfully.

5.2 OS modules

JCOP is designed to support various market segments with different requirements on the feature set. Table 3.1 groups the JCOP 4 P71 features into functional areas and add-ons. Most add-ons map onto technical modules. These modules are discussed in this section.

To allow a customer to use the NVM memory efficiently, functionality can be removed when not needed. The available features can be configured in the following states of the life cycle:

- During ordering of the product using the OEF (Order Entry Form) (installation and removal of modules).
- During the OS initialization phase using the GlobalPlatform DELETE command.
- Using bulk-update by exchanging the entire Flash image (installation and removal of modules).
- In the field by using the GlobalPlatform DELETE command.

Modules are represented within JCOP as packages in the PHEAP with fixed AIDs. Table 5.37 lists the functionality JCOP currently supports as modules and the module AIDs. (Table 3.2 shows the memory consumption by module. Table 5.3 shows the supported values for the Modules enabled tag.)

Tab. 5.37: Supported modules and their AIDs.

Module name	Module AID
Config Module	D276000085304A434F504D4F4406
ECC	D276000085304A434F504D4F4401
eGov accelerators	D276000085304A434F506E
FIPS	D276000085304A434F504D4F4407
Korean Seed	D276000085304A434F504D4F4402
RSA key generation	D276000085304A434F504D4F4400
PACE IM	D276000085304A434F504D4F440C
PIV	D276000085304A434F5076
Secure Box	D276000085304A434F505F

The sections that follow describe the modules, explain their purpose and reference the sections of the manual that discuss them in more detail.

5.2.1 Config Module

The Config Module is used for OS initialization. It is installed by default.

For more information, see Section [7.4 - Config Module](#).

5.2.2 eGov accelerators

This module is part of the SECID add-on.

The eGov accelerators module consists of a Java Card package, `com.nxp.id.jcopx.egovaccelerators`, that helps speed up the processing of secure messages and symmetric key objects. See Section [5.4.2 - JCOPX eGovAccelerators](#).

5.2.3 RSA key generation

This module implements the RSA key generation feature. For details on the cryptographic algorithms and key sizes supported by JCOP, see Section [3.4 - Cryptographic algorithms and key sizes](#).

5.2.4 ECC

This module implements all the Elliptic Curve Cryptography (ECC). For details on the cryptographic algorithms and key sizes supported by JCOP, see Section [3.4 - Cryptographic algorithms and key sizes](#).

ECC is included by default in a SECID configuration.

5.2.5 Korean Seed

This module implements the Korean Seed cryptographic algorithm. For details on the cryptographic algorithms and key sizes supported by JCOP, see Section [3.4 - Cryptographic algorithms and key sizes](#).

5.2.6 FIPS

This module is part of the FIPS add-on.

See Section [8.3.4.1 - FIPS_COMPLIANCE](#) for possible FIPS configuration values. See Section [5.7 - FIPS self-test](#) for more on the JCOP self-test conformance.

5.2.7 PIV (Opacity)

This module is part of the FIPS add-on.

The PIV module ensures that JCOP supports SP 800-73-4 [6] and the Personal Identity Verification (PIV) it defines. The feature is supported by the `com.nxp.id.jcopx.piv` API: see Section [5.4.6 - JCOPX PIV](#).

5.2.8 Secure Box

Secure Box provides an area for 3rd party native code to be added to the module in a securely encapsulated environment. It is supported by the `com.nxp.id.jcopx.securebox` API: see Section [5.4.9 - JCOPX SecureBox](#).

5.2.9 PACE IM

The Password Authenticated Connection Establishment (PACE) IM module provides integrated mapping functionality for the PACE protocol. If this functionality is required, speak to your NXP representative.

5.2.10 Module behavior

The list of available modules can be retrieved using the IDENTIFY command by matching the bits of tag 6 with the mask in Table [5.3](#). When the module is available, the functionality can be used. When a module is deleted or not installed, JCOP will cause an appropriate Java exception.

5.2.11 Limitations

The following limitations in the usage of modules need to be respected:

- Module AIDs will not be listed using the GlobalPlatform command GET STATUS. The list of available modules can be retrieved from tag '06' of the IDENTIFY command when sent within a secure channel.
- The deletion of a module will cause a garbage collection and compaction, equal to any other DELETE command.
- When a module is deleted and an applet is still using functionality of the module, the behavior is as follows:

- For modules that provide only a feature without defining objects, the DELETE operation will always succeed. When the applet is calling a function of the module again, an appropriate Java exception will be thrown. This is the behavior for the RSA Keygen module and the Config Module.
- For modules that provide objects to the applet, the DELETE operation will fail where an applet keeps a reference to an object provided by the module. This is the case for the eGov accelerator module (SMAccelerator objects), for the Korean Seed module and ECC module (for key objects), and for the Secure Box module (when a Secure Box application is still installed).

5.3 MIFARE

JCOP can be ordered with optional MIFARE Plus EV1 or MIFARE DESFire EV2 support. Both implementations rely on the presence of the Secure Box module. The MIFARE applet needs to be installed as the default selected application to function correctly.

The MIFARE applet uses, and provides functionality to, the following APDUs:

- `jcopx.rawcommand` (see Section 5.4.8 - JCOPX RawComm)
- `jcopx.memory` (for the remote interface functionality)
- `javacardx.external` (for MIFARE Classic use cases)
- `jcopx.mifaresupport_4` (see Section 5.4.5 - JCOPX MifareSupport.4)

MIFARE DESFire EV2 as well as MIFARE Plus EV1 can be used with ISO 7816 APDUs and the proprietary APDU interface. To allow the proprietary APDU interface to be used, the RAW command API is used. Due to the nature of this interface the JCOP communication behavior changes.

- During a MIFARE session, no other logical channels can be used.
- MIFARE will only work on the base logical channel.
- An ISO select APDU to an AID that is also installed on the card will be intercepted by JCOP and the Java Card application will be selected (Java Card applets have a higher priority).
- When another application is selected on a logical channel different to the base logical channel, MIFARE will not work.

The MIFARE applets are described a separate user manual. To configure the MIFARE applet to be used with the Remote interface, the config item MIFARE_APPLET_AID (see Section 8.3.6.4 - MIFARE_APPLET_AID).

5.4 JCOPX Java Card API extension

JCOP 4 P71 provides an extension for the Java Card API for special purposes and with additional security algorithms. This chapter describes these Java Card API extensions.

5.4.1 JCOPX Accelerator

The JCOP `accelerator` package contains a class for accelerating the loader service.

This class has no security impact and is not included in any certification process.

Tab. 5.38: APIs of package `com.nxp.id.jcopx.accelerator`

Class	Method	Reference
Accelerator	exchangeAPDU	5.4.1.1 - Class Accelerator

5.4.1.1 Class Accelerator

```
public class Accelerator extends Object
```

This class is part of the `com.nxp.id.jcopx.accelerator` package.

This class provides a facility for any Java Card application to forward APDU commands to OPEN. Such APDU commands are executed/dispatched by OPEN in the same way as if they would have been received on a physical I/O interface.

5.4.1.1.1 exchangeAPDU()

```
public static void exchangeAPDU(APDU apdu, short length)
throws IS07816, NullPointerException, TransactionException
```

The `exchangeAPDU` method uses the current APDU and temporarily replaces it with updated information, like CLA INS etc. Because the APDU case cannot be determined the length of the APDU is passed as a parameter. The APDU CLA is always to be found at offset 0. Before the method returns the data is cleared to ensure no sensitive data is passed back to the caller.

Parameters:

- `length` - Length of the full APDU to be processed including possible Le.
- `apdu` - The APDU buffer.

Returns:

- The number of bytes filled on the APDU buffer by the underlying processed OPEN/applet including the status word.

Throws:

- `java.lang.NullPointerException` - if apdu is null.
- `javacard.framework.TransactionException` - with reason code `TransactionException.IN_PROGRESS` if a transaction was opened (and is still pending) by the caller before calling this API.
- `javacard.framework.ISOException` - with reason code `ISO7816.SW_WRONG_LENGTH` if
 - the provided length parameter is greater than 261.
 - there is an inconsistency of the provided length parameter with the Lc byte of the provided C-APDU.
 - The APDU received by the calling application on the physical interface is an extended length APDU.

5.4.2 JCOPX eGovAccelerators

The `com.nxp.id.jcopx.egovaccelerators` package helps speed up the processing of secure messaging.

Tab. 5.39: Supported APIs of package `com.nxp.id.jcopx.egovaccelerators`

Class	Method	Comment
SMAccelerator	<code>clearSessionKeys()</code>	See Section 5.4.2.2 - Class SMAccelerator
	<code>copySSC()</code>	
	<code>getCipherObject()</code>	
	<code>getInstance()</code>	
	<code>getKeySizeBytes()</code>	
	<code>getMaximumResponseSize()</code>	
	<code>getProtocol()</code>	
	<code>getSignatureObject()</code>	
	<code>getUnwrappedLc()</code>	
	<code>getUnwrappedLe()</code>	
	<code>isExtendedLength()</code>	
	<code>processReadBinary()</code>	
	<code>resetSSC()</code>	
	<code>setEncryptionKey()</code>	
	<code>setMacKey()</code>	
	<code>setProtocol()</code>	
	<code>setSecurityLevel()</code>	
	<code>setSSC()</code>	
	<code>unwrapAPDU()</code>	
	<code>wrapResponse()</code>	
EgovUtils	<code>secureXor()</code>	see Section 5.4.2.1 - Class EgovUtils

5.4.2.1 Class EgovUtils

`public final class EgovUtils extends Object`

This class is part of the `com.nxp.id.jcopx.egovaccelerators` package.

The `EgovUtils` class defines static utility methods

5.4.2.1.1 `secureXor()`

`public static short secureXor(byte[] arrayA, short offsetA, byte[] arrayB, short offsetB, short length)`

throws `java.lang.SecurityException`, `javacard.framework.SystemException`,
`java.lang.NullPointerException`, `java.lang.ArrayIndexOutOfBoundsException`

This method performs a XOR calculation on byte arrays in non-linear order. The XOR operation is performed for each individual byte of the two arrays and the result is written into arrayA. For each index *i* from 0 to `length-1`, the following operation takes place:

`arrayA[offsetA+i] = arrayA[offsetA+i] XOR arrayB[offsetB+i]`.

This method is limited to `length` that is a multiple of 8 Bytes. Passing a `length` not multiple of 8 Bytes throws a `SystemException`.

The array `arrayA` must be transient.

The bytes of the arrays are accessed in non-linear order. If the `arrayA` and `arrayB` refer to the same array object and the offset ranges overlap, then the result is unspecified, but no exception is thrown.

Parameters:

- `arrayA` - the first input array.
- `offsetA` - offset into the first input array.
- `arrayB` - the second input array.
- `offsetB` - offset into the second input array.
- `length` - number of bytes to process.

Returns:

- `offsetA + length`.

Throws:

- `java.lang.NullPointerException` - if the `arrayA` or `arrayB` parameter is `null`.
- `java.lang.SecurityException` - if any of the arrays is not accessible to the caller.
- `java.lang.ArrayIndexOutOfBoundsException` - if the operation would cause access of data outside array bounds.
- `javacard.framework.SystemException` - with the reason code `ILLEGAL_USE` in any of the following conditions:
 - `arrayA` is not transient
 - `length` is not a multiple of 8 Bytes
 - `length` is zero

5.4.2.2 Class SMAccelerator

```
public final class SMAccelerator extends Object
```

This class is part of the `com.nxp.id.jcopx.egovaccelerators` package.

`SMAccelerator` may be used by an applet to perform entity authentication and APDU security. This interface is designed to offer interoperability to the applet in that it requires no knowledge of the mechanisms used to perform the authentication.

This class uses the following fields:

MAX_LENGTH_SSC

```
public static final short MAX_LENGTH_SSC = (short) 0x16;
```

16 Bytes is the maximum secure session counter size when AES Secure Messaging is used.

SECURITY_LEVEL_CONFIDENTIALITY_INTEGRITY

```
public static final byte SECURITY_LEVEL_CONFIDENTIALITY_INTEGRITY = (byte) 0x73;
```

Used for the security level of the secure channel accelerator. The level can be either Integrity or Integrity and Confidentiality.

SECURITY_LEVEL_INTEGRITY

```
public static final byte SECURITY_LEVEL_INTEGRITY = (byte) 0x03;
```

The default value of the secure channel accelerator's security level.

SM_3DES_CBC_CBC

```
public static final short SM_3DES_CBC_CBC = (short) 0x0000;
```

The default BAC secure messaging protocol, also used for the SAC OIDs id-PACE-DH-GM-3DES-CBC-CBC, id-PACE-ECDH-GM-3DES-CBC-CBC, id-PACE-DH-IM-3DES-CBC-CBC and id-PACE-ECDH-IM-3DES-CBC-CBC. This protocol uses two key DES in CBC mode for encryption, using a 16 byte ABA key. The IV is set to all zeros for each APDU. For MAC the MAC in ISO/IEC 9797-1 part 1, algorithm 3 is used, which is referenced from ISO 7816-4 [18]. It uses the SSC for the initial block and used ISO padding (one bit set to 1, the following to 0, mandatory padding) to calculate the MAC. The header is separately padded using the same ISO padding, it is not encrypted. The user should use the key derivation described in the ICAO Technical Report using the SHA-1 algorithm.

SM_3DES_CBC_CBC_IDL

```
public static final short SM_3DES_CBC_CBC_IDL = SM_3DES_CBC_CBC;
```

SM_AES_CBC_CMAC_128

```
public static final short SM_AES_CBC_CMAC_128 = (short) 0x0001;
```

The SAC protocol with the lowest key size for AES, used for the SAC OID's with name id-PACE-DH-GM-AES-CBC-CMAC-128, id-PACE-ECDH-GM-AES-CBC-CMAC-128, id-PACE-DH-IM-AES-CBC-CMAC-128 and id-PACE-ECDH-IM-AES-CBC-CMAC-128. This protocol uses AES in CBC mode for encryption, using a 16 byte (128 bit) key. The IV is set to a single block encrypt with the encryption key of the SSC. For MAC the AES CMAC algorithm is used, which is defined in NIST Special Publication 800-38B [2]. It uses the SSC for the initial block and used ISO padding (one bit set to 1, the following to 0, mandatory padding) to calculate the MAC. The header is separately padded using the same ISO padding, it is not encrypted. The user should use the key derivation described in the ICAO Technical Report using the SHA-1 algorithm.

SM_AES_CBC_CMAC_128_IDL

```
public static final short SM_AES_CBC_CMAC_128_IDL = SM_IDL_MODE | SM_AES_CBC_CMAC_128;
```

SM_AES_CBC_CMAC_192

```
public static final short SM_AES_CBC_CMAC_192 = (short) 0x0002;
```

The SAC protocol with the medium key size for AES, used for the SAC OID's with name id-PACE-DH-GM-AES-CBC-CMAC-192, id-PACE-ECDH-GM-AES-CBC-CMAC-192, id-PACE-DH-IM-AES-CBC-CMAC-192 and id-PACE-ECDH-IM-AES-CBC-CMAC-192. This protocol uses AES in CBC mode for encryption, using a 24 byte (192 bit) key. The IV is set to a single block encrypt with the encryption key of the SSC. For MAC the AES CMAC al-

gorithm is used, which is defined in NIST Special Publication 800-38B [2]. It uses the SSC for the initial block and uses ISO padding (one bit set to 1, the following to 0, mandatory padding) to calculate the MAC. The header is separately padded using the same ISO padding, it is not encrypted. The user should use the key derivation described in the ICAO Technical Report using the SHA-256 algorithm.

SM_AES_CBC_CMACE_192_IDL

```
public static final short SM_AES_CBC_CMACE_192_IDL = SM_IDL_MODE | SM_AES_CBC_CMACE_192;
```

SM_AES_CBC_CMACE_256

```
public static final short SM_AES_CBC_CMACE_256 = (short) 0x0003;
```

The SAC protocol with the high key size for AES, used for the SAC OID's with name id-PACE-DH-GM-AES-CBC-CMAC-256, id-PACE-ECDH-GM-AES-CBC-CMAC-256, id-PACE-DH-IM-AES-CBC-CMAC-256 and id-PACE-ECDH-IM-AES-CBC-CMAC-256. This protocol uses AES in CBC mode for encryption, using a 32 byte (256 bit) key. The IV is set to a single block encrypt with the encryption key of the SSC. For MAC the AES CMAC algorithm is used, which is defined in NIST Special Publication 800-38B [2]. It uses the SSC for the initial block and used ISO padding (one bit set to 1, the following to 0, mandatory padding) to calculate the MAC. The header is separately padded using the same ISO padding, it is not encrypted. The user should use the key derivation described in the ICAO Technical Report using the SHA-256 algorithm.

SM_AES_CBC_CMACE_256_IDL

```
public static final short SM_AES_CBC_CMACE_256_IDL = SM_IDL_MODE | SM_AES_CBC_CMACE_256;
```

SM_IDL_MODE

```
public static final short SM_IDL_MODE = 0100;
```

The following methods are implemented:

5.4.2.2.1 clearSessionKeys()

```
public void clearSessionKeys()
```

Clears the session keys for en/decryption and MAC.

5.4.2.2.2 copySSC()

```
public void copySSC(byte[] sscBuffer, short length)  
throws ArrayIndexOutOfBoundsException, NullPointerException
```

Copies the secure session counter (SSC) to the buffer provided.

Parameters:

- `sscBuffer` - the array the secure session counter should be copied to.
- `length` - the number of bytes to copy.

Throws:

- `ArrayIndexOutOfBoundsException` - if `length` is greater than 16.
- `NullPointerException` - if `sscBuffer` is null.

5.4.2.2.3 getCipherObject()

```
public Cipher getCipherObject(byte[] ivBuffer, short offset, short length)
```

Returns a `Cipher` object initialized with the iv provided by caller and the session ENC key. The cipher object is set to `MODE_ENCRYPT` and `ALG_x_CBC_NOPAD`. The purpose of the method is to facilitate PACE CAM implementation in the calling applet. This means the calling applet needs to implement any padding for example `ISO9797_M2`.

Parameters:

- `ivBuffer` - buffer containing the iv value to use when the `Cipher` object is initialized.
- `offset` - IV value offset.
- `length` - IV value length.

Returns:

- A `Cipher` object initialized with the session ENC key and IV.

5.4.2.2.4 SMAccelerator getInstance() - using CLEAR ON DESELECT memory

```
public static final SMAccelerator getInstance(short protocol)  
throws SystemException
```

Gets an instance of the `SMAccelerator` initialized with keys based on the given protocol. The instance uses CLEAR ON DESELECT memory.

Parameters:

- `protocol` - one of the protocol constants defined in this class, for example `SM_3DES_CBC_CBC`.

Returns:

- The newly initialized `SMAccelerator`.

Throws:

- `SystemException` - if there is an error.

5.4.2.2.5 SMAccelerator getInstance()

```
public static final SMAccelerator getInstance(short protocol, byte event)
throws CryptoException
```

Gets an instance of the `SMAccelerator` initialised with keys based on the given protocol.

Parameters:

- `protocol` - one of the protocol constants defined in this class, for example `SM_3DES_CBC_CBC`.
- `event` - either `JCSysSystem.MEMORY_TYPE_TRANSIENT_DESELECT` or `JCSysSystem.MEMORY_TYPE_TRANSIENT_RESET`.

Returns:

- The newly initialized `SMAccelerator`.

Throws:

- `CryptoException` - with value `ILLEGAL_VALUE`.

5.4.2.2.6 getKeySizeBytes()

```
public short getKeySizeBytes()
```

Returns the key size in bytes of the encryption keys used by the current protocol. For the current protocols the encryption key and MAC key size are equal.

Returns:

- The key size in Bytes, for example the value 16 for 2 key 3DES and AES-128 keys.

5.4.2.2.7 `getMaximumResponseSize()`

```
public short getMaximumResponseSize()
```

Calculates the maximum response data length depending on the incoming APDU. The result will differ from standard size APDUs to extended length APDUs.

It is necessary to call `unwrapAPDU` before calling this method. This is because one calculation factor is the missing padding byte in the case of DO85.

Returns:

- The maximum data length in bytes to be wrapped in the response APDU.

5.4.2.2.8 `getProtocol()`

```
public short getProtocol()
```

Returns the current protocol, for example the default `SM_3DES_CBC_CBC` protocol, or one of the other protocols set by `setProtocol`.

Returns:

- The protocol, as defined by the constants of this class.

5.4.2.2.9 `Signature getSignatureObject()`

```
public Signature getSignatureObject(byte mode)
```

Returns a `Signature` object initialized with the session MAC key.

Parameters:

- `mode` - either `Signature.MODE_SIGN` or `Signature.MODE_VERIFY`.

Returns:

- A `Signature` object initialized with the session MAC key.

5.4.2.2.10 `getUnwrappedLc()`

```
public short getUnwrappedLc()
```

Gets the Lc value of the unwrapped message.

Returns:

- Short Lc value.

5.4.2.2.11 getUnwrappedLe()

```
public short getUnwrappedLe()
```

Gets the Le value of the unwrapped message. The value is cleared (set to -1) by a call to `wrapResponse` or `processReadBinary`.

Returns:

- Short Le value, or -1 if Le has not been set.

5.4.2.2.12 isExtendedLength()

```
public boolean isExtendedLength()
```

Returns `true` if the APDU processed by `unwrapAPDU` was an extended length APDU, `false` otherwise.

5.4.2.2.13 processReadBinary()

```
public short processReadBinary(byte[] content, short offset, short le)  
throws NullPointerException
```

Wraps as many file content bytes as possible (restricted by unwrapped `le` and wrap capacity of the OS) and sets the OS to automatically process any subsequent READ BINARY commands for the same file. After a successful call the calling applet does not need to invoke the Java Card APDU API to transmit the wrapped response data. On an erroneous call the applet should throw an error status word.

The session counter and keys are retained by the OS and are used to automatically process subsequent READ BINARY APDUs (if they are still reading from the same content data).

Note:

- When `offset + le` are greater than the filesize (and no additional file is set up), the response length is truncated and will contain any remaining bytes.
- When `offset` is greater than the filesize (and no additional file is set up) the card is muted.

Parameters:

- `content` - data to be transmitted.
- `offset` - offset of first byte to transmit.
- `le` - `le` value received in READ BINARY command unwrapped by the caller.

Returns:

- The status of the request, (short) `0xA5A5` if the request is successful. Any other value indicates an unsuccessful request.

Throws:

- `NullPointerException` - if `content` is null.

5.4.2.2.14 resetSSC()

```
public void resetSSC()
```

Sets the value of the Secure Session Counter (SSC) to zero.

5.4.2.2.15 setEncryptionKey()

```
public void setEncryptionKey(byte[] encryptionKeyBuffer, short encryptionKeyOffset)  
throws ArrayIndexOutOfBoundsException, NullPointerException
```

Sets the encryption and decryption session key. The key is a 16 Byte 3DES key or an AES key. This method may be called at any time and will immediately replace the session key.

Parameters:

- `encryptionKeyBuffer` - the buffer containing the encryption key. The left-most key byte is located at the lowest array index (`encryptionKeyOffset`) in `encryptionKeyBuffer`.
- `encryptionKeyBuffer` - the offset in array `encryptionKeyBuffer` where the key starts.

Throws:

- `ArrayIndexOutOfBoundsException` - is thrown if `encryptionKeyOffset + key length` is greater than `encryptionKeyBuffer.length`.
- `NullPointerException` - is thrown if `encryptionKeyBuffer` is null.

5.4.2.2.16 `setMacKey()`

```
public void setMacKey(byte[] macKeyBuffer, short macKeyOffset)
throws ArrayIndexOutOfBoundsException, NullPointerException
```

Sets the MAC session key. The key is a 16 Byte 3DES key or an AES key. This method may be called at any time and will immediately replace the session key.

Parameters:

- `macKeyBuffer` - the buffer containing the MAC key. The left-most key byte is located at the lowest array index (`=encryptionKeyOffset`) in `encryptionKeyBuffer`.
- `macKeyOffset` - the offset in array `encryptionKeyBuffer` where the key starts.

Throws:

- `ArrayIndexOutOfBoundsException` - is thrown if `encryptionKeyOffset + key length` is greater than `encryptionKeyBuffer.length`.
- `NullPointerException` - is thrown if `encryptionKeyBuffer` is null.

5.4.2.2.17 `setProtocol()`

```
public void setProtocol(final short protocol)
throws CryptoException
```

Sets the protocol to be used, defined by the constant values of `SM_3DES_CBC_CBC`, `SM_AES_CBC_CMAC_128`, `SM_AES_CBC_CMAC_192` and `SM_AES_CBC_CMAC_256`. This method clears the session keys and SSC as defined by the `clearSessionKeys` and `resetSSC` methods. It may be called at any time and will immediately take effect.

Parameters:

- `protocol` - the protocol to be set.

Throws:

- `CryptoException` - with reason `ILLEGAL_VALUE`.

5.4.2.2.18 setSecurityLevel()

```
public void setSecurityLevel(byte level)
throws CryptoException.ILLEGAL_USE
```

Deprecated. Throws ILLEGAL_USE.

5.4.2.2.19 setSSC()

```
public void setSSC(byte[] sscBuffer, short sscOffset)
throws ArrayIndexOutOfBoundsException, NullPointerException
```

Sets the secure session counter (SSC). This method requires the input array to be at least 16 Bytes long even if the session counter is only 8 Bytes. If the session counter is 8 Bytes, the 8 trailing bytes are not used. There is no need to call `resetSSC` prior to calling this method.

Parameters:

- `sscBuffer` - the array containing the 8 or 16 Byte value for the session counter.
- `sscOffset` - the offset in array `sscBuffer` to the left most byte (msb) of the session counter.

Throws:

- `ArrayIndexOutOfBoundsException` - if array `sscBuffer` starting from the index `sscOffset` does not have at least 16 Bytes available after the offset.
- `NullPointerException` - if `sscBuffer` is null.

5.4.2.2.20 unwrapAPDU()

```
public short unwrapAPDU(byte[] inBuf, short inOffset)
throws ISOException
```

Unwraps (verify and decrypt) the command APDU located in the given `byte[]`. The command buffer has to be filled by the applet with data received using `APDU.setIncomingAndReceive()` method beforehand. The verified and decrypted command data is placed into the same buffer at the same offset.

An encrypted standard APDU results in a standard decrypted APDU and an extended length APDU gets decrypted to extended length representation. In case of standard APDU (not extended length) with a `Le` larger

than 0xFF (taken from DO97) a 2-Byte Le is appended to the standard APDU instead of a one-Byte Le. However, the maximum response data that can be sent will still be limited to 256 Bytes in the case of a standard APDU.

The session counter is automatically incremented.

Parameters:

- `inBuf` - the buffer containing the incoming APDU.
- `inOffset` - the offset in array `inBuf`.

Returns:

- The length of the unwrapped APDU.

Throws:

- `IOException` - with error status word if an error occurred during unwrapping.

5.4.2.2.21 wrapResponse()

```
public short wrapResponse(byte[] dataBuffer, short dataOffset, short dataLength,  
byte[] outBuffer, short outOffset, short sw)  
throws ArrayIndexOutOfBoundsException, NullPointerException
```

Wraps (encrypts and generates MAC) the response data and places it in the out buffer starting at the offset provided. The buffer can be any buffer including the APDU buffer itself. If the length is zero the buffer will not be addressed and no response data will be present in the wrapped output.

Session counter is automatically incremented.

Parameters:

- `dataBuffer` - array with plain data which has to be packed into a secure message response APDU.
- `dataOffset` - offset in array `dataBuffer` where the unencrypted source data starts.
- `dataLength` - length of the unencrypted source data in array `dataBuffer`.
- `outBuffer` - array where the generated secure messaging response is placed.
- `outOffset` - offset in array `outBuffer`.
- `sw` - the status word which is part of the wrapped secure message.

Returns:

- The length of the wrapped data in the APDU buffer.

Throws:

- `ArrayIndexOutOfBoundsException` - is thrown if `dataOffset + dataLength` is longer than the array length of `dataBuffer` or the wrapped response length exceeds the maximum reported by method `getMaximumResponseSize` (see Section 5.4.2.2.7 - [getMaximumResponseSize\(\)](#)).
- `NullPointerException` - is thrown if `dataBuffer` is null.

5.4.3 JCOPX Math

This class defines modular arithmetic functions not provided by the standard Java Card API.

The `Math` functions use the modular arithmetic functions provided by the crypto library.

Tab. 5.40: Supported APIs of package `com.nxp.id.jcopx.math`

Class	Method	Comment
Math	<code>modularAdd</code> <code>modularMultiply</code> <code>modularReduce</code> <code>modularSubtract</code>	see Section 5.4.3.1 - Class Math

5.4.3.1 Class Math

```
public class Math extends Object
```

This class is part of the `com.nxp.id.jcopx.math` package.

This class defines modular arithmetic functions not provided by the standard Java Card API.

This API operates on unsigned numbers represented as sequences of unsigned bytes with the most-significant byte first (big-endian representation). The numbers are stored in byte arrays which can start with leading zeros if needed.

All numbers must have a byte count less than a maximum supported size, which depends on the product configuration.

All methods take the following parameters:

- a modulus N, which must be odd;
- a first operand A;
- a second operand B, except for `modularReduce`.

The buffer of A is also the output buffer where the result of the operation is written. So the size of A must be at least the size of N to avoid overflows. All the bytes in the output buffer are written: if the result fits in fewer bytes than the size of A then it is padded with leading zeros and right-aligned.

They are two kinds of methods:

- The methods `modularReduce` and `modularMultiply` accept any numbers A and B;
- The methods `modularAdd` and `modularSubtract` work only if A and B are both already reduced modulo N, i.e., are both strictly less than N. Otherwise, the implementation may silently generate an undefined and incorrect result and not throw an exception.

All the byte arrays passed to these methods can be persistent or transient.

5.4.3.1.1 `modularAdd()`

```
public static void modularAdd(byte[] a, short aOffset, short aLength, byte[] b, short bOffset, short bLength, byte[] n, short nOffset, short nLength)
throws NullPointerException, SecurityException, ArrayIndexOutOfBoundsException, CryptoException
```

Modular adds A and B modulo N, that is, it performs $A = A + B \bmod N$.

Modular addition is guaranteed to work correctly only if both A and B are already reduced, i.e., are $< N$. Otherwise, the implementation may silently generate an undefined and incorrect result and not throw an exception. As a result, the lengths `aLength` and `bLength` must not be larger than `nLength`. Note that because `aLength` must also be large enough to hold the result, it must therefore be equal to `nLength`.

Parameters:

- `a` - byte array containing the operand A.
- `aOffset` - offset to start of operand A in the array `a`.
- `aLength` - length of operand A in the array `a`.

- `b` - byte array containing the operand B.
- `bOffset` - offset to start of operand B in the array `b`.
- `bLength` - length of operand B in the array `b`.
- `n` - byte array containing the modulus N.
- `nOffset` - offset to start of modulus in `n`.
- `nLength` - length of modulus in `n`.

Throws:

- `java.lang.NullPointerException` - if any of the arrays is null.
- `java.lang.SecurityException` - if any array is not accessible to the caller.
- `java.lang.ArrayIndexOutOfBoundsException` - if any of the offsets or lengths do not fit in their arrays.
- `CryptoException` - with the following reason code:
 - `ILLEGAL_USE` - if `aLength`, `bLength`, or `nLength` is greater than the maximum length supported by the implementation.
 - `ILLEGAL_VALUE` - if `aLength` $\neq$ `nLength` or if `bLength` $==$ 0 or if `bLength` $>$ `nLength` or if N is not odd.

5.4.3.1.2 modularMultiply()

```
static void modularMultiply(byte[] a, short aOffset, short aLength, byte[] b,  
short bOffset, short bLength, byte[] n, short nOffset, short nLength)  
throws NullPointerException, SecurityException, ArrayIndexOutOfBoundsException,  
CryptoException.ILLEGAL_USE, CryptoException.ILLEGAL_VALUE
```

Modular multiplies A and B modulo N, i.e., performs $A = A * B \bmod N$.

The numbers A and B can be any number, provided they are within the length supported by the implementation.

Parameters:

- `a` - byte array containing the operand A.
- `aOffset` - offset to the start of A in `a`.
- `aLength` - length of A in `a`.

- `b` - byte array containing the operand B.
- `bOffset` - offset to start of operand B in the array `b`.
- `bLength` - length of operand B in the array `b`.
- `n` - byte array containing the modulus N.
- `nOffset` - offset to start of modulus in `n`.
- `nLength` - length of modulus in `n`.

Throws:

- `NullPointerException` - if any of the arrays is null.
- `SecurityException` - if any of the arrays is not accessible to the caller.
- `ArrayIndexOutOfBoundsException` - if any of the offsets or lengths do not fit in their arrays.
- `CryptoException` - with the following reason code:
 - `ILLEGAL_USE` - if `aLength`, `bLength`, or `nLength` is greater than the maximum length supported by the implementation.
 - `ILLEGAL_VALUE` - if `aLength` < `nLength` or if `bLength` == 0 or if N is not odd.

5.4.3.1.3 modularReduce()

```
public static native void modularReduce(modularReduce(byte[] a, short aOffset,  
short aLength, byte[] n, short nOffset, short nLength)  
throws NullPointerException, SecurityException, ArrayIndexOutOfBoundsException,  
CryptoException.ILLEGAL_USE, CryptoException.ILLEGAL_VALUE
```

Modular reduces operand A by modulus N, i.e., performs $A = A \bmod N$.

The number A can be any number, provided it is within the length supported by the implementation.

Parameters:

- `a` - byte array containing the operand A.
- `aOffset` - offset to the start of A in `a`.
- `aLength` - length of A in `a`.

- `n` - byte array containing the modulus `N`.
- `nOffset` - offset to the start of the modulus in `n`.
- `nLength` - length of the modulus in `n`.

Throws:

- `NullPointerException` - if any of the arrays is `null`.
- `SecurityException` - if any array is not accessible to the caller.
- `ArrayIndexOutOfBoundsException` - if any of the offsets or lengths do not fit in their arrays.
- `CryptoException` - with the following reason code:
 - `ILLEGAL_USE` - if `aLength` or `nLength` is greater than the maximum length supported by the implementation.
 - `ILLEGAL_VALUE` - if `aLength` < `nLength` or if the value of the number `N` is not odd.

5.4.3.1.4 modularSubtract()

```
public static void modularSubtract(byte[] a, short aOffset, short aLength, byte[] b, short bOffset,
short bLength, byte[] n, short nOffset, short nLength)
throws NullPointerException, SecurityException, ArrayIndexOutOfBoundsException, CryptoException
```

Modular subtracts `A` and `B` modulo `N`, i.e., performs $A = A - B \bmod N$.

Modular subtraction is guaranteed to work correctly only if both `A` and `B` are already reduced, i.e., are < `N`. Otherwise, the implementation may silently generate an undefined and incorrect result and not throw an exception. As a result, the lengths `aLength` and `bLength` must not be larger than `nLength`. Note that because `aLength` must also be large enough to hold the result, it must therefore be equal to `nLength`).

Parameters:

- `a` - byte array containing the operand `A`.
- `aOffset` - offset to start of operand `A` in the array `a`.
- `aLength` - length of operand `A` in the array `a`.
- `b` - byte array containing the operand `B`.
- `bOffset` - offset to start of operand `B` in the array `b`.

- `bLength` - length of operand B in the array `b`.
- `n` - byte array containing the modulus N.
- `nOffset` - offset to start of modulus in `n`.
- `nLength` - length of modulus in `n`.

Throws:

- `java.lang.NullPointerException` - if any of the arrays is null.
- `java.lang.SecurityException` - if any array is not accessible to the caller.
- `java.lang.ArrayIndexOutOfBoundsException` - if any of the offsets or lengths do not fit in their arrays.
- `CryptoException` - with the following reason code:
 - `ILLEGAL_USE` - if `aLength`, `bLength`, or `nLength` is greater than the maximum length supported by the implementation.
 - `ILLEGAL_VALUE` - if `aLength != nLength` or if `bLength == 0` or if `bLength > nLength` or if N is not odd.

5.4.4 JCOPX Memory

5.4.4.1 MIFARE applet overview

JCOP OS can be ordered with a MIFARE applet, implementing either MIFARE Plus EV1 or MIFARE DESFire EV2 functionality. A dedicated user manual of the applet is available. As soon as an instance with the same AID defined in the `MIFARE_APPLET_AID` configuration item (see Section 8.3.6.4 - [MIFARE_APPLET_AID](#)) is available, an applet can access basic functionality of the MIFARE applet using the API for `MemoryAccessX`.

Tab. 5.41: Supported APIs of package `com.nxp.id.jcopx.memoryaccessx`

Class	Method	Comment
MemoryAccessX	<code>closeProcessDataSession()</code> <code>openProcessDataSession()</code> <code>processData()</code> <code>resetSector()</code>	See Section 5.4.4.2 - Class MemoryAccessX
MemoryX	<code>getMemoryAccessInstance()</code>	See Section 5.4.4.3 - Class MemoryX

5.4.4.2 Class MemoryAccessX

public interface MemoryAccessX javacardx.external.MemoryAccess

This class is part of the com.nxp.id.jcopx.memory package.

This interface allows an applet to send native commands to the MIFARE Plus EV1 or MIFARE DESFire EV2 subsystem. This is not supported in the standard MemoryAccess interface.

Some details of the implementation of this API are specific to the MIFARE Plus EV1 or MIFARE DESFire EV2 subsystems and can be found in their respective User Manuals.

The following methods are implemented:

5.4.4.2.1 closeProcessDataSession()

short closeProcessDataSession()

throws javacardx.external.ExternalException,
javacard.framework.SystemException

Closes the current session of processData calls. This cleans all the subsystem internal session state.

Returns:

- ConstantX.TRUE16 if closing was successful. Otherwise, an exception is thrown.

Throws:

- javacardx.external.ExternalException - INVALID_PARAM if there is no processData session opened or if one is opened, but not associated to the same calling context..
- javacardx.external.ExternalException - INTERNAL_ERROR for any other internal error..

5.4.4.2.2 openProcessDataSession()

short openProcessDataSession()

throws javacardx.external.ExternalException,
javacard.framework.SystemException

Opens a session of processData calls. This method must be called before the first execution of processData.

The session can be closed by calling closeProcessDataSession. A call to any method other than processData also implicitly closes the session before executing the method. This includes calling openProcessDataSession itself: the current session is closed and a new one is opened.

At any point in time, there can be only one `processData` session opened on each of the MIFARE Plus EV1 and MIFARE DESFire EV2 subsystems. When the session is opened, the subsystem records the AID of the calling context (by calling `JCSystem.getPreviousContextAID`). A subsequent call to `processData` or `closeProcessDataSession` from a different calling context throws a `ExternalException` with reason `INVALID_PARAM`. A call to any other method from a different context implicitly closes the session.

Returns:

- `ConstantX.TRUE16` if the session opening was successful. Otherwise, an exception is thrown.

Throws:

- `javacard.framework.SystemException - ILLEGAL_USE` if the API is not allowed to be executed by the subsystem. For example, the subsystem can reject a `processData` session when FIPS is enabled. See the subsystem User Manual for more detail.
- `javacardx.external.ExternalException - INTERNAL_ERROR` for any other internal error.

5.4.4.2.3 processData()

```
short processData(byte[] inData, short inDataOffset, short inDataLength, byte[] outData,  
short outDataOffset, short options)  
throws javacardx.external.ExternalException, javacard.framework.SystemException,  
java.lang.SecurityException, java.lang.ArrayIndexOutOfBoundsException,  
java.lang.NullPointerException
```

Processes the native MIFARE Plus EV1 or MIFARE DESFire EV2 command stored in `inData`. A `processData` session must have been previously opened with `openProcessDataSession`.

Returns:

- `options` - This short is passed to the MIFARE Plus EV1 or MIFARE DESFire EV2 subsystem. See their User Manuals for more details.
- `inData` - Reference to a buffer containing the native MIFARE Plus EV1 or MIFARE DESFire EV2 command.
- `inDataOffset` - Offset where the native command starts within `inData`.
- `inDataLength` - Length of the native command.
- `outData` - Reference to a buffer where the native response must be stored. This must be large enough to store the response.

- `outDataOffset` - Offset where the MIFARE Plus EV1 or MIFARE DESFire EV2 response should be stored within `outData`.

Returns:

- Length of response data or any other short returned by the MIFARE Plus EV1 or MIFARE DESFire EV2 subsystem. See their User Manuals for more details. .

Throws:

- `java.lang.NullPointerException` - If any of the arrays is null.
- `java.lang.SecurityException` - If any of the arrays is not accessible to the caller.
- `java.lang.ArrayIndexOutOfBoundsException` - If any of the offsets or lengths do not fit in their arrays.
- `javacard.framework.SystemException` - `ILLEGAL_USE` if the API is not allowed to be executed by the subsystem.
- `javacardx.external.ExternalException` - `INVALID_PARAM` if there is no `processData` session opened or if one is opened, but not associated to the same calling context.
- `javacardx.external.ExternalException` - `INTERNAL_ERROR` for any other internal error.

5.4.4.2.4 resetSector()

`short resetSector(byte sector)`

throws `javacardx.external.ExternalException`

Resets the provided MIFARE Classic sector to its default value.

This method can be used only for a MIFARE Plus EV1 memory access object configured as MIFARE Classic (security level 1) .

Parameters:

- `sector` - The sector number that shall be reset.

Returns:

- `ConstantX.TRUE16` if reset sector was successful, else an exception is thrown.

Throws:

- `javacardx.external.ExternalException` - `NO_SUCH_SUBSYSTEM` if this memory access object is not connected to a MIFARE Plus EV1 implementation configured as MIFARE Classic.

- `javacard.framework.SystemException - ILLEGAL_USE` if the API is not allowed to be executed by the MIFARE Plus EV1 subsystem, e.g., if FIPS is enabled.
- `javacardx.external.ExternalException - INVALID_PARAM` if the sector number is not valid. Valid sector numbers are from 0 to 31 in case of 2k MIFARE and in case of 4k MIFARE from 0 to 39.
- `javacardx.external.ExternalException - INTERNAL_ERROR` for any other internal error.

5.4.4.3 Class MemoryX

```
public final class MemoryX extends java.lang.Object
```

This class is part of the `com.nxp.id.jcopx.memory` package.

This class contains a factory method to create objects implementing the `MemoryAccessX` interface.

This class uses the following fields:

MEMORY_TYPE_MIFARE_DESFIRE

Identifier for the MIFARE DESFire EV2 memory subsystem

MEMORY_TYPE_MIFARE_PLUS

Identifier for the MIFARE Plus EV1 memory system.

5.4.4.3.1 `getMemoryAccessInstance()`

```
public static getMemoryAccessInstance(short memoryType)
```

```
throws NullPointerException, SecurityException, ArrayOutOfBoundsException, ExternalException,  
SystemException
```

Creates a `MemoryAccessX` object instance for the selected memory subsystem.

Two memory subsystem can be supported:

- The type `MEMORY_TYPE_MIFARE_DESFIRE` is to access a MIFARE DESFire EV2 system, if available.
- The type `MEMORY_TYPE_MIFARE_PLUS` is to access a MIFARE Plus EV1 system, if available. In this case, the object returned is the same as if you call the standard factory `Memory.getMemoryAccessInstance` with the constant `MEMORY_TYPE_MIFARE`, but the object is already cast to a `MemoryAccessX`.

Parameters:

- `memoryType` - the desired external memory subsystem. Valid codes listed in `MEMORY_TYPE_*` constants above.

Returns:

- The `MemoryAccessX` object instance of the requested memory subsystem.

Throws:

- `java.lang.NullPointerException` - if `memorySize` is null.
- `java.lang.SecurityException` - if `memorySize` is not accessible to the caller.
- `java.lang.ArrayIndexOutOfBoundsException` - if `memorySizeOffset` causes an access outside of the boundaries of `memorySize`.
- `ExternalException` - with the following reason code:
 - `NO_SUCH_SUBSYSTEM` - if the required subsystem is not available.
- `javacard.framework.SystemException` - with the following reason code:
 - `ILLEGAL_USE` - if the required subsystem is available but the access is not allowed in the current configuration, for example if FIPS is enabled.

5.4.5 JCOPX MifareSupport.4

Tab. 5.42: Supported APIs of package `com.nxp.id.jcopx.mifaresupport.4`

Class	Method	Comment
<code>MemoryAccessServiceInterface</code>	<code>closeProcessDataSession()</code> <code>openProcessDataSession()</code> <code>processData()</code> <code>readData()</code> <code>resetSector()</code> <code>writeData()</code>	see Section 5.4.5.1 - Class MemoryAccessServiceInterface

5.4.5.1 Class `MemoryAccessServiceInterface`

`public interface MemoryAccessServiceInterface` extends `javacard.framework.Shareable`

This class is part of the `com.nxp.id.jcopx.mifaresupport.4` package.

Shareable interface that must be implemented by server applets to make them accessible through the `MemoryAccessX` or `MemoryAccess` API.

5.4.5.1.1 About array views

This interface is called by JCOP. Operation data are passed through temporary array views stored internally in JCOP and accessible by a Secure Box native library through get and set syscalls.

While array views are not first class references accessible to Java code, the firewall checks are identical to the array views provided in the forthcoming Java Card 3.1 specification:

- The `MemoryAccessX` and `MemoryAccess` objects are created by JCOP and owned by the calling applet. When JCOP creates the access object, it determines the instance of the server applet and retrieves an object implementing `MemoryAccessServiceInterface` object. The low byte of the parameter `memoryType` provided to the factory `MemoryX.getInstance` is forwarded to the applet within the parameter parameter of the method `Applet.getShareableInterfaceObject`.
- When the calling applet invokes one of the methods of the access object, JCOP creates zero, one, or two views for the data parameters. The source array for each view is one of the parameters passed to the `MemoryAccessX` or `MemoryAccess` method and it must be accessible to the calling context. The owner of the view is the server applet, i.e., the owner of the `MemoryAccessServiceInterface` object.
- JCOP then calls the method of the `MemoryAccessServiceInterface` object corresponding to the original call.
- The target applet can access the views by calling get or set syscalls from a Secure Box library.
- The views are closed before returning from the call to the access object method, either through a normal return or through an exception.

There two possible array views, named `View#0` and `View#1`. They can have read-only, write-only, or read-write access rights. The documentation of each method in this interface specifies what views are used and what are their access rights.

`MemoryAccessServiceInterface` implements the following methods:

5.4.5.1.2 `readData()`

```
short readData(other_sector, short other_block)
throws javacardx.external.ExternalException
```

Implements `MemoryAccess.readData`.

Array View Parameters:

- View#0 - write-only view on dest array from offset dest_off with length dest_len
- View#1 - read-only view on auth_key array from offset auth_key_off with length auth_key_len

Parameters:

- other_sector - the parameter other_section of the call to MemoryAccess.readData.
- other_block - the parameter other_block of the call to MemoryAccess.readData.

Returns:

- The length in bytes of the data written in View#0 or 0 if none.

Throws:

- javacardx.external.ExternalException - INVALID_PARAM if any of the input parameters are invalid.
- javacardx.external.ExternalException - INTERNAL_ERROR if an unrecoverable external memory access error occurred.

5.4.5.1.3 writeData()

short writeData(other_sector, short other_block)

throws javacardx.external.ExternalException

Implements MemoryAccess.writeData.

Array View Parameters:

- View#0 - read-only view on src array from offset src_off with length src_blen
- View#1 - read-only view on auth_key array from offset auth_key_off with length auth_key_len

Parameters:

- other_sector - the parameter other_section of the call to MemoryAccess.
- other_block - the parameter other_block of the call to MemoryAccess.

Returns:

- ConstantX.TRUE16 if the call was successful.
ConstantX.FALSE16 otherwise .

Throws:

- javacardx.external.ExternalException - INVALID_PARAM if any of the input parameters are invalid.
- javacardx.external.ExternalException - INTERNAL_ERROR if an unrecoverable external memory access error occurred.

5.4.5.1.4 openProcessDataSession()

short openProcessDataSession()
throws javacardx.external.ExternalException

Implements MemoryAccessX.openProcessDataSession.

Note:

- It is the responsibility of the server applet to implement the access control specified in MemoryAccessX.openProcessDataSession: the AID of the caller must be determined by calling JCSysm.getPreviousContextAID and associated to the session. Subsequent calls to processData or closeProcessDataSession coming from a different caller must be rejected with ExternalException.INVALID_PARAM.

Returns:

- ConstantX.TRUE16 if initialization was successful.
Otherwise, an exception is thrown.

Throws:

- javacardx.external.ExternalException - INTERNAL_ERROR if an unrecoverable external memory access error occurred.

5.4.5.1.5 processData()

short processData(short options)
throws javacardx.external.ExternalException

Implements MemoryAccessX.processData.

It is the responsibility of the server applet to throw ExternalException.INVALID_PARAM when there is no session opened or when the caller AID is not in the the one recorded during openProcessDataSession.

Array View Parameters:

- View#0 - read-only view on inData array from offset inDataOffset with length inDataLength.
- View#1 - write-only view on outData array from offset outDataOffset until the end of the array.

Parameters:

- `options` - The parameter `options` passed to `MemoryAccessX.processData`. This can be used for the caller to activate certain options for the data processing like chaining.

Returns:

- The short that must be returned from `MemoryAccessX.processData`. JCOP does not interpret this value. It should contain the length of the data written into `View#1` but the server applet can also add more information in some of the high bits.

Throws:

- `javacardx.external.ExternalException` - `INVALID_PARAM` if there is no `processData` session opened or if one is opened, but not associated to the same calling context.
- `javacardx.external.ExternalException` - `INTERNAL_ERROR` for any other internal error.

5.4.5.1.6 closeProcessDataSession()

```
short closeProcessDataSession()
```

```
throws javacardx.external.ExternalException
```

Implements `MemoryAccessX.closeProcessDataSession`.

It is the responsibility of the server applet to throw `ExternalException.INVALID_PARAM` when there is no session opened or when the caller AID is not in the one recorded during `openProcessDataSession`.

Returns:

- `ConstantX.TRUE16` if closing was successful.
Otherwise, an exception is thrown.

Throws:

- `javacardx.external.ExternalException` - `INVALID_PARAM` if there is no `processData` session opened or if one is opened, but not associated to the same calling context.
- `javacardx.external.ExternalException` - `INTERNAL_ERROR` for any other internal error.

5.4.5.1.7 resetSector()

```
short resetSector(byte sector)
```

```
throws javacardx.external.ExternalException
```

Implements `MemoryAccessX.resetSector`.

Parameters:

- `sector` - The sector number that shall be reset.

Returns:

- `ConstantX.TRUE16` if reset sector was successful, else an exception is thrown. .

Throws:

- `javacardx.external.ExternalException - NO_SUCH_SUBSYSTEM` if this memory access object is not connected to a MIFARE Plus EV1 implementation configured as MIFARE Classic.
- `javacardx.external.ExternalException - INVALID_PARAM` if the sector number is not valid. Valid sector numbers are from 0 to 31 in case of 2K MIFARE and in case of 4K MIFARE from 0 to 39.
- `javacardx.external.ExternalException - INTERNAL_ERROR` for any other internal error.

5.4.6 JCOPX PIV

The JCOP `piv` package contains a class that support secure messaging for PIV.

Tab. 5.43: Supported APIs of package `com.nxp.id.jcopx.piv`

Class	Method	Comment
<code>OpacitySmKeyEstablishment</code>	<code>generateSecret()</code>	see Section 5.4.6.1 - Class <code>OpacitySmKeyEstablishment</code>

5.4.6.1 Class `OpacitySmKeyEstablishment`

```
public final class OpacitySmKeyEstablishment extends Object
```

This class is part of the `com.nxp.id.jcopx.piv` package.

`OpacitySmKeyEstablishment` may be used by an applet to support the PIV opacity secure messaging key establishment protocol in accordance with NIST SP.800-73-4 [6].

5.4.6.1.1 `generateSecret()`

```
public static short generateSecret(byte[] publicData, short publicOffset, short publicLength, byte[] secret, short secretOffset, javacard.security.ECPrivateKey privateKey)
```

throws `SecurityException`, `ArrayIndexOutOfBoundsException`, `SystemException`, `CryptoException`

Performs PIV opacity key establishment protocol for secure messaging see NIST SP.800-73-4 [6] Section 4.1 The Key Establishment Protocol. This method performs PIV Card Application steps C4 to C10 using the public data placed in the `publicData` buffer together with the private EC key `dsicc` and writes the secret data into the secret buffer.

The `publicData` buffer contents when calling this method must be:

[IDsh(8 Bytes)|04|Qeh(64 or 96 Bytes)|IDsicc(8 Bytes)]

The public EC point `Qeh` shall always be preceded by '04' and shall be either 64 Bytes or 96 Bytes, that is, `publicLength` must be either 81 or 113 Bytes. The size of `privateKey` must also be consistent with the size of the `Qeh`, i.e., be equal to either 256 or 384. There is no check that `privateKey` is exactly P-256 or P-384, just a check on the key size.

Successful key establishment results in the following data being written in the secret buffer with the following format:

[Nicc(8 bytes)|AuthCryptogramIcc(16 or 24 bytes)|SKMAC|SKENC|SKRMAC]

The session keys SKMAC, SKENC, and SKRMAC are either AES128 (16 Bytes) or AES256 (32 Bytes) depending on the size of `Qeh` and `privateKey`.

Parameters:

- `publicData` - the buffer containing the public input data.
- `publicOffset` - the offset to the start of the public input data.
- `publicLength` - the length of the public input data. Must be either 81 or 113.
- `secret` - the buffer containing the output data. Must be transient.
- `secretOffset` - the offset to the start of the output data.
- `privateKey` - the EC private key - `dsicc`.

Returns:

- The length of the secret data (either 72 or 128 Bytes).

Throws:

- `java.lang.NullPointerException` - if any of the arrays is null.

- `java.lang.SecurityException` - if any of the arrays is not accessible to the caller.
- `java.lang.ArrayIndexOutOfBoundsException` - if any of the offsets or lengths do not fit in their arrays.
- `javacard.framework.SystemException` - `ILLEGAL_USE` if `secretData` is not of type `TRANSIENT`.
- `javacard.security.CryptoException` - with the following reason code:
 - `UNINITIALIZED_KEY` - if the private key object is not initialized.
 - `ILLEGAL_VALUE` - if the public data is invalid or is inconsistent with the private key.

5.4.7 JCOPX PUF

The Physically Unclonable Function (PUF) API.

Tab. 5.44: APIs of package `com.nxp.id.jcopx.accelerator`

Class	Method	Reference
PUF	<code>pufDecrypt()</code> <code>pufEncrypt()</code>	See Section 5.4.7.1 - Class PUF

5.4.7.1 Class PUF

```
public class PUF extends Object
```

This class is part of the `com.nxp.id.jcopx.puf` package.

The PUF function uses randomised inter-atomic effects generated in the manufacturing process to create a unique encryption and decryption process which is specific to a particular IC.

The intended use case is to provide an encrypted data store independent of keys accessible for JCOP and/or applets. The underlying cryptographic algorithm is AES, the key for the cryptographic operation is derived from physical effects which are individual for each card. Data are to be encrypted and integrity protected by the PUF.

5.4.7.1.1 `pufDecrypt()`

```
static short pufDecrypt(byte[] inbuf, short inbufOff, short inbufLen, byte[] outbuf,  
short outbufOff) )
```

Decrypts the input data using the PUF mechanism and writes the plain data to the output buffer.

Only data that has been encrypted with `pufEncrypt` on the same device will deliver useful plain data. The minimum size of encrypted data is 32 Bytes and it is always padded to a multiple of 16 Bytes. So if the input data passed to decrypt is smaller than 32 Bytes, or is not a multiple of 16 Bytes, this will be seen as an illegal use of the API.

For decryption, the output data will be sensitive data and so it shall be output to transient memory .

Parameters:

- `inbuf` - the buffer with the encrypted input data. The input buffer can be in non-volatile or transient memory.
- `inbufOff` - offset of the input data in the `inputbuf`.
- `inbufLen` - size of the input data to decrypt.
- `outbuf` - the buffer the decrypted data is written to. The output buffer must be located in transient memory.
- `outbufOff` - offset of the output data in the output buffer.

Returns:

- `outbufLen`: the exact length of the decrypted data. This is `inbufLen` minus 16 Bytes minus padding from 0 to 15 Bytes.

Throws:

- `SystemException.NO_RESOURCE` - when PUF is not available on the device or if the output buffer is not of type `TRANSIENT`.
- `SecurityException` - upon a processing error.
- `ArrayIndexOutOfBoundsException` - when there is insufficient space in the output buffer due to its size or offset.

5.4.7.1.2 `pufEncrypt()`

```
public static native short pufEncrypt(byte[] inbuf, short inbufOff, short inbufLen,  
byte[] outbuf, short outbufOff)  
throws NullPointerException, ArrayIndexOutOfBoundsException,  
SystemException
```

Encrypts the input data using the PUF mechanism and writes the encrypted data to the output buffer.

The encryption process pads the input data to a multiple of 16 Bytes and adds 16 Bytes of overhead. The

output buffer needs to be bigger to store this additional overhead.

For encryption, the input data will be sensitive data and so it shall be passed in transient memory.

The output buffer can be either in transient or persistent memory. When in persistent memory, the encrypted data is written atomically and the operation participates to the current transaction, if any .

Parameters:

- `inbuf` - the buffer with the plain input data. The input buffer needs to be located in transient memory.
- `inbufOff` - offset of the input data in the input buffer.
- `inbufLen` - size of the input data to encrypt.
- `outbuf` - the buffer the encrypted data is written to. The output buffer can be in non volatile or transient memory.
- `outbutOff` - offset of the encrypted data in the output buffer.

Returns:

- Returns `outbufLen` which is the length of the input data + 16 bytes of overhead.

Throws:

- `ArrayIndexOutOfBoundsException` - if the operation would cause access outside the bounds of an array.
- `SecurityException` - upon a processing error.
- `SystemException` - with the following reason code:
 - `NO_RESOURCE` - when the size of encrypted data exceeds the internal PUF workspace.

5.4.8 JCOPX RawComm

Tab. 5.45: APIs of package `com.nxp.id.jcopx.rawcomm`

Class	Method	Reference
<code>RawCommand</code>		See Section 5.4.8.1 - Class RawCommand
<code>RawSession</code>	<code>openRawSession()</code> <code>isRawSessionOpen()</code> <code>hasPostponedApuError()</code>	See Section 5.4.8.2 - Class RawSession

5.4.8.1 Class RawCommand

```
public interface RawCommand
```

This class is part of the `com.nxp.id.jcopx.rawcomm` package.

The `RawCommand` interface serves as a tagging interface to indicate that an applet supports non-ISO7816 incoming transmissions. The implementation of the interface is pre-condition to change the behavior of the communication.

To start sessions in raw communication mode, the API provided in `RawSession` (see Section 5.4.8.2 - [Class RawSession](#)) needs to be used.

5.4.8.2 Class RawSession

```
public final class RawSession
```

This class is part of the `com.nxp.id.jcopx.rawcomm` package.

Raw sessions allow transmissions to be received from external interfaces that do not comply with ISO7816-4 APDU [18] definitions. Full support of raw communication requires that the applet implements the tagging interface `RawCommand` (see Section 5.4.8.1 - [Class RawCommand](#)). The static methods of `RawSession` allow a raw communication to start.

A raw session will be closed implicitly by the OS when a valid ISO SELECT by DF NAME to any logical channel is detected during the reception of the APDU. In this scenario the raw session is terminated and the APDU is processed normally.

5.4.8.2.1 openRawSession()

```
public static void openRawSession()  
throws java.lang.SecurityException, javacard.framework.APDUException
```

This method opens a raw session for handling of non-ISO commands. It must be used in combination with the tagging interface `RawCommand` (see Section 5.4.8.1 - [Class RawCommand](#)).

After a raw session is opened, the communication behaviour is changed in the following way:

- The first chunk of incoming data is placed at offset 0 in the APDU buffer.
- All incoming data can be received using `APDU.setIncomingAndReceive` and `APDU.receiveBytes`.
- `APDU.getOffsetCdata` will return (short)-1.

- `APDU.getIncomingLength` will return in any case (short)-1 .
- `APDU.setIncomingAndReceive` will not handle the Le byte specially: the full number of bytes received is returned.
- `APDU.receiveBytes` receives all the incoming bytes, including Le if the command is formatted as an ISO APDU.
- `APDU.setOutgoing` or `APDU.setOutgoingAndSend` must still be called before responding with data.
- `APDU.setOutgoingLength` will not check the outgoing length as there is no Le in a raw command.
- `APDU.getLe` will return in any case (short)-1.
- The status word is not appended by the OS after leaving the process method.

Note that after this method returns, the current APDU is already part of the raw session.

For the protocol T=0 raw sessions are not allowed.

Throws:

`java.lang.SecurityException` - if

- the current context is not the context of the currently selected applet instance, or
- this method was not called, directly or indirectly, from the applet's process method (called directly by the Java Card runtime environment),
- the method is called during applet installation or deletion.

`javacard.framework.APDUException` - with reason code `APDUException.ILLEGAL_USE` if

- There is already a RAW session open.
- The currently selected applet does not implement the tagging interface `RawCommand`.
- The currently selected applet is not selected on the base logical channel.
- An applet is selected on another logical channel.
- One of the `APDU.setOutgoing()` or `APDU.setOutgoingNoChaining()` methods has already been invoked.
- The currently active communication is T=0.

5.4.8.2.2 isRawSessionOpen()

```
public static short isRawSessionOpen()
```

Check if raw command session is open. A call to this method is also allowed when the tagging interface `RawCommand` is not implemented by the currently selected applet..

Returns:

- `ConstantX.TRUE16` if session is open or `ConstantX.FALSE16` if session is not open.

5.4.8.2.3 hasPostponedAduError()

```
public static short hasPostponedAduError()
```

Check if last received command is not an ISO APDU. A call to this method is also allowed when no raw session is open but will respond with `ConstantX.FALSE16` in such a case.

Returns:

- `ConstantX.TRUE16` if last command was not an ISO APDU otherwise `ConstantX.FALSE16`.

5.4.9 JCOPX SecureBox

Secure Box provides the possibility to execute 3rd party native code in a securely encapsulated environment in JCOP 4 P71. This enables customers to develop algorithms that have a higher performance than a pure Java Card implementation.

The native library can be developed in C or assembly language for the hardware platform. To simplify the development process a native library development framework is available as part of the JCOP tool distribution. This framework provides the possibility to debug and analyse the implemented native code.

The following hardware components are available to be used by Secure Box to develop performance optimized algorithms:

- Transient memory (crypto RAM, CPU stack, transient state, APDU buffer)
- Persistent memory (read-only byte arrays from the applet, persistent state (read/write))
- A defined set of CPU peripherals
- Symmetric coprocessor
- FAME

5.4.9.1 Installing a Secure Box native library

The native library is always placed in Flash similar to a Java Card applet. In any case NXP needs to receive the native library bundled with a Java Card CAP file to prepare the integration into the JCOP OS. NXP will not modify or reverse-engineer the native library. The native library has to be installed in the card during OS initialization. For security reasons customers cannot upload a native library using the regular GlobalPlatform mechanisms.

A native library is associated to a particular Java Card package. Only applets from within this package can invoke the native library, which is then executed in the Java Card context of the package. There is a limitation of one native library per package.

The maximum size of the native library is limited by the available PHEAP size.

5.4.9.2 Java Card API

The native library can be invoked by calling the SecureBox API function `runNativeLib()`.

Tab. 5.46: Supported APIs of package `com.nxp.id.jcopx.securebox`

Class	Method	Comment
SecureBox	<code>runNativeLib</code>	See Section 5.4.9.3 - Class SecureBox
	<code>createPersistentSecureBoxArray</code>	
SecureBoxException	<code>getReason</code>	See Section 5.4.9.4 - Class SecureBoxException
	<code>setReason</code>	
	<code>throwit</code>	

5.4.9.3 Class SecureBox

```
public final class SecureBox extends Object
```

This class is part of the `com.nxp.id.jcopx.securebox` package.

5.4.9.3.1 `runNativeLib()`

```
public static short runNativeLib(  
short functionID,  
byte[] persistentArray0,  
byte[] persistentArray1,  
byte[] persistentArray2,  
byte[] persistentArray3,
```

```

byte[] apduBuffer,
byte[] ramInitializationData,
short ramInitializationDataOffset,
short ramInitializationDataLength,
byte[] resultArray,
short resultArrayOffset)
throws SecureBoxException, ArrayIndexOutOfBoundsException, NullPointerException
  
```

Runs a function in the native library associated to the caller's CAP file.

When this method is called, the following operations take place:

- The JCRE determines if there is a native library to call. The library needs to be associated to the CAP file of the caller's context. If the CAP file is not a Secure Box CAP file and has no associated native library, then a `SecureBoxException` is thrown with reason `NO_NATIVE_LIBRARY`. Note that the firewall context of the caller might be a different package than the caller's code location. This can happen in particular in the case where the caller CAP file is a library CAP file.
- The JCRE then checks the parameters:
 - The persistent array parameters `persistentArray0` to `persistentArray3` must either be null or must belong to the caller's firewall context, otherwise a `SecurityException` is thrown. Furthermore, the arrays must have been allocated with the method `createPersistentSecureBoxArray(short)` otherwise a `SecureBoxException` is thrown with reason `ARRAY_IS_NOT_MAPABLE`.
 - Then, the JCRE checks the parameters `ramInitializationData`, `ramInitializationDataOffset`, and `ramInitializationDataLength`:
 if `ramInitializationDataLength` is zero, it means there is no data to copy and the other parameters are ignored.
 If data for initializing the Secure Box accessible memory location for crypto RAM is provided, `ramInitializationData` must be a byte array accessible to the caller's firewall context, otherwise a `SecurityException` is thrown. If `ramInitializationData` is not null but `ramInitializationDataLength` is not zero, a `NullPointerException` is thrown. The offset and length must describe a valid range of bytes in the array or an `ArrayIndexOutOfBoundsException` is thrown.
 If the initialization data are too big for the native library, a `SecureBoxException` with reason code `INITIALIZATION_OVERFLOW` is thrown.
 - The JCRE checks the parameters `resultArray` and `resultArrayOffset`: if `resultArray` is null the caller indicates that it is not interested in the result data. If `resultArray` is not null then it must be accessible to the caller's firewall context, otherwise a `SecurityException` is thrown. The offset must be within the array, otherwise an `ArrayIndexOutOfBoundsException` is thrown.

- The JCRE will map the APDU buffer to the accessible memory locations of Secure Box only, if the reference of the APDU buffer is provided to the `RunNativeLib` method call. If the provided reference is not the APDU buffer, a `SecureBoxException` with reason code `ARRAY_IS_NOT_MAPABLE` is thrown.
- The JCRE prepares the execution of the native library by mapping the library code, the parameter arrays, initializing the workspace, etc. It then calls the native library.
- The native library is called.
- If the native library returns with an exception byte `X`, then a `SecureBoxException` is thrown with the reason code `(NATIVE_LIB_RUNTIME|X)` and no data is copied back.
- If the native library returns normally, then the JCRE attempts to copy back the result data into the `resultArray`:
 - if there is no result data, nothing is done;
 - if `resultArray` is `null` nothing is copied and the result data is ignored.
 - Otherwise, the result data must fit into the `resultArray` at offset `resultArrayOffset` or a `SecureBoxException` with reason `RESULT_OVERFLOW` is thrown.
 - The result data are written as if `Util.arrayCopy(byte[], short, byte[], short, short)` or `Util.arrayCopyNonAtomic(byte[], short, byte[], short, short)` were used. The method to be used is defined by native library.
- Finally, in case of a normal return, this method returns the `short` value returned by the native library.

Parameters:

- `functionID` - This function ID is passed to the native library. The library can decide with this parameter, what branch it will access. It is not mandatory for the native library to make use of this parameter.
- `persistentArray0` - First persistent array that can be accessed by the native library. Must be either `null` or have been allocated with the method `createPersistentSecureBoxArray`.
- `persistentArray1` - Second persistent array that can be accessed by the native library. Must be either `null` or have been allocated with the method `createPersistentSecureBoxArray`.
- `persistentArray2` - Third persistent array that can be accessed by the native library. Must be either `null` or have been allocated with the method `createPersistentSecureBoxArray`.
- `persistentArray3` - Fourth persistent array that can be accessed by the native library. Must be either `null` or have been allocated with the method `createPersistentSecureBoxArray`.
- `APDUBuffer` - The reference to the APDU buffer. Must be either `null` or is the reference to the APDU buffer.

- `ramInitializationData` - Array containing initial data to be copied in the internal Secure Box library workspace.
- `ramInitializationDataOffset` - Offset of the initialization data in `ramInitializationData`.
- `ramInitializationDataLength` - Number of bytes to copy from `ramInitializationData`.
- `resultArray` - Array where the library output data is written. The actual length of the output data is provided by the native library.
- `resultArrayOffset` - Offset in `resultArray1` array.

Returns:

- The short value returned by the native library.

Throws:

- `NullPointerException` - see description above.
- `ArrayIndexOutOfBoundsException` - see description above.
- `com.nxp.id.jcopx.securebox.SecureBoxException` - see description above.
- `SecurityException` - see description above.

5.4.9.3.2 createPersistentSecureBoxArray()

```
public native static createPersistentSecureBoxArray(short size)
throws java.lang.NegativeArraySizeException, javacard.framework.SystemException
```

Allocate a persistent byte array suitable for passing as one of the `persistentArrayX` parameter to the method `runNativeLib`.

The returned byte array is usable exactly like a normal persistent byte array but is internally allocated in a way that allows sharing with a native library..

Parameters:

- `size` - The size of the byte array in Bytes.

Returns:

- The allocated byte array.

Throws:

- `java.lang.NegativeArraySizeException` - on attempt to create an array with negative size.
- `javacard.framework.SystemException` - with reason code `SystemException.NO_RESOURCE` if not enough resources are available for the allocation.

5.4.9.4 Class SecureBoxException

`public class SecureBoxException extends javacard.framework.CardRuntimeException`

This class is part of the `com.nxp.id.jcopx.securebox` package.

The `SecureBoxException` is used by the Secure Box implementation to indicate invalid usage. It defines the following constants:

INITIALIZATION_OVERFLOW

`public static final short INITIALIZATION_OVERFLOW = (short) 0x3BD0;`

The number of bytes to initialize the native library workspace is too high.

RESULT_OVERFLOW

`public static final short RESULT_OVERFLOW = (short) 0x3BC2;`

The result data located in RAM that needs to be copied back from the native library does not fit in the result array.

ARRAY_IS_NOT_MAPABLE

`public static final short ARRAY_IS_NOT_MAPABLE = (short) 0x3BAB;`

The array handed over to `runNativeLib` has not been allocated by `createPersistentSecureBoxArray` or the APDU buffer provided as parameter to `runNativeLib` is not the reference to the APDU buffer.

UNKNOWN

`public static final short UNKNOWN = (short) 0x3B2C;`

Unknown error condition.

NOT_AVAILABLE

`public static final short NOT_AVAILABLE = (short) 0x4B0C;`

Indicates that either Secure Box is disabled or not available on the product.

NO_NATIVE_LIBRARY

`public static final short NO_NATIVE_LIBRARY = (short) 0x3B4B;`

The caller is not a Secure Box CAP file and has no native library.

HW_VIOLATION

```
public static final short HW_VIOLATION = (short) 0x2C33;
```

The native library raised a hardware exception during execution.

NATIVE_LIB_RUNTIME

```
public static final short NATIVE_LIB_RUNTIME = (short) 0x3300;
```

Exception triggered by the native library. The high byte is fixed (0x33), the lower byte is set by the native library.

5.4.9.4.1 getReason()

```
public short getReason()
```

Gets the reason code.

The purpose of this method is to return the reason code of the `SecureBoxException`. If the exception instance is the singleton from the exception array stored in the lower software layer, then the reason code is taken from the reason array stored in that layer.

If this exception is its own instance then the reason code is taken from the instance variable.

Returns:

- The reason code of the singleton instance or the reason code of this instance.

5.4.9.4.2 setReason()

```
public void setReason(final short pReason)
```

Sets the reason code.

The purpose of this method is to set the reason code of the `SecureBoxException`.

If the exception instance is the singleton from the exception array stored in the lower software layer then the reason code is assigned to the reason array stored in the lower software layer.

If this exception is its own instance then the reason code is taken from the instance variable.

Parameters:

- `pReason` - the reason for exception.

5.4.9.4.3 `throwIt()`

```
public static void throwIt(short pReason)
throws SystemException
```

Throws the Java Card runtime environment-owned instance of `SecureBoxException` with the specified reason. Java Card runtime environment-owned instances of exception classes are temporary Java Card runtime environment Entry Point Objects and can be accessed from any applet context. References to these temporary objects cannot be stored in class variables or instance variables or array components. See Runtime Environment Specification, Java Card Platform, Classic Edition [22], section 6.2.1 for details.

The purpose of this method is to invoke `JCSystem.throwIt(byte, short)` to throw the singleton instance of this exception.

Parameters:

- `short pReason` - the reason for the exception.

Throws:

- `SecureBoxException` - the singleton instance of this exception.

5.4.9.5 Native API - calls into the native library

The native code of a Secure Box native library is entered at the User Mode Vector 0. At this time the memory of the FRAM is initialized with data provided in the array `ramInitializationData` of the Java method call `runNativeLib`. The `SecureBoxNativeLibrary` has the ability to handle exceptions caused by system calls to functions provided by the Secure Box framework. Such exceptions will enter the `SecureBoxNativeLibrary` in User Mode Vector 1.

5.4.9.6 Native API - system calls

The Secure Box native library is allowed to call functions provided by the Secure Box framework. This functions are called system calls and will be handled by the JCOP OS. After a successful execution of the function, the function call returns back to the user mode. In case of an error, two scenarios are possible:

- Regular return of the function call with an error code. The native library needs to check the return value and react accordingly.
- Functions can detect problems in the execution and convert the error condition to an exception. This exception has to be implemented in the native library using the handler of User Mode Vector 1. In this scenario it is the sole responsibility of the native library to handle the native stack and react on the problem.

The following system calls are provided by the Secure Box framework:

Tab. 5.47: Secure Box system calls

Function	Parameter	Return value	Short description
Exception	uint16_t liExceptionType uint16_t liExceptionReason	Does not return	Leave the native library and raise Java exception.
Exit_OK	uint16_t riReturnValue	Does not return	Leave the native library and return to the applet.
OpenTransaction	void	void	Equal functionality to JCSys <code>tem.openTransaction</code> .
GetTransactionDepth	void	uint8_t	1 = transaction is open. 0 = no transaction is open <code>Transaction</code> . Equal to JCSys <code>tem.getTransactionDepth</code>
CommitTransaction	void	void	Equal to JCSys <code>tem.CommitTransaction</code> .
AbortTransaction	void	void	Equal to JCSys <code>tem.AbortTransaction</code> .
GetRemainingTransactionCapacity	void	uint16_t	Equal to JCSys <code>tem.getUnusedCommitCapacity</code> .
ArrayCopyToPersistentState	pcuint8_uni_t pSrc, uint16_t len, uint16_t offset	void	Copy data into the persistent state using transactions (similar to JCSys <code>tem.ArrayCopy</code>).
ArrayCopyToPersistentStateNonAtomic	pcuint8_uni_t pSrc, uint16_t len, uint16_t offset	void	Copy data into the persistent state using transactions (similar to JCSys <code>tem.ArrayCopyNonAtomic</code>).
ArrayCopyToPersistentStateAndCommit	pcuint8_uni_t pSrc, uint16_t len, uint16_t offset	void	Copy data into the persistent state using transactions (similar to JCSys <code>tem.ArrayCopy</code>) and commit the transaction afterwards.
ArrayFillToPersistentState	uint8_t val, uint16_t len, uint16_t offset	void	Write len bytes of value val into the persistent state within a transaction.
ArrayFillToPersistentStateNonAtomic	uint8_t val, uint16_t len, uint16_t offset	void	Write len bytes of value val into the persistent state not using the transaction system.
ArrayFillToPersistentStateAndCommit	uint8_t val, uint16_t len, uint16_t offset	void	Write len bytes of value val into the persistent state within a transaction and commit the transaction afterwards.
OpenResources	uint16_t resToOpen	uint16_t openedRes	Request access to platform resources
CloseResources	uint16_t resToClose	uint16_t closedRes	Free up platform resource

Continued on the next page.

Tab. 5.47 – Continued from the previous page.

Function	Parameter	Return value	Short description
GetResourceStatus	uint16_t * permResLocked-ByOther	uint16_t re-sources	Get the information on available resources

5.4.9.7 Security

For a secure implementation of the native library it is required to respect the user guidance manual of the hardware. The JCOP platform evaluation covers the strict split between JCOP internal resources and resources assigned to the native library.

Note: Secure Box will not intercept counter measures of the hardware that will lead to an attack detection and subsequently to a decrement of the attack counter. Please refer to the Hardware User Manual to identify such situations (for example MMU access violations). As the native library is out of the control of NXP and the JCOP OS, NXP cannot take over responsibility for products that get in-reactive due to usage of Secure Box.

5.4.10 JCOPX Security

In addition to the standard Java Card APIs JCOP supports additional APIs and cryptographic algorithms. The following chapter gives information on functionality supported by JCOP through the JCOPX Security package.

Tab. 5.48: Supported APIs of package `com.nxp.id.jcopx.security`

Class	Method	Reference
BankingPIN	setTryLimit()	5.4.10.1 - Class BankingPIN
CipherX	getInstance()	5.4.10.2 - Class CipherX
ConstantX		5.4.10.3 - Class ConstantX
CryptoBaseX	doFinal() sign() verify() shaOperation()	5.4.10.4 - Class CryptoBaseX
KeyAgreementX	getInstance()	5.4.10.5 - Class KeyAgreementX
SignatureX	getInstance()	5.4.10.7 - Class SignatureX

5.4.10.1 Class BankingPIN

```
public final class BankingPIN extends Object
```

This class is part of the `com.nxp.id.jcopx.security` package.

The `BankingPIN` class provides the method `setTryLimit()` to set the try limit of an `OwnerPIN` after the `OwnerPIN` has been created.

5.4.10.1.1 setTryLimit()

```
public static void setTryLimit(OwnerPIN pin, byte tryLimit)
throws PINException
```

Sets the try limit of the given OwnerPIN to the new value.

This method never changes the **blocked state** of the PIN object, even when the new try limit is greater than the old try limit:

- Example: If a PIN has been created with PTL_old=3 and there were 3 failed attempts to validate the PIN, then the PIN becomes blocked and OwnerPIN.getTriesRemaining() returns 0. If setTryLimit() is now called with PTL_new=5, then the PIN remains blocked and OwnerPIN.getTriesRemaining() still returns 0. However, the PIN try limit will still be set to 5 so that after the method OwnerPIN.resetAndUnblock() was called, OwnerPIN.getTriesRemaining() will return 5, not 3.

This method affects the current PIN **try counter** (the number of remaining tries) as follows:

- If the new PIN try limit (PTL_new) is greater than or equal to the current try limit (PTL_old) and the PIN is currently not blocked, then the number of remaining tries is incremented by PTL_new-PTL_old:
 - Example: If the PIN has been created with PTL_old=3 and there was one failed attempt to validate the PIN, then the number of remaining tries as returned by OwnerPIN.getTriesRemaining() is equal to 2. If setTryLimit() is called with PTL_new=5, then the number of remaining tries will be 4.
- If the new PIN try limit (PTL_new) is less than the current try limit (PTL_old), but greater than the number of failed attempts, then the PIN try counter is decremented by PTL_old-PTL_new:
 - Example: If the PIN has been created with PTL_old=5 and there was one failed attempt to validate the PIN, then the number of remaining tries as returned by OwnerPIN.getTriesRemaining() is equal to 4. If setTryLimit() is called with PTL_new=3, then the number of remaining tries will be 2.
- If the new PIN try limit (PTL_new) is less than the current try limit (PTL_old), and less than or equal to the number of failed attempts, then the PIN try counter is set to 0 and the PIN is blocked:
 - Example: If the PIN has been created with PTL_old=5 and there were 3 failed attempt to validate the PIN, then the number of remaining tries as returned by OwnerPIN.getTriesRemaining() is equal to 2. If setTryLimit() is called with PTL_new=3, then the PIN is blocked and the number of remaining tries will be 0. If setTryLimit() is called a second time with PTL_new=4, then the PIN remains blocked and the number of remaining tries is still 0.

The changed try limit is valid immediately when returning from this method.

Parameters:

- `pin` - the OwnerPIN of which the new PIN try limit must be set.
- `tryLimitVal` - value of the new PIN try limit.

Throws:

- `NullPointerException` - if `pin` is null.
- `SecurityException` - if the specified `pin` is not accessible to the caller context.
- `PINException` - with the following reason code:
 - `PINException.ILLEGAL_VALUE` - if the try limit to set is smaller than or equal to 0.

5.4.10.2 Class CipherX

```
public final class CipherX extends Object
```

This class is part of the `com.nxp.id.jcopx.security` package.

The `CipherX` class defines static methods which can be used to generate Java Card objects that implement JCOP-specific Cipher objects.

The `getInstance` methods of this class return Cipher objects which can be used like any other Cipher object that implement algorithms defined in the Java Card API.

The following cipher algorithms are supported by `CipherX`:

ALG_KOREAN_SEED_CBC_NRPAD

```
public static final byte ALG_KOREAN_SEED_CBC_NRPAD = (byte) 0x72;
```

`ALG_KOREAN_SEED_CBC_FPAD` cipher algorithm option for the `getInstance(byte, byte, byte, boolean)` method's `cipherAlgorithm` parameter. The algorithm generates a Korean SEED cipher using Korean Seed in CBC mode.

ALG_KOREAN_SEED_ECB_NRPAD

```
public static final byte ALG_KOREAN_SEED_ECB_NRPAD = (byte) 0x73;
```

`ALG_KOREAN_SEED_ECB_FPAD` cipher algorithm option for the `getInstance(byte, byte, byte, boolean)` method's `cipherAlgorithm` parameter. The algorithm generates Korean SEED cipher using Korean Seed in ECB mode.

ALG_KOREAN_SEED_CBC_PKCS7

```
public static final byte ALG_KOREAN_SEED_CBC_PKCS7 = (byte) 0x76;
```

ALG_KOREAN_SEED_CBC_PKCS7 cipher algorithm option for the getInstance(byte, byte, byte, boolean) method's cipherAlgorithm parameter. The algorithm generates Korean SEED cipher using Korean Seed in CBC mode.

ALG_KOREAN_SEED_ECB_PKCS7

```
public static final byte ALG_KOREAN_SEED_ECB_PKCS7 = (byte) 0x77;
```

ALG_KOREAN_SEED_ECB_PKCS7 cipher algorithm option for the getInstance(byte, byte, byte, boolean) method's cipherAlgorithm parameter. The algorithm generates Korean SEED cipher using Korean Seed in ECB mode.

ALG_AES_CBC_PKCS7

```
public static final byte ALG_AES_CBC_PKCS7 = (byte) 0x78;
```

ALG_AES_CBC_PKCS7 cipher algorithm option for the getInstance(byte, byte, byte, boolean) method's cipherAlgorithm parameter. The algorithm generates a DES cipher using AES in CBC mode.

ALG_AES_ECB_PKCS7

```
public static final byte ALG_AES_ECB_PKCS7 = (byte) 0x79;
```

ALG_AES_ECB_PKCS7 cipher algorithm option for the getInstance(byte, byte, byte, boolean) method's cipherAlgorithm parameter. The algorithm generates a DES cipher using AES in ECB mode.

ALG_DES_CBC_PKCS7

```
public static final byte ALG_DES_CBC_PKCS7 = (byte) 0x80;
```

ALG_DES_CBC_PKCS7 cipher algorithm option for the getInstance(byte, byte, byte, boolean) method's cipherAlgorithm parameter. The algorithm generates a DES cipher using DES in CBC mode.

ALG_DES_ECB_PKCS7

```
public static final byte ALG_DES_ECB_PKCS7 = (byte) 0x81;
```

ALG_DES_ECB_PKCS7 cipher algorithm option for the getInstance(byte, byte, byte, boolean) method's cipherAlgorithm parameter. The algorithm generates a DES cipher using DES in ECB mode.

5.4.10.2.1 getInstance() - combined cipher and padding algorithm

```
public static final Cipher getInstance(byte algorithm, boolean externalAccess)
throws CryptoException
```

Creates a Cipher object instance of the selected algorithm. It invokes the getInstance(byte, byte, byte, boolean) method.

Parameters:

- `algorithm` - the desired CipherX algorithm. Valid codes listed in `ALG_*` constants above, for example `ALG_KOREAN_SEED_CBC_NRPAD`.
- `externalAccess` - true indicates that the instance will be shared among multiple applet instances and that the Cipher instance will also be accessed (via a Shareable interface) when the owner of the Cipher instance is not the currently selected applet. If true, the implementation must not allocate `CLEAR_ON_DESELECT` transient space for internal data..

Returns:

- The Cipher object instance of the requested algorithm.

Throws:

- `CryptoException` - with the following reason code:
 - `CryptoException.NO_SUCH_ALGORITHM` - if the requested algorithm or shared access mode is not supported.

5.4.10.2.2 getInstance() - separate cipher and padding algorithm

```
public static final Cipher getInstance(byte cipherAlgorithm, byte paddingAlgorithm,
boolean externalAccess)
throws CryptoException
```

Creates a Cipher object instance with the selected cipher algorithm and padding algorithm.

Note:

- When there is no discrete message digest algorithm, use the `MessageDigest.ALG_NULL` option for the message digest algorithm.

- When the padding algorithm is built into the cipher algorithm, use the `Cipher.PAD_NULL` option for the padding algorithm.

Parameters:

- `algorithm` - the desired cipher algorithm, either `Cipher.CIPHER_KOREAN_SEED_CBC` or `Cipher.CIPHER_KOREAN_SEED_ECB..`
- `paddingAlgorithm` - the desired padding algorithm, either `SignatureX.NRPAD` or `SignatureX.PKCS7..`
- `externalAccess` - true indicates that the instance will be shared among multiple applet instances and that the `Cipher` instance will also be accessed (via a `Shareable` interface) when the owner of the `Cipher` instance is not the currently selected applet. If true the implementation must not allocate `CLEAR_ON_DESELECT` transient space for internal data.

Returns:

- The `Cipher` object instance of the requested algorithm.

Throws:

- `CryptoException` - with the following reason code:
 - `CryptoException.NO_SUCH_ALGORITHM` - if the requested cipher algorithm or padding algorithm or their combination or the requested shared access mode is not supported.

5.4.10.3 Class ConstantX

`public final class ConstantX extends Object`

This class is part of the `com.nxp.id.jcopx.security` package.

The `ConstantX` class defines the following JCOPX constants:

TRUE16

`public static final short TRUE16 = (short) 0x5A5A;`

16 bit secure return value true.

FALSE16

`public static final short FALSE16 = (short) 0xA5A5;`

16 bit secure return value false.

5.4.10.4 Class CryptoBaseX

```
public final class CryptoBaseX extends Object
```

This class is part of the `com.nxp.id.jcopx.security` package.

`CryptoBaseX` defines static methods which can be used to provide symmetric and asymmetric services (cipher and signature functionality).

The main purpose of this class is to reduce memory footprint and improve performance. The services provided by the `CryptoBaseX` class can be used by the Global Platform Framework and some internal applets. `CryptoBaseX` provides the necessary buffers and general methods.

This class also connects the upper software layer with the lower software layer by invoking the `ph0sNativeCrypto` methods.

It offers:

- Static methods to connect the Java and the native world. It basically comes down to wrap the Java calls to the `ph0sNativeCrypto` functions.
- Methods implementing common functionality for classes using cryptographic operations.

Constants used in the upper and lower software layer are defined in `ClWrapperConstants`.

This class supports the algorithms listed below.

Cipher algorithms:

ALG_AES_CBC_ISO9797_M1

```
public static final byte ALG_AES_CBC_ISO9797_M1 = (byte) 0x22;
```

AES in CBC mode with M1 padding (UICC req).

ALG_AES_CBC_ISO9797_M2

```
public static final byte ALG_AES_CBC_ISO9797_M2 = 23;
```

AES in CBC mode with M2 padding (UICC req).

ALG_AES_BLOCK_128_CBC_NOPAD

```
public static final byte ALG_AES_BLOCK_128_CBC_NOPAD = (byte) 0x13;
```

AES in CBC mode with no padding.

ALG_DES_CBC_NOPAD

```
public static final byte ALG_DES_CBC_NOPAD = 01;
```

DES in CBC mode with no padding.

ALG_DES_ECB_NOPAD

```
public static final byte ALG_DES_ECB_NOPAD = (byte) 0x05;
```

DES in ECB mode with no padding.

ALG_DES_ECB_ISO9797_M1

```
public static final byte ALG_DES_ECB_ISO9797_M1 = 02;
```

DES in CBC mode with M1 padding (UICC req).

ALG_DES_CBC_ISO9797_M2

```
public static final byte ALG_DES_CBC_ISO9797_M2 = (byte) 0x03;
```

DES in ECB mode with M2 padding (UICC required).

ALG_DES_CMAC8

```
public static final byte ALG_DES_CMAC8 = (byte) 0x7A;
```

Generating 8-Byte MAC with AES according to CMAC algorithm.

ALG_RSA_NOPAD

```
public static final byte ALG_RSA_NOPAD = 12;
```

RSA with no padding.

ALG_RSA_PKCS1

```
public static final byte ALG_RSA_PKCS1 = 10;
```

RSA with PKCS1 padding.

Signature algorithms:**ALG_DES_MAC4_ISO9797_M2**

```
public static final byte ALG_DES_MAC4_ISO9797_M2 = 5;
```

Generates an 4-Byte MAC with DES and M2 padding.

ALG_DES_MAC4_ISO9797_1_M2_ALG3

```
public static final byte ALG_DES_MAC4_ISO9797_1_M2_ALG3 = 19;
```

Generates an 4-Byte MAC with DES and M2 padding.

ALG_DES_MAC4_ISO9797_M1

```
public static final byte ALG_DES_MAC4_ISO9797_M1 = 3;
```

Generates an 4-Byte MAC with DES and M1 padding.

ALG_DES_MAC4_ISO9797_1_M1_ALG3

```
public static final byte ALG_DES_MAC4_ISO9797_1_M1_ALG3 = 47;
```

Generates an 4-Byte MAC with DES and M1 padding.

ALG_DES_MAC8_ISO9797_M2

```
public static final byte ALG_DES_MAC8_ISO9797_M2 = 06;
```

Generates an 8-Byte MAC with DES and M2 padding.

ALG_DES_MAC8_ISO9797_1_M2_ALG3

```
public static final byte ALG_DES_MAC8_ISO9797_1_M2_ALG3 = (byte) 0x20;
```

Generates an 8-Byte MAC with DES and M2 padding.

ALG_DES_MAC8_ISO9797_M1

```
public static final byte ALG_DES_MAC8_ISO9797_M1 = (byte) 0x04;
```

Generates an 8-Byte MAC with DES and M1 padding.

ALG_DES_MAC8_ISO9797_1_M1_ALG3

```
public static final byte ALG_DES_MAC8_ISO9797_1_M1_ALG3 = 48;
```

Generates an 8-Byte MAC with DES and M1 padding.

ALG_AES_CMAC16

```
public static final byte ALG_AES_CMAC16 = (byte) 0x66;
```

Signature algorithm ALG_AES_CMAC16 generates a 16-Byte Message Authentication Code (MAC) using AES key according to CMAC algorithm NIST SP_800-38B [2].

ALG_RSA_SHA_PKCS1

```
public static final byte ALG_RSA_SHA_PKCS1 = 11;
```

Generating hash using SHA1 and performing an RSA operation.

ALG_RSA_SECURE_SHA_PKCS1

```
public static final byte ALG_RSA_SECURE_SHA_PKCS1 = (byte) 0xF1;
```

Signature algorithm ALG_RSA_SECURE_SHA_PKCS1 generates a 20-byte SHA digest which is protected against side channel analysis, pads the digest according to the PKCS#1 (v1.5) scheme, and encrypts it using

RSA.

Tab. 5.49: Cryptographic Algorithms supported by CryptoBaseX

Algorithm	Ciphering	Signature	
		generation	verification
ALG_RSA_NOPAD	yes	no	no
ALG_RSA_PKCS1	yes	no	no
ALG_RSA_SHA_PKCS1	no	yes	yes
ALG_ECDSA_SHA_256	no	no	yes
ALG_DES_CBC_NOPAD	yes	no	no
ALG_DES_ECB_NOPAD	yes	no	no
ALG_DES_CBC_ISO9797_M2	yes	no	no
ALG_DES_MAC8_ISO9797_1_M1_ALG3	no	yes	yes
ALG_DES_MAC8_ISO9797_1_M2_ALG3	no	yes	yes
ALG_DES_MAC8_ISO9797_M2	no	yes	yes
ALG_AES_BLOCK_128_CBC_NOPAD	yes	no	no
ALG_AES_CBC_ISO9797_M2	yes	no	no
ALG_AES_CMAC16	no	yes	no

The method `CryptoBaseX.shaOperation()` supports the following hashing algorithms:

- `MessageDigest.ALG_SHA`
- `MessageDigest.ALG_SHA_224`
- `MessageDigest.ALG_SHA_256`
- `MessageDigest.ALG_SHA_384`
- `MessageDigest.ALG_SHA_512`

5.4.10.4.1 doFinal() - without ICV

```
public static short doFinal(Key theKey, byte algorithm, byte theMode, byte[] inBuff,
short inOffset, short inLength, byte[] outBuff, short outOffset)
throws CryptoException, NullPointerException, ArrayIndexOutOfBoundsException
```

The purpose of this method is to invoke `CryptoBaseX.doFinal(Key, byte, byte, byte[], short, short, byte[], short, short, byte[], short)` with IV set to null or IV as set by `setIV()`.

After the operation is finished, D.CRYPTO is cleared.

Parameters:

- `theKey` - the key object to use for cipher functionality.
- `algorithm` - the desired algorithm from the available cipher algorithms of this class. Valid codes listed in the `ALG_*` constants above, for example, `ALG_DES_CBC_NOPAD`.
- `theMode` - indicates the encryption/decryption to be conformed.
- `inBuff` - the input buffer containing data to be encrypted/decrypted.
- `inOffset` - the offset into the input buffer at which to begin encryption/decryption.
- `inLength` - the byte length to be encrypted/decrypted.
- `outBuff` - the output buffer where the resulting data is written. May be the same as the input buffer.
- `outOffset` - the offset into the output buffer where the resulting output data begins.

Returns:

- Number of bytes output in `outBuff`.

5.4.10.4.2 doFinal() - with ICV

```
public static short doFinal(Key theKey, byte algorithm, byte theMode, byte[] inBuff,
short inOffset, short inLength, byte[] ivBuffer, short ivBufferOffset,
short ivBufferLength, byte[] authData, short authDataOffset, short authDataLength,
byte[] outBuff, short outOffset, byte[] tagBuff, short tagOffset, short tagLength)
throws CryptoException, NullPointerException, ArrayIndexOutOfBoundsException
```

The purpose of this method is to implement the AES GCM (Galois/Counter Mode). The method will take special care for the differences between other symmetrical algorithms and GCM. As methods from other algorithms are re-used, the GCM special parameters are stored separately in the native layer.

Parameters:

- `theKey` - see `doFinal` implementation above.
- `algorithm` - see above, but will extent the list of algorithms by `ALG_AES_GCM`.
- `theMode` - see `doFinal` implementation above.

- `inBuff` - see `doFinal` implementation above.
- `inOffset` - see `doFinal` implementation above.
- `inLength` - see `doFinal` implementation above.
- `ivBuffer` - see `doFinal` implementation above.
- `ivBufferOffset` - Offset inside `ivBuffer`.
- `ivBufferLength` - Length of `ivBuffer`.
- `authData` - the auth data buffer.
- `authDataOffset` - the offset into the `authData` buffer.
- `authDataLength` - the byte length of used `authData` buffer.
- `outBuff` - see `doFinal` implementation above.
- `outOffset` - see `doFinal` implementation above.
- `tagBuff` - for encryption this is the output buffer for the generated authentication tag. For decryption this is the input buffer for the authentication tag.
- `tagOffset` - the offset into the `tagBuffer`.
- `tagLength` - the byte length of the tag, can be one of the range from `AES_GCM_TAGLEN_96` to `AES_GCM_TAGLEN_128`.

5.4.10.4.3 `doFinal()`

```
public static short doFinal(Key theKey, byte algorithm, byte theMode, byte[] inBuff,  
short inOffset, short inLength, byte[] ivBuffer, short ivBufferOffset,  
short ivBufferLength, byte[] outBuff, short outOffset)  
throws CryptoException
```

The purpose of this method is to implement the functionality of javacard API `doFinal` method along with additional checks needed to validate the key, algorithm and mode. This method first checks the input parameters to have allowed values. It then invokes `CryptoBaseX.doFinalSymmetric(key, byte, byte, byte[], short, short, byte[], short)` for symmetric algorithms or `CryptoBaseX.doFinalAssymmetricRSA(key, byte, byte, byte[], short, short, byte[], short)` for asymmetric algorithm. Clears the used message buffer securely.

Parameters:

- `theKey` - Reference to the key to be used for cipher functionality.
- `algorithm` - the desired algorithm from the available cipher algorithms of this class. Valid codes listed in `ALG_*` constants above, for example, `ALG_DES_CBC_NOPAD`.
- `theMode` - It indicates encryption/decryption to be performed.
- `inBuff` - the input buffer containing data to be encrypted/decrypted.
- `inOffset` - the offset into the input buffer to begin encryption/decryption.
- `inLength` - the byte length of data to be encrypted/decrypted.
- `ivBuffer` - the buffer containing the initial vector.
- `ivBufferOffset` - the offset into the buffer containing the initial vector.
- `ivBufferLength` - length of the initial vector.
- `outBuff` - output buffer where resulting output data is written the output buffer, may be the same as the input buffer.
- `outOffset` - the offset into the output buffer where the resulting output data begins.

Throws:

- `CryptoException` - with the following reason code:
 - `Exception.SINGLETON_IDX_NULL_POINTER_EXCEPTION` - If `theKey/inBuff/outBuff` is null.
 - `CryptoException.UNINITIALIZED_KEY` - if the key is not initialized.
 - `CryptoException.ILLEGAL_VALUE` - if the mode is neither `Cipher.MODE_ENCRYPT` or `Cipher.MODE_DECRYPT`, or if the key is inconsistent with the cipher algorithm.
 - `Exception.SINGLETON_IDX_ARRAY_INDEX_OUT_OF_BOUNDS_EXCEPTION` - if the offset of input/output array is negative or array length is less than the input length.
 - `CryptoException.NO_SUCH_ALGORITHM` - if the algorithm is not one of the supported algorithms.
 - `CryptoException.ILLEGAL_USE` - if the input data to be encrypted is not block aligned and no padding is supported or If the input data to be decrypted is not block aligned.
 - `SecurityException` - if required access to `theKey/inBuff/outBuff/ivBuffer` is not allowed in current context.

Returns:

- Number of bytes output in `outBuff`.

5.4.10.4.4 setIV()

```
public static void setIV(byte[] bArray, short bOff, short bLen)
```

Deprecated. The IV is passed as a parameter to `doFinal`, `sign`, `verify`. Therefore `setIV` must not be used as it has no functionality anymore. This method will throw an exception on newer JCOP products. It is not removed from the API due to compatibility issues. The purpose of this method is to copy the initial vector to the internal `ivBuffer` for a subsequent `doFinal` operation. The input array containing Initial Chaining Vector is validated then if the copy of IV is not successful, Attack Counter is triggered.

5.4.10.4.5 sign() - without ICV

```
public static short sign(Key theKey, byte algorithm, byte[] inBuff, short inOffset,
short inLength, byte[] outBuff, short outOffset)
throws CryptoException
```

The purpose of this method is to implement the functionality of `javacard.security.Signature.sign()` in a resource efficient manner, that is without the need to create a dedicated `Signature` instance.

Note:

- For symmetric cryptographic operations a default ICV of '00' .. '00' is used.
- If the `inOffset` or `outOffset` parameter is negative, an `ArrayIndexOutOfBoundsException` exception is thrown.
- If `inOffset+inLength` is greater than `inBuff.length`, the length of the `inBuff` array, an `ArrayIndexOutOfBoundsException` exception is thrown.
- If the length of the result of the cryptographic operation is greater than `outBuff.length-outOffset-1`, an `ArrayIndexOutOfBoundsException` exception is thrown.
- If `inBuff` or `outBuff` parameter is null a `NullPointerException` exception is thrown.

Parameters:

- `theKey` - the key object to use for signature generation.
- `algorithm` - the desired signature algorithm.
- `inBuff` - the input buffer of data to be signed.

- `inOffset` - the offset into the input buffer at which to begin signature generation.
- `inLength` - the byte length to sign.
- `outBuff` - the output buffer to store signature data. The output buffer may be the same as the input buffer.
- `outOffset` - the offset into the output buffer where the resulting signature begins.

Returns:

- Number of bytes of signature in `outBuff`.

Throws:

- `CryptoException` - with the following reason code:
 - `Exception.SINGLETON_IDX_NULL_POINTER_EXCEPTION` - if the `theKey/inBuff/outBuff` is null.
 - `CryptoException.UNINITIALIZED_KEY` - if the key is not initialized.
 - `CryptoException.ILLEGAL_VALUE` - if the key is inconsistent with the signature algorithm.
 - `Exception.SINGLETON_IDX_ARRAY_INDEX_OUT_OF_BOUNDS_EXCEPTION` - if the offset of input/output array is negative or if the array length is less than the input length.
 - `CryptoException.NO_SUCH_ALGORITHM` - if the algorithm is not one of the supported algorithms.
 - `SecurityException` - if access to `theKey/inBuff/outBuff` is not allowed in current context.

5.4.10.4.6 sign() - with ICV

```
public static short sign(Key theKey, byte algorithm, byte[] inBuff, short inOffset,
short inLength, byte[] ivBuffer, short ivBufferOffset, short ivBufferLength,
byte[] outBuff, short outOffset)
throws CryptoException, Exception, SecurityException
```

The purpose of this method is to implement the functionality of `javacard.security.Signature.sign()` with additional checks to validate the key and the algorithm. This method first checks the input parameters to have allowed values, then it invokes `CryptoBaseX.doFinalSymmetric(key, byte, byte, byte[], short, short, byte[], short)` for symmetric algorithms or `CryptoBaseX.doFinalAsymmetricRSA(key, byte, byte, byte[], short, short, byte[], short)` for asymmetric algorithm. Clears the used message buffer securely.

Parameters:

- `theKey` - the key object to use for signing.
- `algorithm` - the desired algorithm from the available signature algorithms of this class. Valid codes listed in `ALG_*` constants above, for example, `ALG_DES_MAC8_ISO9797_M2`.

- `inBuff` - the input buffer of data to be signed.
- `inOffset` - the offset into the input buffer at which to begin signature generation.
- `inLength` - the byte length to sign.
- `ivBuffer` - the buffer containing the initial vector.
- `ivBufferOffset` - the offset into the buffer containing the initial vector.
- `ivBufferLength` - length of the initial vector.
- `outBuff` - the output buffer to store signature data. The output buffer, may be the same as the input buffer.
- `outOffset` - the offset into the output buffer where the resulting signature begins.

Returns:

- Number of bytes of signature in `outBuff`.

Throws:

- `CryptoException` - with the following reason code:
 - `Exception.SINGLETON_IDX_NULL_POINTER_EXCEPTION` - if the `theKey/inBuff/outBuff` is null.
 - `CryptoException.UNINITIALIZED_KEY` - if the key is not initialized.
 - `CryptoException.ILLEGAL_VALUE` - if the key is inconsistent with the signature algorithm.
 - `CryptoException.NO_SUCH_ALGORITHM` - if the algorithm is not one of the supported algorithms.
- `Exception` - with the following reason code:
 - `Exception.SINGLETON_IDX_NULL_POINTER_EXCEPTION` - if the `theKey/inBuff/outBuff` is null.
 - `Exception.SINGLETON_IDX_ARRAY_INDEX_OUT_OF_BOUNDS_EXCEPTION` - if the offset of input/output array is negative or if the array length is less than the input length.
- `SecurityException` - if access to `theKey/inBuff/outBuff` is not allowed in current context.

5.4.10.4.7 sign() - Session key

```
public static short sign(byte[] theKey, short keyOffset, short keyLength, byte algorithm,
byte[] inBuff, short inOffset, short inLength, byte[] outBuff, short outOffset)
throws CryptoException, NullPointerException, ArrayIndexOutOfBoundsException
```

The purpose of this method is to implement the functionality of `javacard.security.Signature.sign()` using an unprotected session key passed in plain.

Because this method uses the key material out of a plain byte array, the method cannot check the integrity of the provided key data. It should be noted that the platform does not guarantee confidentiality on byte arrays. Typically this API is used with session keys of limited life time.

For use cases where the platform needs to ensure confidentiality or integrity on the key material, appropriate APIs that deal with key objects shall be used.

This method is limited to the algorithms `CryptoBaseX.ALG_DES_MAC8_IS09797_1_M2_ALG3` and `CryptoBaseX.ALG_DES_MAC8_IS09797_1_M1_ALG3`.

This method first checks the input parameters to have allowed values, then it invokes `CryptoBaseX.signSymmetricFinal(byte[], short, short, byte, byte[], short, short, byte[], short, short)`. The key data is not further copied or modified within the API for the crypto graphic operation. After the sign operation is finished, `D.CRYPTO` is cleared.

Parameters:

- `theKey` - the array containing the key to be used.
- `keyOffset` - the offset into the given key array to the first byte of the key.
- `keyLength` - the length of the given key in bytes, starting from given key offset.
- `algorithm` - the algorithm of the requested signature.
- `inBuff` - the array containing the data to be signed.
- `inOffset` - the offset to the data to be signed in the data array.
- `inLength` - the length of the data to be signed starting from given data offset.
- `outBuff` - the array to store the calculated signature. Must be in transient memory.
- `outOffset` - the offset into calculated signature array to store the signature.

Returns:

- The length of the written signature starting from given start offset.

Throws:

- `CryptoException` - with the following reason code:

- `CryptoException.ILLEGAL_VALUE` - if the key length is unsupported or if algorithm is unsupported or if `outBuf` is not transient or null.
- `NullPointerException` - if the `theKey/inBuff` is null.
- `ArrayIndexOutOfBoundsException` - if the offset of the input/output array is negative or if the array length is less than the input length.

5.4.10.4.8 `verify()` - without ICV

```
public static boolean verify(Key theKey, byte algorithm, byte[] inBuff, short inOffset,  
short inLength, byte[] sigBuff, short sigOffset, short sigLength)  
throws CryptoException
```

The purpose of this method is to invoke `CryptoBaseX.verify(Key, byte, byte[], short, short, byte[], short, short, byte[], short, short)` with IV set to null. IV will then be handled as zero in the called method. After the operation, the cryptographic buffer will be cleared.

Parameters:

- `theKey` - the key object to use for verification.
- `algorithm` - the desired algorithm from the available cipher and signature algorithms of this class. Valid codes are listed in the `ALG_*` constants above, for example, `ALG_DES_CBC_NOPAD` or `ALG_DES_MAC8_ISO9797_M2`.
- `inBuff` - the input buffer of data to be verified.
- `inOffset` - the offset into the input buffer at which to begin signature generation.
- `inLength` - the byte length to sign.
- `sigBuff` - the buffer containing signature data.
- `sigOffset` - the offset into `outBuff` where signature data begins.
- `sigLength` - the byte length of the signature data.

Returns:

- `ConstantX.TRUE16` if the signature verifies, `ConstantX.FALSE16` otherwise.

Note: if `outLength` is inconsistent with this signature algorithm, `ConstantX.FALSE16` is returned.

Throws:

- `CryptoException` - with the following reason code:
 - `CryptoException.UNINITIALIZED_KEY` - if the key is not initialized.
 - `CryptoException.ILLEGAL_VALUE` - if the key is inconsistent with the signature algorithm.
 - `CryptoException.NO_SUCH_ALGORITHM` - if the algorithm is not one of the supported algorithms.
 - `CryptoException.ILLEGAL_USE` - if the signature length is not equal to the block length for symmetric algorithms or is not equal to the key length for asymmetric algorithms.
- `Exception` - with the following reason code:
 - `Exception.SINGLETON_IDX_NULL_POINTER_EXCEPTION` - if the `theKey/inBuff/outBuff` is null.
 - `Exception.SINGLETON_IDX_ARRAY_INDEX_OUT_OF_BOUNDS_EXCEPTION` - if the offset of input/output array is negative or if the array length is less than the input length.
- `SecurityException` - if access to input/output array is not allowed in current context.

5.4.10.4.9 `verify()` - with ICV

```
public static boolean verify(Key theKey, byte algorithm, byte[] inBuff, short inOffset,
short inLength, byte[] ivBuffer, short ivBufferOffset, short ivBufferLength, byte[]
sigBuffer, short sigOffset, short sigLength)
```

The purpose of this method is to implement the functionality of `javacard.security.Signature.verify()` with additional checks to validate the key and the algorithm. This method first checks the input parameters to have allowed values, it then invokes `CryptoBaseX.doFinalSymmetric(key, byte, byte, byte[], short, short, byte[], short)` for symmetric algorithms to generate signature or calculates hash for asymmetric algorithms and compare calculated IV with the given signature or hash with the hash extracted from the given signature. Clears the used message buffer securely.

Parameters:

- `theKey` - the key object to use for signature verification.
- `algorithm` - the desired algorithm from the available cipher and signature algorithms of this class. Valid codes listed in `ALG_*` constants above, for example, `ALG_DES_CBC_NOPAD` or `ALG_DES_MAC8_ISO9797_M2`.
- `inBuff` - the input buffer of data to be verified.
- `inOffset` - the offset into the input buffer at which to begin signature generation.
- `inLength` - the byte length to sign.

- `ivBuffer` - the buffer containing the initial vector.
- `ivBufferOffset` - the offset into the buffer containing the initial vector.
- `ivBufferLength` - length of the initial vector.
- `sigBuffer` - the buffer containing signature data.
- `sigOffset` - the offset into `outBuff` where signature data begins.
- `sigLength` - the byte length of the signature data.

Returns:

- `ConstantX.TRUE16` if the signature verifies, `ConstantX.FALSE16` otherwise.

Note: if `outLength` is inconsistent with this signature algorithm, `ConstantX.FALSE16` is returned.

Throws:

- `CryptoException` - with the following reason code:
 - `CryptoException.UNINITIALIZED_KEY` - if the key is not initialized.
 - `CryptoException.ILLEGAL_VALUE` - if the key is inconsistent with the signature algorithm.
 - `CryptoException.NO_SUCH_ALGORITHM` - if the algorithm is not one of the supported algorithms.
 - `CryptoException.ILLEGAL_USE` - if the signature length is not equal to the block length for symmetric algorithms or is not equal to the key length for asymmetric algorithms.
- `Exception` - with the following reason code:
 - `Exception.SINGLETON_IDX_NULL_POINTER_EXCEPTION` - if the `theKey/inBuff/outBuff` is null.
 - `Exception.SINGLETON_IDX_ARRAY_INDEX_OUT_OF_BOUNDS_EXCEPTION` - if the offset of input/output array is negative or if the array length is less than the input length.
- `SecurityException` - if access to `theKey/inBuff/outBuff` is not allowed in current context.

5.4.10.4.10 resetBuffer()

```
public static void resetBuffer(byte[] buffer, short length)
```

The purpose of this method is to clear a buffer storing cipher/signature data.

Parameters:

- `buffer` - byte array holding the data.
- `length` - length of the data stored in the buffer.

5.4.10.5 Class `KeyAgreementX`

`public abstract class KeyAgreementX extends Object`

This class is part of the `com.nxp.id.jcopx.security` package.

The `KeyAgreementX` class provides an additional algorithm for key agreement and a `getInstance` method which generates a `KeyAgreement` instance supporting the chosen algorithm. The returned `KeyAgreement` instance can be used in the same way as the Java Card `KeyAgreement` class in the `javacard.security` package.

The following algorithms are supported by `KeyAgreementX`:

ALG_EC_SVDP_DH_PLAIN_XY

`public static final byte ALG_EC_SVDP_DH_PLAIN_XY = (byte) 0x7E;`

`KeyAgreement` algorithm `ALG_EC_SVDP_DH_PLAIN_XY` is the same algorithm as `ALG_EC_SVDP_DH_PLAIN` but but with additional Y component and a leading 0x04, which is the format of an uncompressed point.

Note: This algorithm requires the eGov module (see Section [5.2 - OS modules](#)).

ALG_EC_SVDP_DHC_PACE

`public static final byte ALG_EC_SVDP_DHC_PACE = (byte) 0x80;`

Proprietary algorithm which computes several steps of the PACE protocol:

$H = \text{TermPubKey} * \text{CardPrivKey}$

$GMap = G * s + H$

$\text{MappedPubKey} = \text{CardMappedPrivKey} * GMap$

$\text{SharedSecret} = \text{CardMappedPrivKey} * \text{TermMappedPubKey}$

For more information on ECDH, see Section [10.2 - A note on ECDH GM](#).

Note: This algorithm requires the eGov module (see Section [5.2 - OS modules](#)).

5.4.10.5.1 `getInstance()`

`public static KeyAgreement getInstance(byte algorithm, boolean externalAccess)`

throws `CryptoException`

Creates a `KeyAgreement` object instance of the selected algorithm.

Parameters:

- `algorithm` - the desired `KeyAgreementX` algorithm. Valid codes listed in `ALG_*` constants above, for example `ALG_EC_SVDP_DH_PLAIN_XY`.
- `externalAccess` - `true` indicates that the instance will be shared among multiple applet instances and that the `KeyAgreement` instance will also be accessed (via a `Shareable` interface) when the owner of the `KeyAgreement` instance is not the currently selected applet. If `true`, the implementation must not allocate `CLEAR_ON_DESELECT` transient space for internal data.

Returns:

- The `KeyAgreement` object instance of the requested algorithm.

Throws:

- `CryptoException` - with the following reason code:
 - `NO_SUCH_ALGORITHM` - if the requested algorithm or shared access mode is not supported.

5.4.10.6 Class SensitiveResultX

```
public class SensitiveResultX extends Object
```

This class is part of the `com.nxp.id.jcopx.security` package.

The `SensitiveResultX` class provides static methods for asserting results of sensitive methods. Sensitive methods store their results so that callers of these methods can assert their return values. If such a method returns abnormally with an exception then the stored result is tagged as unassigned and any subsequent assertion of the result will fail.

The stored result is unaffected by context switches. Particularly, the stored result from an API method called by the method of a `Shareable` interface object is not automatically reset upon switching back to the context of the caller. It is the responsibility of the `Shareable` interface object implementation to reset the stored result if necessary using the `reset` method. Upon entering any of the `Applet` entry point methods the stored result is tagged as `Unassigned`.

The sample code below illustrates the use of the `SensitiveResultX` class:

```
1 try {  
    boolean res = signature.verify (...);  
3     if (res) {
```

```

5         SensitiveResult.assertTrue();
           // Grant service
7     } else {
           SensitiveResult.assertFalse();
           // Deny service
9     }
11 } finally {
        SensitiveResult.reset();
    }

```

Note: Results from methods with a `byte` return type are stored as `short` after conversion (with sign-extension).

5.4.10.6.1 Relationship with JavaCard 3.0.5 SensitiveResult

The API of the `SensitiveResultX` class is identical to the class `javacardx.security.SensitiveResult` defined in JavaCard 3.0.5. However, `SensitiveResultX` is updated only for a subset of the sensitive methods defined in JavaCard 3.0.5. Additionally, `SensitiveResultX` is also updated for one more method than the standard JavaCard 3.0.5.

5.4.10.6.2 List of sensitive methods updating SensitiveResultX

The following Java Card methods reset the sensitive result on entry and sets it on normal return:

- `OwnerPIN`
 - `check` — sets the sensitive result to the result of the PIN check.
 - `isValidated` — sets the sensitive result to true if a valid PIN has been presented since the last card reset or last call to `reset`.
 - `getValidatedFlag` — sets the sensitive result to the value of the validated flag.
 - `getTriesRemaining` — sets the sensitive result to the remaining tries.
- `OwnerPINx`
 - `getTryLimit` — sets the sensitive result to the try limit.
- `OwnerPINxWithPreDecrement`
 - `check` — sets the sensitive result to the result of the PIN check.
 - `decrementTrieRemaining` — sets the sensitive result to the remaining tries.
- `Util`

- `arrayCompare` — sets the sensitive result to the result of the array comparison.
- `arrayCopy` — sets the sensitive result to `srcOff + length`.
- `arrayCopyNonAtomic` — sets the sensitive result to `srcOff + length`.
- `arrayFill` — sets the sensitive result to `bOff + bLen`.
- `arrayFillNonAtomic` — sets the sensitive result to `bOff + bLen`.
- `RandomData`
 - `generateData` — sets the sensitive result to `offset + length`.
 - `nextBytes` — sets the sensitive result to `offset + length`.
- `RandomData.OneShot`
 - `generateData` — sets the sensitive result to `offset + length`.
 - `nextByte` — sets the sensitive result to `offset + length`.
- `Signature`
 - `verify` — sets the sensitive result to the signature verification result.
 - `verifyPreComputedHash` — sets the sensitive result to the verification result of pre-computed hash.
- `Signature.OneShot`
 - `verify` — sets the sensitive result to the signature verification result.
 - `verifyPreComputedHash` — sets the sensitive result to the verification result of the pre-computed hash.
- `SignatureMessageRecovery`
 - `verify` — sets the sensitive result to the signature verification result.

The following JCOP method resets the sensitive result on entry and sets it on normal return:

`com.nxp.id.jcopx.egovaccelerators.EgovUtils.secureXor` — sets the sensitive result to `offsetA + length`.

The following methods are implemented:

5.4.10.6.3 `assertEquals()`

```
public static void assertEquals(Object obj)
throws SecurityException
```

Asserts the stored result to be an object reference identical to the provided object reference. This method throws an exception if and only if the stored result reference `res` and the provided object reference `obj` do not refer to the

same object or are not both null; in other words (`res == obj`) evaluates to false.

Parameters:

- `obj` - The object reference to compare with the stored result.

Throws:

- `SecurityException` - if the provided object reference is not identical to that of the stored result.

5.4.10.6.4 `assertTrue()`

```
public static void assertTrue()  
throws SecurityException
```

Asserts the stored result to be a boolean value equal to true.

Throws:

- `SecurityException` - if the stored result is not set to true.

5.4.10.6.5 `assertFalse()`

```
public static void assertFalse()  
throws SecurityException
```

Asserts the stored result to be a boolean value equal to false.

Throws:

- `SecurityException` - if the stored result is not set to false.

5.4.10.6.6 `assertNegative()`

```
public static void assertNegative()  
throws SecurityException
```

Asserts the stored result to be a short value strictly negative. A call to this method is semantically equivalent to a call to `assertLessThan(short)` with parameter 0.

Throws:

- `SecurityException` - if the stored result is not negative.

5.4.10.6.7 assertPositive()

```
public static void assertPositive()  
throws SecurityException
```

Asserts the stored result to be a short value strictly positive. A call to this method is semantically equivalent to a call to `assertGreaterThan(short)` with parameter 0.

Throws:

- `SecurityException` - if the stored result is not positive.

5.4.10.6.8 assertZero()

```
public static void assertZero()  
throws SecurityException
```

Asserts the stored result to be a short value equal to zero. A call to this method is semantically equivalent to a call to `assertEquals(short)` with parameter 0.

Throws:

- `SecurityException` - if the stored result is not zero.

5.4.10.6.9 assertEquals()

```
public static void assertEquals(short val)  
throws SecurityException
```

Asserts the stored result to be a short value equal to the provided short value.

Parameters:

- `val` - The short value to compare with the stored result.

Throws:

- `SecurityException` - if the provided value is not equal to that of the stored result.

5.4.10.6.10 assertGreaterThan()

```
public static void assertGreaterThan(short val)  
throws SecurityException
```

Asserts the stored result to be a short value strictly greater than the provided short value.

Parameters:

- `val` - The short value to compare with the stored result.

Throws:

- `SecurityException` - if the provided value is not greater than that of the stored result.

5.4.10.6.11 `assertLessThan()`

```
public static void assertLessThan(short val)
throws SecurityException
```

Asserts the stored result to be a short value strictly less than the provided short value.

Parameters:

- `val` - The short value to compare with the stored result.

Throws:

- `SecurityException` - if the provided value is not less than that of the stored result.

5.4.10.6.12 `reset()`

```
public static void reset()
```

Resets the stored result. The stored result is tagged as `Unassigned` and any subsequent assertion of the result will fail.

5.4.10.7 Class `SignatureX`

```
public final class SignatureX extends Object
```

This class is part of the `com.nxp.id.jcopx.security` package.

The `SignatureX` class provides additional signature algorithms and two `getInstance` methods which generate `Signature` instances supporting the chosen algorithm. The returned `Signature` instance can be used in the same way as the Java Card `Signature` class in the `javacard.security` package.

The following signature algorithms are supported by SignatureX:

ALG_AES_CMAC16

```
public static final byte ALG_AES_CMAC16 = (byte) 0x66;
```

Signature algorithm ALG_AES_CMAC16 generates a 16-byte MAC using AES key according to CMAC algorithm NIST SP_800-38B [2]. To request this algorithm using the `getInstance(byte, byte, byte, boolean)` method use the `MessageDigest.ALG_NULL`, `SIG_CIPHER_AES_CMAC16`, `Cipher.PAD_NOPAD` constants respectively.

ALG_DES_CMAC8

```
public static final byte ALG_DES_CMAC8 = (byte) 0x7A;
```

Signature algorithm ALG_DES_CMAC8 generates a 8-byte MAC using DES key according to CMAC algorithm NIST SP_800-38BDES. To request this algorithm using the `getInstance(byte, byte, byte, boolean)` method use the `MessageDigest.ALG_NULL`, `SIG_CIPHER_DES_CMAC8`, or `Cipher.PAD_NOPAD` constants respectively.

ALG_KOREAN_SEED_MAC_NRPAD

```
public static final byte ALG_KOREAN_SEED_MAC_NRPAD = (byte) 0x68;
```

Signature algorithm ALG_KOREAN_SEED_MAC_NRPAD generates a 16-byte MAC using Korean Seed in CBC mode.

ALG_KOREAN_SEED_MAC_PKCS7

```
public static final byte ALG_KOREAN_SEED_MAC_PKCS7 = (byte) 0x78;
```

Signature algorithm ALG_KOREAN_SEED_MAC_PKCS7 generates a 16-bytes MAC using Korean Seed in CBC mode and the padding scheme PKSC7.

NRPAD

```
public static final byte NRPAD = (byte) 0x72;
```

Padding algorithm NRPAD choice for the `paddingAlgorithm` parameter of the `getInstance(byte, byte, byte, boolean)` method.

PKCS7

```
public static final byte PKCS7 = (byte) 0x74;
```

Padding algorithm PKCS7 choice for the `paddingAlgorithm` parameter of the `getInstance(byte, byte, byte, boolean)` method.

SIG_CIPHER_AES_CMACH16

```
public static final byte SIG_CIPHER_AES_CMACH16 = (byte) 0x67;
```

Cipher algorithm `SIG_CIPHER_AES_CMACH16` choice for the `cipherAlgorithm` parameter of the `getInstance(byte, byte, byte, boolean)` method. The signature algorithm generates a 16-byte MAC with block size 128 in CBC mode.

SIG_CIPHER_DES_CMACH8

```
public static final byte SIG_CIPHER_DES_CMACH8 = 7B;
```

Cipher algorithm `SIG_CIPHER_DES_CMACH8` choice for the `cipherAlgorithm` parameter of the `getInstance(byte, byte, byte, boolean)` method. The signature algorithm generates a 8-byte MAC using with DES block size in CBC mode.

SIG_CIPHER_KOREAN_SEED_MAC_NRPAD

```
public static final byte SIG_CIPHER_KOREAN_SEED_MAC_NRPAD = (byte) 0x70;
```

Cipher algorithm `SIG_CIPHER_KOREAN_SEED_MAC_NRPAD` choice for the `cipherAlgorithm` parameter of the `getInstance(byte, byte, byte, boolean)` method. The signature algorithm generates a Korean Seed MAC using Korean Seed in CBC mode.

SIG_CIPHER_KOREAN_SEED_MAC_PKCS7

```
public static final byte SIG_CIPHER_KOREAN_SEED_MAC_PKCS7 = (byte) 0x79;
```

Cipher algorithm `SIG_CIPHER_KOREAN_SEED_MAC_PKCS7` choice for the `cipherAlgorithm` parameter of the `getInstance(byte, byte, byte, boolean)` method. The signature algorithm generates a Korean Seed MAC using Korean Seed in CBC mode.

5.4.10.7.1 getInstance() - combined algorithm

```
public static final Signature getInstance(byte algorithm, boolean externalAccess)
throws CryptoException
```

Creates a `Signature` object instance of the selected algorithm.

Parameters:

- `algorithm` - the desired `SignatureX` algorithm. Valid codes listed in `ALG_*` constants above, for example `ALG_AES_CMAC16`.
- `externalAccess` - `true` indicates that the instance will be shared among multiple applet instances and that the `Signature` instance will also be accessed (via a `Shareable` interface) when the owner of the `Signature` instance is not the currently selected applet. If `true`, the implementation must not allocate `CLEAR_ON_DESELECT` transient space for internal data.

Returns:

- The `Signature` object instance of the requested algorithm.

Throws:

- `CryptoException` - with the following reason code:
 - `CryptoException.NO_SUCH_ALGORITHM` - if the requested algorithm or shared access mode is not supported.

5.4.10.7.2 getInstance() - separate digest, cipher and padding algorithm

```
public static final Signature getInstance(byte messageDigestAlgorithm,  
byte cipherAlgorithm, byte paddingAlgorithm, boolean externalAccess)  
throws CryptoException
```

Creates a `Signature` object instance with the selected message digest algorithm, cipher algorithm and padding algorithm.

Note:

- When there is no discrete message digest algorithm, use the `MessageDigest.ALG_NULL` choice for the message digest algorithm.
- When the padding algorithm is built into the cipher algorithm use the `PAD_NULL` choice for the padding algorithm.

Parameters:

- `messageDigestAlgorithm` - the desired message digest algorithm. Valid codes listed in `ALG_*` constants in the `MessageDigest` class, for example `ALG_NULL`.

- `cipherAlgorithm` - the desired cipher algorithm. Valid codes listed in `SIG_CIPHER_*` constants in this class, for example `SIG_CIPHER_AES_CMAC16`.
- `paddingAlgorithm` - the desired padding algorithm. Valid codes listed in `PAD_*` constants in the `Cipher` class, for example `PAD_NULL`.
- `externalAccess` - `true` indicates that the instance will be shared among multiple applet instances and that the `Signature` instance will also be accessed (via a `Shareable` interface) when the owner of the `Signature` instance is not the currently selected applet. If `true` the implementation must not allocate `CLEAR_ON_DESELECT` transient space for internal data.

Returns:

- The `Signature` object instance of the requested algorithm.

Throws:

- `CryptoException` - with the following reason code:
 - `CryptoException.NO_SUCH_ALGORITHM` - if the requested message digest algorithm or cipher algorithm or padding algorithm or their combination or the requested shared access mode is not supported.

5.4.11 JCOPX System

The package `com.nxp.id.jcopx.system` provides an interface between Java Card applets and hardware functionality. The class `APDUx` allows retrieval of the Le Byte of the currently processed APDU.

Tab. 5.50: Supported APIs of package `com.nxp.id.jcopx.system`

Class	Method	Reference
SysControl	<code>enableRandomUID()</code> <code>getConfigItem()</code> <code>getCPLCData()</code> <code>getIdentificationData()</code> <code>getStaticUID()</code> <code>setConfigItem()</code>	5.4.11.1 - Class SysControl

5.4.11.1 Class SysControl

```
public final class SysControl extends Object
```

This class is part of the `com.nxp.id.jcopx.system` package.

Class to provide functionality to control the system behavior.

The following methods are available:

- Retrieve the device identification data
- Set and get selected config items, including the contactless parameters and the warm and cold contact ATR historical characters
- Get the hardware UID
- Enable a random UID

This class uses the following fields:

CONFIG_TAG_TCL_SAK_INCOMPLETE

```
public static final short CONFIG_TAG_TCL_SAK_INCOMPLETE = ;
```

Config item tag corresponding to 10A4 (TCL_SAK_INCOMPLETE) in the config module

CONFIG_TAG_TCL_SAK_COMPLETE

```
public static final short CONFIG_TAG_TCL_SAK_COMPLETE = ;
```

Config item tag corresponding to 10A3 (TCL_SAK_COMPLETE) in the config module

CONFIG_TAG_TCL_L3_ACTIVATION_CONTROL

```
public static final short CONFIG_TAG_TCL_L3_ACTIVATION_CONTROL = ;
```

Config item tag corresponding to 10A5 (TCL_L3_ACTIVATION_CONTROL) in the config module

CONFIG_TAG_TCL_ATS_IF

```
public static final short CONFIG_TAG_TCL_ATS_IF = ;
```

Config item tag corresponding to 109E (TCL_ATS_IF) in the config module. See the user manual for more details

CONFIG_TAG_TCL_ATS_CURRENT_HISTLEN

```
public static final short CONFIG_TAG_TCL_ATS_CURRENT_HISTLEN = ;
```

Config item tag corresponding to 109F (TCL_ATS_CURRENT_HISTLEN) in the config module. Note: it is recommended to update this config item together with CONFIG_TAG_TCL_ATS_HISTCHARS in a transaction to avoid inconsistencies in case of tear-down

CONFIG_TAG_TCL_ATS_HISTCHARS

```
public static final short CONFIG_TAG_TCL_ATS_HISTCHARS = ;
```

Config item tag corresponding to 10A0 (TCL_ATS_HISTCHARS) in the config module. Note: it is recommended to update this config item together with CONFIG_TAG_TCL_ATS_CURRENT_HISTLEN in a transaction to avoid inconsistencies in case of tear-down

CONFIG_TAG_TCL_ATQA_MSB

```
public static final short CONFIG_TAG_TCL_ATQA_MSB = ;
```

Config item tag corresponding to 10A1 (TCL_ATQA_MSB) in the config module. See the user manual for more details

CONFIG_TAG_TCL_ATQA_LSB

```
public static final short CONFIG_TAG_TCL_ATQA_LSB = ;
```

Config item tag corresponding to 10A2 (TCL_ATQA_LSB) in the config module. See the user manual for more details

CONFIG_TAG_7816_ATR_COLD_HIST_LEN

```
public static final short CONFIG_TAG_7816_ATR_COLD_HIST_LEN = ;
```

Config item tag corresponding to 108E (7816_ATR_COLD_HIST_LEN) in the config module. Note: it is recommended to update this config item together with CONFIG_TAG_7816_ATR_COLD_HIST in a transaction to avoid inconsistencies in case of tear-down

CONFIG_TAG_7816_ATR_COLD_HIST

```
public static final short CONFIG_TAG_7816_ATR_COLD_HIST = ;
```

Config item tag corresponding to 108F (7816_ATR_COLD_HIST) in the config module. Note: it is recommended to update this config item together with CONFIG_TAG_7816_ATR_COLD_HIST_LEN in a transaction to avoid inconsistencies in case of tear-down

CONFIG_TAG_7816_ATR_WARM_HIST_LEN

```
public static final short CONFIG_TAG_7816_ATR_WARM_HIST_LEN = ;
```

Config item tag corresponding to 1096 (7816_ATR_WARM_HIST_LEN) in the config module. Note: it is recommended to update this config item together with CONFIG_TAG_7816_ATR_WARM_HIST in a transaction to avoid inconsistencies in case of tear-down

CONFIG_TAG_7816_ATR_WARM_HIST

```
public static final short CONFIG_TAG_7816_ATR_WARM_HIST = ;
```

Config item tag corresponding to 1097 (7816_ATR_WARM_HIST) in the config module. Note: it is recommended to update this config item together with CONFIG_TAG_7816_ATR_WARM_HIST_LEN in a transaction to avoid

inconsistencies in case of tear-down

The following methods are implemented:

5.4.11.1.1 `getIdentificationData()`

```
public static short getIdentificationData(byte[] buffer, short offset)
throws java.lang.NullPointerException, java.lang.ArrayIndexOutOfBoundsException,
java.lang.SecurityException, javacard.framework.SystemException
```

Gets the product identification data and writes it into the specified destination buffer. The identification data is identical to the result of the GET DATA (IDENTIFY) command that can be sent to the ISD (tag FE, subtag DF28).

Note:

- Only the identification data is returned, not the header of the GlobalPlatform GET DATA APDU answer.
- The returned data contains all the identification tags, including the ones that are returned by the GET DATA (IDENTIFY) command when there was a successful authentication with the card manager.
- If the ISD would reject the GET DATA (IDENTIFY) command even after a successful authentication, then the method throws `SystemException.ILLEGAL_USE`. This happens when the GET DATA (IDENTIFY) command has been disabled.
- `buffer` must be a transient byte array.

Parameters:

- `buffer` - destination byte array. Must be transient.
- `offset` - offset in the destination byte array where to write the data. The number of bytes written is defined in the user manual.

Returns:

- `destOff` + the number of bytes written in the buffer.

Throws:

- `java.lang.NullPointerException` - if `buffer` is null.
- `java.lang.SecurityException` - if `buffer` is not accessible by the calling context.
- `java.lang.ArrayIndexOutOfBoundsException` - if the data would be written beyond the end of `buffer`.
- `javacard.framework.SystemException` - `ILLEGAL_USE` if `buffer` is not of type `TRANSIENT`.
- `javacard.framework.SystemException` - `ILLEGAL_USE` if the ISD would reject the GET DATA (IDENTIFY) command even after successful authentication, for example, because the command has been disabled.

5.4.11.1.2 getConfigItem()

```
public static short getConfigItem(short tag, byte[] buffer, short offset, short length)
throws java.lang.NullPointerException, java.lang.ArrayIndexOutOfBoundsException,
java.lang.SecurityException, javacard.framework.SystemException
```

Get the content of a config item. This method is similar to sending a GET CONFIG ITEM to the config module on DGI DF2B. The differences are:

- Only selected config items are accessible and their tags are not the same as for the config module. Use one of the constants CONFIG_TAG_XXX instead.
- The method works even if the config module is deleted.
- Only the config item data is written to the byte array, not the header of the GET CONFIG ITEM APDU.

The parameter buffer must be a transient array.

If the config item is a number, the parameter length must be equal to the actual length of the config item, otherwise `ArrayIndexOutOfBoundsException` is thrown.

If the config item is an array, the parameter length can be different from the actual length of the config item (`actualLength`), in which case, the following applies:

- If `length < actualLength`, then the content of the config item array is truncated to length bytes and exactly length bytes are written to buffer. In this case `offset+length` is returned.
- If `length > actualLength`, then the full content of the config item array is copied at offset and `offset+actualLength` is returned. The remaining bytes beyond `actualLength` are left untouched.

Parameters:

- `tag` - the config item tag, one of the CONFIG_TAG_XXX constants.
- `buffer` - the destination byte array. Must be transient.
- `length` - the number of bytes to read.

Returns:

- `offset + the number of bytes written in the buffer.`

Throws:

- `java.lang.NullPointerException` - if buffer is null.
- `java.lang.SecurityException` - if buffer is not accessible to the caller.
- `java.lang.ArrayIndexOutOfBoundsException` - if `offset+length` is beyond the length of buffer.
- `javacard.framework.SystemException` - `ILLEGAL_USE` if buffer is not transient.
- `javacard.framework.SystemException` - `ILLEGAL_USE` if tag is not supported.
- `java.lang.ArrayIndexOutOfBoundsException` - if the config item is a number and length is not exactly the actual length of the config item.

5.4.11.1.3 `setConfigItem()`

```
public static short setConfigItem(short tag, byte[] buffer, short offset, short length)
throws java.lang.NullPointerException, java.lang.ArrayIndexOutOfBoundsException,
java.lang.SecurityException, javacard.framework.SystemException,
javacard.framework.TransactionException
```

Set the content of a config item.

This method is similar to sending a SET CONFIG ITEM to the config module on DGI DF2B. The differences are:

- Only selected config items are accessible and their tags are not the same as for the config module. Use one of the constants `CONFIG_TAG_XXX` instead.
- The method works even if the config module is deleted.

If a transaction is opened, the writing to the config item participates to the transaction. It is recommended to use a transaction when setting multiple related config items.

If the config item is a number, the parameter length must be equal to the actual length of the config item, otherwise `ArrayIndexOutOfBoundsException` is thrown.

If the config item is an array, the parameter length can be smaller than the actual length of the config item. In this case, the first length bytes of the config item are set from buffer and the remaining bytes of the config item are set to zero. However, length cannot be larger than the actual length of the config item, otherwise `ArrayIndexOutOfBoundsException` is thrown.

Returns:

- offset+length.

Throws:

- java.lang.NullPointerException - if buffer is null.
- java.lang.SecurityException - if buffer is not accessible to the caller.
- ArrayIndexOutOfBoundsException - if offset+length is beyond the length of buffer.
- javacard.framework.SystemException - ILLEGAL_USE if tag is not supported.
- java.lang.ArrayIndexOutOfBoundsException - if the config item is a number and length is not exactly the actual length of the config item.
- java.lang.ArrayIndexOutOfBoundsException - if the config item is an array and length is bigger than the actual length of the config item.
- javacard.framework.TransactionException - if copying would cause the commit capacity to be exceeded.

5.4.11.1.4 enableRandomUID()

```
public static short enableRandomUID()
```

Enable random UID for contactless activation. In effect, this set the bits 4 and 5 of the config item 10A5 (TCL_L3_ACTIVATION_CONTROL) to '00'.

This is a convenience method to avoid calling getConfigItem, modifying the bits, and then calling setConfigItem.

Returns:

- Always returns ConstantX.TRUE16.

5.4.11.1.5 getStaticUID()

```
public static short getStaticUID(byte[] buffer, short destOff)
throws java.lang.NullPointerException, java.lang.ArrayIndexOutOfBoundsException,
java.lang.SecurityException, javacard.framework.SystemException
```

Get the hardware UID. This method always returns the device's hardware UID independently of the current UID selection, that is, even if a random UID is currently enabled.

Parameters:

- `buffer` - the destination buffer to which the UID gets copied. The buffer must be large enough to contain the UID. The buffer must be a transient.
- `destOff` - offset of the first byte in buffer.

Returns:

- `destOff` + the number of bytes written in the buffer (either 4 or 7 bytes).

Throws:

- `java.lang.NullPointerException` - if buffer is null.
- `java.lang.SecurityException` - if buffer is not accessible by the calling context.
- `java.lang.ArrayIndexOutOfBoundsException` - if the data would be written beyond the end of buffer.
- `javacard.framework.SystemException` - `ILLEGAL_USE` if buffer is not transient.

5.4.11.1.6 `getCPLCData()`

```
public static short getCPLCData(byte[] buffer, short destOff)
```

Returns Card Production Life Cycle data. The purpose of this method is to provide the Card Production Life Cycle data (tag '9F7F') as defined in VGP 2.1.1 Card Implementation Requirements. The returned length of CPLC data is a constant value (42 bytes).

Parameters:

- `buffer` - the destination buffer to which the CPLC data gets copied. The buffer must be large enough to contain 42 bytes. The buffer must be a transient.
- `destOff` - offset of the first byte in buffer.

Returns:

- `responseOffset` + the number of bytes written in the buffer (42 bytes).

Throws:

- `java.lang.NullPointerException` - if buffer is null.
- `java.lang.SecurityException` - if buffer is not accessible by the calling context.
- `javacard.framework.SystemException` - `ILLEGAL_USE` if buffer is not transient.

5.4.12 JCOPX Util

The JCOPX Util package contains a class supporting arrays with a checksum (Error Detection Code (EDC)) and a class to support addition of Binary Coded Decimal (BCD) numbers.

Tab. 5.51: APIs of package com.nxp.id.jcopx.util

Class	Method	Reference
BCD	add()	See Section 5.4.12.1 - Class BCD
	addBCDupdateEDC()	
EDC	arrayCopyNonAtomicVerifyEDC()	See Section 5.4.12.2 - Class EDC
	arrayCalcEDC()	
	arrayVerifyEDC()	

5.4.12.1 Class BCD

```
public class BCD extends Object
```

This class is part of the `com.nxp.id.jcopx.util` package.

The class BCD provides a method to add two 12 digit BCD numbers. The sum of the two input numbers is stored in the `bcdRhs` input array and is protected by a EDC.

Note: The BCD addition is not intended to be used with confidential data.

5.4.12.1.1 add()

```
public static final void add(byte[] bcdLhs, short offsLhs, byte[] bcdRhs, short offsRhs)
throws ArrayIndexOutOfBoundsException, NullPointerException, SystemException,
ArithmeticException
```

The method adds the two 12 digit BCD numbers in the input arrays `bcdLhs` and `bcdRhs` and stores the sum of both values in the `bcdRhs` array at `offsRhs`. During the addition of both values the method computes an EDC checksum and stores it at the end of the calculation in the `bcdRhs` array at `offsRhs+6`. The calculated EDC can be verified with the `EDC.arrayVerifyEDC()` method.

The EDC of the `bcdRhs` array is not recalculated or checked before the addition.

The offset of the most significant digit (MSD) of each array is given by `offsLhs` and `offsRhs` respectively.

The `bcdRhs` array shall be located in transient memory.

Note:

- If the `offsLhs` or the `offsRhs` parameter is negative, an `ArrayIndexOutOfBoundsException` exception is thrown.
- If `offsLhs+6` is greater than `bcdLhs.length`, the length of the `bcdLhs` array, an `ArrayIndexOutOfBoundsException` exception is thrown.
- If `offsRhs+7` is greater than `bcdRhs.length`, the length of the `bcdRhs` array, an `ArrayIndexOutOfBoundsException` exception is thrown.
- If `bcdLhs` or `bcdRhs` parameter is null a `NullPointerException` exception is thrown.
- The method returns the BCD value '999999999999' in case of overflow of the addition.

Parameters:

- `bcdLhs` - contains the value to be added to `bcdRhs`.
- `offsLhs` - offset of the `bcdLhs`.
- `bcdRhs` - contains the `bcdRhs` and keeps the sum of both numbers.
- `offsRhs` - offset of the `bcdRhs` and the sum of both numbers.

Throws:

- `ArrayIndexOutOfBoundsException` - if copying would cause access of data outside array bounds.
- `NullPointerException` - if either `bcdLhs` or `bcdRhs` is null.
- `SystemException` - with the following reason code:
 - `SystemException.ILLEGAL_VALUE` - if `bcdRhs` is not located in transient memory.
- `ArithmeticException` - if any of the BCD digits is not in the allowed range (0 - 9) or if the sum of both BCD values exceeds the maximum value '999999999999'.

5.4.12.1.2 addBCDupdateEDC()

```
public static final short addBCDupdateEDC(byte[] bcdLhs, short offsLhs,  
byte[] bcdRhs, short offsRhs, short offsEdc)  
throws ArrayIndexOutOfBoundsException, NullPointerException, SystemException,  
ArithmeticException, ArithmeticException
```

The method adds the two 12 digit BCD numbers in the input arrays `bcdLhs` and `bcdRhs` and stores the sum

of both values in the `bcdRhs` array at `offsRhs`. During the addition of both values the method computes an EDC checksum and stores it at `offsEdc`. The EDC is calculated by this method differently to the method `add()` as follows:

$EDC2 = EDC1 \oplus rhsx \oplus yx$, with

`EDC2` - value at position `offsEdc` after the API call

`EDC1` - value at position `offsEdc` before the API call

`rhsx` - the xor value calculated over 6 bytes of `rhs` values (initial `rhs` values)

`yx` - the xor value calculated over 6 bytes of `lhs + rhs` values (final `rhs` values)

The `EDC1` is not recalculated or checked before the addition.

The offset of the most significant digit (MSD) of each array is given by `offsLhs` and `offsRhs` respectively.

The `bcdRhs` array shall be located in transient memory.

Note:

- If the `offsLhs` or the `offsRhs` parameter is negative, an `ArrayIndexOutOfBoundsException` exception is thrown.
- If `offsLhs+6` is greater than `bcdLhs.length`, the length of the `bcdLhs` array, an `ArrayIndexOutOfBoundsException` exception is thrown.
- If `offsRhs+6` is greater than `bcdRhs.length`, the length of the `bcdRhs` array, an `ArrayIndexOutOfBoundsException` exception is thrown.
- If `offsEdc` is greater than `bcdRhs.length`, the length of the `bcdRhs` array, an `ArrayIndexOutOfBoundsException` exception is thrown.
- If `bcdLhs` or `bcdRhs` parameter is null a `NullPointerException` exception is thrown.

Parameters:

- `bcdLhs` - contains the value to be added to `bcdRhs`.
- `offsLhs` - offset of the `bcdLhs`.
- `bcdRhs` - contains the `bcdRhs` and keeps the sum of both numbers.
- `offsRhs` - offset of the `bcdRhs` and the sum of both numbers.

Throws:

- `ArrayIndexOutOfBoundsException` - if copying would cause access of data outside array bounds.
- `NullPointerException` - if either `bcdLhs` or `bcdRhs` is null.

- `SystemException` - with the following reason code:
 - `SystemException.ILLEGAL_VALUE` - if `bcdRhs` is not located in transient memory.
- `ArithmeticException` - if any of the BCD digits is not in the allowed range (0 - 9) or if the sum of both BCD values exceeds the maximum value '999999999999'.

5.4.12.2 Class EDC

`public class EDC extends Object`

This class is part of the `com.nxp.id.jcopx.util` package.

The class EDC provides methods to calculate and verify checksums for arrays. The EDC is calculated by an XOR operation over the input data. It is stored inside the byte array at the last position. All methods in this API which receive a length information include this EDC byte in the length, so that the byte array shall be one byte longer than the data.

5.4.12.2.1 `arrayCopyNonAtomicVerifyEDC()`

```
public static final short arrayCopyNonAtomicVerifyEDC(byte[] src, short srcOff,
byte[] dest, short destOff, short length)
throws ArrayIndexOutOfBoundsException, NullPointerException, SecurityException
```

Copies an array or a subset of an array, including the checksum, from the specified source array, beginning at the specified position, to the specified position of the destination array (non-atomically) and performs a checksum verification on the data in the destination array. The copy operation does not recalculate any checksum nor it is checked if the checksum out of the source array (or the subset) are valid before the copy operation starts. If the copy operation finished with success, the checksum verification on the destination array is executed. If the verification fails a `SecurityException` is thrown.

This method does not use the transaction facility during the copy operation even if a transaction is in progress. Thus, this method is suitable for use only when the contents of the destination array can be left in a partially modified state in the event of a power loss in the middle of the copy operation.

Note:

- If the `srcOff` or the `dstOff` is negative or the `length` parameter is less than 1 or negative, an `ArrayIndexOutOfBoundsException` exception is thrown and no copy and no checksum verification is performed.
- If `srcOff+length` is greater than `src.length`, the length of the `src` array, an `ArrayIndexOutOfBoundsException` exception is thrown and no copy and no checksum verification is performed.

- If `dstOff+length` is greater than `dest.length`, the length of the `dest` array, an `ArrayIndexOutOfBoundsException` exception is thrown and no copy and no checksum verification is performed.
- If `src` or `dest` parameter is null a `NullPointerException` exception is thrown and no copy and no checksum verification is performed.
- If the final EDC verification starting at `dstOff` of `dest` array up to `dstOff+length-1` of `dest` array fails, a `SecurityException` is thrown. Note that the EDC byte is located at offset `dstOff+length-1`.
- If the `src` and `dest` arguments refer to the same array object, then the copying is performed as if the components at positions `srcOff` through `srcOff+length-1` were first copied to a temporary array with `length` components and then the contents of the temporary array were copied into positions `dstOff` through `dstOff+length-1` of the argument array.
- If power is lost during the copy operation and the destination array is persistent, the destination array could be partially changed. In this case the included checksum(s) could become invalid.
- The copy `length` parameter is not constrained by the atomic commit capacity limitations.

Precondition:

- If this method performs copy operations on a subset of an array, the caller shall make sure that the input array has the correct EDC at position `srcOff+length-1`.

Parameters:

- `src` - the source byte array.
- `srcOff` - the offset within the source byte array to start the copy operation from.
- `dest` - the destination byte array.
- `dstOff` - the offset within the destination byte array to start copying into.
- `length` - the byte length to be copied.

Returns:

- `dstOff+length`.

Throws:

- `ArrayIndexOutOfBoundsException` - if copying would cause access of data outside array bounds or `length` is less than 1.
- `NullPointerException` - if either `src` or `dest` is null.
- `SecurityException` - if EDC checksum verification on the destination byte array fails.

5.4.12.2.2 arrayCalcEDC()

```
public static final short arrayCalcEDC(byte[] src, short srcOff, short length)
throws ArrayIndexOutOfBoundsException, NullPointerException
```

Calculates checksum across specified part of `src` array and stores the result in `src` array at position `srcOff+length-1`. The EDC is an XOR operation over `length-1` bytes starting at position `srcOff` of `src` array. The calculated EDC checksum is stored in `src` array at position `srcOff+length-1`. After this method is called, the result of the XOR operation starting at position `srcOff` of `src` array and ending at position `srcOff+length-1` of `src` array will be zero.

Note:

- If the `srcOff` is negative or the `length` parameter is less than 1 or negative, an `ArrayIndexOutOfBoundsException` exception is thrown and no EDC is calculated.
- If `srcOff+length` is greater than `src.length`, the length of the `src` array, an `ArrayIndexOutOfBoundsException` exception is thrown and no EDC is calculated.
- If `src` parameter is null a `NullPointerException` exception is thrown and no EDC is calculated.

Parameters:

- `src` - source byte array to calculate the EDC on.
- `srcOff` - offset within the source byte array to start EDC calculation from.
- `length` - specifies the number of bytes over which EDC calculation is done (`length-1`) and specifies the position in `src` array where to place the calculated EDC value (`srcOff+length-1`).

Returns:

- `srcOff+length`.

Throws:

- `ArrayIndexOutOfBoundsException` - if EDC calculation would cause access of data outside array bounds or `length` is less than 1.
- `NullPointerException` - if `src` array is null.

5.4.12.2.3 arrayVerifyEDC()

```
public static final short arrayVerifyEDC(byte[] src, short srcOff, short length)
throws ArrayIndexOutOfBoundsException, NullPointerException, SecurityException
```

Verifies if the EDC value of a byte array is valid. The EDC calculation is done by a XOR operation over length-1 bytes starting at position srcOff. The EDC checksum must be located at position srcOff+length-1 in src array. If the verification fails a SecurityException is thrown.

Note:

- If srcOff is negative or length parameter is less than 1 or negative, an ArrayIndexOutOfBoundsException exception is thrown and no checksum verification is performed.
- If srcOff+length is greater than src.length, the length of the src array, an ArrayIndexOutOfBoundsException exception is thrown and no checksum verification is performed.
- If src parameter is null, a NullPointerException exception is thrown and no checksum verification is performed.
- If the EDC verification fails, a SecurityException exception is thrown. Note that the EDC byte is located at offset dstOff+length-1.

Parameters:

- src - source byte array to calculate the EDC on.
- srcOff - offset within source byte array to start EDC calculation from.
- length - specifies the number of bytes over which EDC calculation is done (length-1) and specifies the position in src array against which the calculated EDC value is compared to (srcOff+length-1).

Returns:

- destOff+length.

Throws:

- ArrayIndexOutOfBoundsException - if EDC calculation would cause access of data outside array bounds or length is less than 1.
- NullPointerException - if src array is null.
- SecurityException - if EDC checksum verification fails.

5.5 Statistical random number generator test

JCOP initializes the Pseudo Random Number Generator of the CryptoLib during start up. The CryptoLib performs a Chi-squared test on the output of the hardware True Random Number Generator and reseeds the PRNG with new bytes from the TRNG. When the CryptoLib detects a failure on the TRNG, this information is returned to JCOP which afterwards updates the AC.

The probability of this statistical test failing on a functional JCOP device is extremely low.

5.6 Uninitialized OwnerPin

The Java Card specification does not describe the behavior when the `OwnerPIN.check()` method is invoked on an uninitialized Personal Identification Number (PIN) (when the `OwnerPIN.update()` method has not been invoked before).

JCOP 4 P71 initializes the PIN with a die-individual default value to prevent attacks on newly created PIN values.

5.7 FIPS self-test

When operating in FIPS mode the device shall perform a series of self-tests to ensure data and functional integrity. The self-tests performed are configurable (see Section 8.3.4 - [FIPS support](#)). The tests include verification of the complete Flash and ROM contents, RNG check and crypto algorithmic validation against known answers.

5.8 Attack detection

JCOP 4 P71 permanently monitors its internal operational state and detects illegal and logically impossible operations. This ensures that manipulations are detected and the card is protected against manipulation.

5.8.1 Behavior in case of an attack

JCOP contains an internal Attack Counter which is updated on each detected attack. The AC value can be reset as described in [5.1.2.4](#).

When an attack is sensed the card internally triggers a reset to abort the current operation and restart the card.

To prevent continuous attacks, the card sets an internal flag when an attack is detected. As long as this flag is set, JCOP prevents going into normal operation mode after the reset and stays mute. The flag expires after some seconds, independently of whether the card is powered or not.

5.8.2 Restricted mode

When the attack counter reaches its limit, the card changes its operating mode into restricted mode. JCOP allows a limited set of commands when the AC has expired:

- Select ISD with the full AID of the ISD.
- Authenticate to the ISD with security level C_MAC.
- Reading CPLC data.
- GET DATA command to retrieve following information:
 - Platform ID (see [5.1.1.3 - Get platform ID](#))
 - Event log information (DF26) (see [5.1.3.3 - Read attack counter log](#))
 - Get available memory (DF25)
 - IDENTIFY command
- Reset the attack counter, if the feature is activated.

All other commands are responded with the status word 66A5h (RESTRICTED_MODE_ERROR).

5.9 Velocity limitations

5.9.1 SCP protocols

The SCP protocols SCP02 and SCP03 both contain an authentication counter to prevent replay attacks. The GlobalPlatform specification defines when the counter value has to be incremented, see GlobalPlatform 2.3 [15].

In JCOP the maximum counter values for the SCP protocols are as follows:

SCP protocol	Limit
SCP02	32 767
SCP03	500 000

If the counter for the SCP02 protocol reaches its maximum value, the keys belonging to this SCP protocol are deleted.

If the counter for the SCP03 protocol reaches its maximum value, the following error status word is returned: 6985h (CONDITIONS_NOT_SATISFIED).

5.9.2 Unsuccessful or incomplete authentication

JCOP 4 P71 limits the usage of some keys after a consecutive number of unsuccessful or incomplete authentication requests:

- An unsuccessful authentication request is an authentication with a wrong key.
- An incomplete authentication request is encountered if the authentication requires a sequence of commands and this sequence was not completed successfully (like for the SCP protocols which consist of a pair of card authentication (INITIALIZE UPDATE) and terminal authentication (EXTERNAL AUTHENTICATE) commands).

Unsuccessful or incomplete authentication request are counted by an Authentication Retry Counter (ARC). Each instance in JCOP maintains its own ARC. After a successful authentication procedure the ARC of this instance is reset.

Instance	Protected key
ISD	SCP keys
SSD	SCP keys

After 59 consecutive wrong authentication requests the ARC expires.

5.9.2.1 ARC behavior of the ISD

After the ARC of the ISD is expired, the ISD will allow only a limited set of commands. Especially the commands to open a SCP channel are no longer available. Any command which is not in the set of supported commands is rejected with the error status word 69FFh (ISD_LIMITED_ARC_EXPIRED).

Supported commands after ARC expiration	Reference
Get CLPC Data	5.1.1.2 - Get CPLC data
Read attack counter log	5.1.3.3 - Read attack counter log

When the ARC of the ISD expired, then further SELECT ISD APDUs are responded with the warning status word 6280h (AUTH_EXCEEDED) and the following data will be returned in the FCI template: 'auth exceed'.

The response data of the ISD APDU is formatted as follows when the ARC expired:

Tab. 5.52: Response data of SELECT(ISD) if the ARC of the ISD expired

Field	Length	Content	Remark
Response data	'0A'	'auth exceed'	ARC expired
SW	'02'	'6280'	

5.9.2.2 ARC behavior of a SSD

When the ARC of a SSD expires, the life cycle state of the SSD is set to *LOCKED*. This prevents further authentication requests to this SSD unless it is set back to life cycle state *PERSONALIZED*. This requires a prior successful authentication to the ISD. Each SD maintains its own ARC.

5.10 Flash statistics

Flash statistic information can be retrieved using the following command sequence:

1. Reset card (cold or warm).
2. SELECT BY NAME with AID D276000085304A434F800001
3. GET DATA (Flash statistic).
4. Consequent Flash statistic information must be retrieved using GET RESPONSE commands [18] with Le 'xy' as specified in status word '61XY' of the previous response. All available information is obtained when 9000h (NO_ERROR) is returned and JCOP 4 P71 will return to standard behavior after reset.

Note: In case of wrong command sequence JCOP 4 P71 will return to standard behavior after reset.

Note: Information retrieved by the Flash statistic command sequence is encrypted and can only be interpreted by NXP.

The command GET DATA (Flash statistic) shall be formatted as follows:

Tab. 5.53: GET DATA (Flash Statistic)

Code	Value	Parameter settings
CLA	'80'	GlobalPlatform/ISO/IEC
INS	'CA'	GET DATA
P1	'00'	High order tag value
P2	'FE'	Low order tag value - proprietary data
Lc	'06'	Length of data field
Data	'DF2600FEA500'	Flash Statistic
Le	'00'	Length of response data

The response data of the GET DATA (Flash Statistic) APDU is formatted as follows:

Tab. 5.54: Response data of GET DATA (Flash Statistic)

Field	Length	Content	Remark
Response data		Encrypted Flash Statistic information	
SW	'02'	'9000 OR 61XY'	'xy' indicates Le for next GET RESPONSE command

5.11 Proprietary APDUs

JCOP 4 P71 does not implement proprietary commands. Proprietary functionality is implemented by proprietary tags for STORE DATA, GET DATA (see Section 5.1 - [Additional APDUs](#)) and PUT DATA (see Section 7.1.1 - [PUT DATA \(Configuration\) commands in transport mode](#)).

5.12 Proprietary status words

JCOP may return the following proprietary status words:

Status word	Reference
6280h (AUTH_EXCEEDED)	See Section 5.9.2.1 - ARC behavior of the ISD
66A5h (RESTRICTED_MODE_ERROR)	See Section 5.8.1 - Behavior in case of an attack
69FFh (ISD_LIMITED_ARC_EXPIRED)	See Section 5.1.2.3 - Disable ISD and 5.9.2.1 - ARC behavior of the ISD

5.13 Platform-specific Java Card 3.0.5 features

This section lists any Java Card 3.0.5 features and behaviors specific to JCOP.

5.13.1 OwnerPINxWithPrededcrement.decrementTriesRemaining

Every call to `OwnerPINxWithPrededcrement.decrementTriesRemaining` will decrement the tries remaining, regardless of whether this has been done before. This change on the `OwnerPINxWithPrededcrement` instance is persistent.

6 Product security

This chapter describes requirements and recommendations which shall be followed to use the product in a secure and certified way.

Requirements are stated with the key word 'shall' (identifier prefix REQ_) and recommendations with the key word 'should' (identifier prefix REC_).

Depending on the role of the user of JCOP different requirements and recommendations apply:

- **Administrators** of JCOP shall follow [6.1 - Procedural requirements](#) and [6.2 - Operating environment requirements](#).
An Administrator performs card OS initialization and/or applet personalization:
 - OS initialization: JCOP is configured and (optionally) applets are loaded,
 - Personalization: applets are loaded and individualization data is stored within.
- **Applet developers** for JCOP shall follow [6.3 - Applet requirements](#) and [6.4 - Cryptographic requirements](#).

6.1 Procedural requirements

For a certified usage of JCOP 4 P71 the following requirements and recommendations shall be fulfilled:

- REQ_TKCUST: The initial ISD Transport Keys shall not be made available to the end user.
- REC_TKPERs: The initial ISD Transport Keys should be made available to the card personalizer only if the card personalizer needs to perform OS initialization.
- REQ_CAP_PREP: The entity preparing a CAP file for JCOP shall ensure that the CAP file does not contain malicious code or introduces any threat to JCOP or applets.
- REQ_DAP_SIGN: The entity creating the DAP signature for a CAP file shall verify the integrity of the CAP file and that the content of the CAP file does not introduce any threat to JCOP or applets.

The following requirements and recommendation shall be fulfilled, before the card is delivered to the end user:

- REQ_DEL_CA: The Config Module shall be deleted.
- REQ_ISD: The ISD and any SSD which allows loading of CAP files shall be initialized with new keys. These keys shall be kept secret.
- REC_SSD: Each SSD should be initialized with new keys.
- REQ_LIFE_CYCLE: The card life cycle shall be set to *SECURED*.
- REC_PIN: The default values of the Owner PIN and Global PIN should be replaced.

6.2 Operating environment requirements

The following procedures shall be followed by the customer during delivery and after reception of the product:

- REQ_CONF: All material and information delivered from NXP to the customer shall be kept confidential.
- REQ_PROC: The confidentiality and integrity of the product and of its manufacturing and test data shall be ensured by appropriate procedures (to prevent any possible copy, modification, retention, theft or unauthorized use).
- REQ_DELIVER: Material and information under delivery shall be protected. This includes the following objectives:
 - Any security relevant information of the element under delivery shall be disclosed,
 - confidentiality rules (confidentiality level, transmittal form, reception acknowledgement) shall be met,
 - all delivered material shall be physically protected to prevent external damage, to ensure secure storage and handling procedures (including rejected products),
 - the product shall be traced during delivery.
- REQ_CORR: Corrective actions shall be taken in case of improper operation in the delivery process (including, if applicable, any non-conformance to the confidentiality convention) and highlight all non-conformance to this process.

The environment at the customer shall ensure:

- REQ_ENV: Secure communication protocols offered by the product shall be used for security sensitive tasks if the communication is not secured by other means (e.g. a secure environment).
- REQ_KEYSTOR: Keys which are stored outside the product and which are used for secure communication and authentication between the Smart Card and terminals shall be protected for confidentiality and integrity in their storage environment.
- REQ_SENS: Sensitive Data and documentation shall be delivered to either the Card Manufacturer, or to the Personalizer through a trusted delivery and verification procedure that shall be able to maintain the integrity and confidentiality of the product's sensitive Data.
- REQ_SCP02: SCP02 is deprecated by GlobalPlatform. Card Content Management operations and applications relying on SCP02 confidentiality protection of static data shall adopt one of the possible mitigations:
 1. Encrypt all sensitive data transmitted in SCP02 using the Data Encryption Key (DEK) or any applet key.
 2. Disable SCP02.
 3. Transition to SCP03.

6.3 Applet requirements

- REQ_BCV: All CAP files shall be byte code verified. This can be done using the latest version of Oracle's Byte Code Verifier which also is included in NXP's JCOP Tools. Appropriate procedures shall be in place to guarantee that the applet code is not modified after verification.
- REQ_MAL: Applets and CAP files shall not contain malicious code.
- REQ_TPY: CAP files shall only be loaded by a trusted party.
- REQ_CAP: CAP files loaded via GlobalPlatform Card Content Management shall contain no other components than those defined in [22].

6.4 Cryptographic requirements

- REQ_CRYPT_CSF: The usage of SHA-1, Single-DES, and short key lengths for RSA and ECC (see TR02102 [10] for recommendations on key length) shall be assessed during a composite evaluation if scenarios with high attack potential are applicable in the composite application context. For example using SHA-1 algorithm in combination with temporary data (for example as used for generating session keys) where the SHA-1 algorithm is applied to the same input data once might make some attacks not applicable.
- REC_CRYPT_SESS_API: It is recommended to invoke the method `CryptoBaseX.sign()` - Session key only with session keys (see 5.4.10.4.7 - [sign\(\)](#) - Session key).
- REC_CRYPT_SESS_API_INT: The applet should implement appropriate integrity protection for the key used by `CryptoBaseX.sign()` - Session key if specific requirements for key integrity are mandated by the application (see 5.4.10.4.7 - [sign\(\)](#) - Session key).
- REC_CRYPT_KEY_OBJ: Key material should be stored in Java Card key objects only. The operating system especially secures these objects.
- REC_CRYPT_OWN_PIN: PIN functionality should be implemented using `OwnerPIN` objects or GlobalPlatform Cardholder Verification Method (CVM) only. The system takes special security measures for such objects. However, the applet developer is responsible for managing the PIN properties. This includes restricting the minimum allowed PIN size and to set an appropriate PIN try limit. Also, it is recommended to encrypt PIN values when they are sent to the card and never send out the PIN value in plain. It is further recommended to replace the initial values of the `OwnerPin` and GlobalPlatform CVM.
- REQ_CRYPT_MATH: The usage of the methods `Math.modularSubtract`, `Math.modularAdd`, `Math.modularMultiply` and `Math.modularReduce` with input length shorter than 4 bytes shall be assessed during a composite evaluation if scenarios with high attack potential for side channel attacks are applicable in the composite application context.

- REQ_CRYPT_SHA: For scenarios where side channel attacks are applicable in the context of the composite application, the following considerations shall be taken into account for the use of the SHA-1 and SHA-2 hashing algorithms or the RSA PSS verification:
 1. if the input message is constant and confidential then the same secret message shall not be used more than 2 times,
 2. if a constant and confidential input message is used more than 2 times, a minor entropy loss of 1.62 bit per byte shall be taken into account.
- REQ_CRYPT_RSA_OAEP_DECODE: For scenarios where side channel attacks are applicable in the context of the composite application, the RSA OAEP decode functionality with output length less than 8 Bytes shall be assessed in the composite evaluation if the output data is secret.
- REQ_CRYPT_KOREANSEED: The Korean SEED algorithm is not included in the certification of this product. The usage shall be assessed during a composite evaluation if scenarios with high attack potential are applicable in the composite application context.
- REQ_CRYPT_DES_CMAC: If scenarios with high attack potential for side channel attacks are applicable in the composite application context, then the DES CMAC functionality shall be used only once per key. Otherwise the confidentiality requirement of the generated DES CMAC SubKeys shall be assessed during the composite certification.

7 Pre-personalization (OS initialization)

This chapter describes the steps for the OS initialization of JCOP 4 P71. The OS initialization state diagram outlines the possible states and transitions until the OS initialization phase is closed.

JCOP 4 P71 provides as central configuration element a dedicated module (the Config Module) for the OS initialization of the card. As long as the module is available on-card, the product's configuration items can be modified. (See Section 2 - [Product identification](#) for information on how to get a list of available modules.) The OS initialization phase needs to be finished by deleting the Config Module.

The Config Module must be removed before the product is issued to the end customer (see Section 6.1 - [Procedural requirements](#)).

Fig. 7.1: OS initialization states

For further information about the configuration options, see Section 8 - [Product configuration](#).

The first time the IC is powered it is in OS initialization state. In this state, the Config Module can write OS initialization data to configure the product. The Config Module is part of JCOP 4 P71 and may therefore already be present on the card when it is powered on for the first time. (It is removed for configurations that are fully initialized prior to delivery.)

7.1 Transport state

After production, JCOP 4 P71 is already fully operational and in transport mode. Within the transport mode the available functionality is limited. The following operations can be performed:

- Selection of the ISD. This is optional, the default selected applet is the ISD.
- Retrieval of public information like IDENTIFY [5.1.1.1](#) or CPLC data [5.1.1.2](#).
- Pre-configuration of the card using signed APDUs from NXP (see Section [7.1.1 - PUT DATA \(Configuration\) commands in transport mode](#)).
- Initiate a secure channel with the ISD using the key set provided by the NXP helpdesk.
- Bulk update (see Section [7.4.3 - Bulk Update](#)).

After a successful authentication to the ISD the transport mode is disabled. All configurations and card content management activities can be done.

7.1.1 PUT DATA (Configuration) commands in transport mode

JCOP supports configuration commands via PUT DATA which can be sent to the ISD before the Transport Key is authenticated. The PUT DATA command allows modification of the same configuration items as the SET CONFIG ITEM command (see Section [7.4.4.2 - SET CONFIG ITEM command](#)). The data of the PUT DATA command is encrypted with NXP-owned keys and cannot be modified by a customer. The PUT DATA commands are processed by the Config Module in a way similar to the processing of SET CONFIG ITEM commands.

Further information will be provided on a need-to-know basis.

The command PUT DATA (Configuration) shall be formatted as follows:

Tab. 7.1: PUT DATA (Configuration) command format and parameter settings

Code	Value	Parameter settings
CLA	'00'	ISO/IEC 7816
INS	'DA'	PUT DATA (Configuration)
P1	'01'	Proprietary tag
P2	'FE'	ConfigModule
Lc	xx	Length of data field
Data	Data	provided by NXP

The PUT DATA (Configuration) command may return the following error status word:

Tab. 7.2: Error status words returned by PUT DATA (Configuration)

SW	Meaning	Condition
9000h	NO_ERROR	Successful processing. The config item(s) have been modified.
6A81h	FUNC_NOT_SUPPORTED	Config Module has been deleted. Configuration is no longer possible.
6985h	CONDITIONS_NOT_SATISFIED	Operation failed for another reason.
6A80h	WRONG_DATA	Operation failed for another reason.

7.2 Masking process

The JCOP OS consists of two different components, the base OS mask and the PHEAP image as part of the Flash content. Both are generated by NXP, and the PHEAP image may contain customer-specific components.

The upload of the Flash image is part of the silicon manufacturing process. The ordered product has this image pre-loaded. In some cases, the card manufacturer can also get a Flash image that can be applied using the Bulk Update feature (see [7.4.3 - Bulk Update](#)).

7.3 Key definition

The following keys need to be exchanged between the customer and NXP in a secure way before the OS can be used. For details on the key exchange see [Section 9.1 - Key exchange](#).

Transport Key, which is a key set with Key Version Number (KVN) 0xFF for the ISD with either is a DES or AES key set. The Transport Key is used to:

- allow the card to authenticate the customer and forbid unauthorized usage on its way from NXP to the customer site,
- authenticate to the product in order to start the OS initialization.

The default SCP protocol of the product is SCP02. To change the default SCP protocol, the config item SCP_ENABLE (tag 1057) can be set. NXP can also provide signed PUT DATA commands that can be sent to the card prior to authentication. Such PUT DATA commands re-configure the ISD to accept SCP01, SCP02 or SCP03 see [Section 8.3.5.1 - SCP_ENABLE](#).

7.4 Config Module

The Config Module is used for OS initialization of the product. It provides functionality to

- configure the card,
- perform a bulk update,
- re-enable the transport mode.

To close the OS initialization of the product, the ELF of the Config Module needs to be deleted using the GlobalPlatform command DELETE:

```
DELETE D276000085304A434F504D4F4406
```

Note: It is the sole responsibility of the card manufacturer to ensure that the Config Module is deleted and only products in card life cycle state *SECURED* are delivered to the end customer.

The following AIDs are defined for the Config Module:

Tab. 7.3: Config Module AID

AID	Meaning
D276000085304A434F504D4F4406h	Config Module Executable Load File

7.4.1 Velocity limitation

The ISD contains a counter for unsuccessful attempts to authenticate, see Section 5.9.2 - [Unsuccessful or incomplete authentication](#) for details.

The counter is reset after a successful authentication to the ISD.

7.4.2 OS initialization command processing

The following commands are used during the OS initialization phase. These commands are handled by a SD or the Card Manager and follow the command formatting as defined by GlobalPlatform [15].

The commands SELECT(by Name), SET STATUS and DELETE are part of the ISD. SET STATUS and DELETE are available only after the Card Manager was selected and authenticated.

7.4.2.1 SELECT (by name) command

The SELECT(by name) command allows an applet on the card to be selected. During the OS initialization it is used to select the ISD. The command format is identical to the SELECT command defined in the GlobalPlatform specification [15].

The command SELECT (by Name) shall be formatted as follows:

Tab. 7.4: SELECT (by Name) command format and parameter settings

Code	Value	Parameter settings
CLA	'00'	ISO/IEC 7816
INS	'A4'	SELECT (by Name)
P1	'04'	SELECT by Name
P2	'00'	First or only occurrence
Lc	'08' or '0B'	Length of data field
Data	AID	see below

With the following values for the data field:

AID length	AID	Meaning
08'	A000000151000000h	ISD

SELECT (ISD) response data is formatted as follows:

Tab. 7.5: Response data of SELECT (ISD)

Field	Length	Content	Remark
FCI Tag	'01'	'6F'	
FCI Length	'01'	'10'	
SD AID Tag	'01'	'84'	
SD AID Length	'01'	'08'	
ISD AID	'08'	'A000000151000000'	
Proprietary Data Tag	'01'	'A5'	
Proprietary Data Length	'01'	'04'	
Proprietary Data	'04'	'9F6501FF'	
SW	'02'	9000h	Normal ending

The SELECT (ISD) command may return status words as specified in the GlobalPlatform Card Specification [15].

7.4.2.2 SET STATUS command

This command sets the GP card life cycle status. The command format is identical to the SET STATUS command defined in the GlobalPlatform specification [15].

Note: It is the sole responsibility of the card manufacturer to ensure that only products in life cycle state *SECURED* are delivered to the end customer.

The command SET STATUS shall be formatted as follows:

Tab. 7.6: SET STATUS command format and parameter settings

Code	Value	Parameter settings
CLA	'80'	GlobalPlatform
INS	'F0'	SET STATUS
P1	'80'	Indicate Issuer Security Domain
P2	'XX'	Card life cycle state
Lc	'08'	Length of data field
Data	AID	see below

Where the card life cycle state 'XX' is one of the following:

Card life cycle state	Coding
<i>INITIALIZED</i>	'07'
<i>SECURED</i>	'0F'

Note: For a complete list of available states, refer to the GlobalPlatform specification [15], Section 5.1.1.

SET STATUS data field is formatted as follows:

Tab. 7.7: Data field of SET STATUS

Field	Length	Content	Remark
AID	'08'	'A000000151000000'	AID of ISD

The SET STATUS command may return status words as specified in the GlobalPlatform Card Specification [15].

7.4.2.3 DELETE command

This command is used to delete the Config Module at the end of the OS initialization phase. The command format is identical to the DELETE command defined in the GlobalPlatform specification [15].

Note: It is the sole responsibility of the card manufacturer to ensure that the Config Module is deleted before the product is delivered to the end customer.

The command DELETE (Config Module) shall be formatted as follows:

Tab. 7.8: DELETE (Config Module) command format and parameter settings

Code	Value	Parameter settings
CLA	'80'	GlobalPlatform
INS	'E4'	DELETE (Config Module)
P1	'00'	Last (or only) command
P2	'00'	Delete object and related object
Lc	'10'	Length of data field
Data	see below	Data field

DELETE (Config Module) data field is formatted as follows:

Tab. 7.9: Data field of DELETE (Config Module)

Field	Length	Content	Remark
Tag	'01'	'4F'	Tag for AID
Length	'01'	'0E'	Length of following AID
AID	'0B'	'D276000085304A434F504D4F4406'	AID of the Config Module load file

The DELETE command may return status words as specified in the GlobalPlatform Card Specification [15].

7.4.3 Bulk Update

The Bulk Update feature can be used to write a Flash image into the card. A bulk update is a set of APDUs provided by NXP. The sequence of those APDUs must not be altered. Bulk Update is not limited to replace the current Flash image with a different image, it can also be used to overwrite the currently active image in order to do a factory reset. The upload of a new Flash image may contain:

- Configuration settings,
- pre-loaded customer applets,
- OS modules (including the Config Module),
- Secure Box native library,
- OS patches.

A bulk update contains a sequence of APDUs which will be provided by NXP together with detailed instructions on how to apply them.

The IDENTIFY command can be used to verify the successful execution of the bulk update. The Flash identifier will be different after the bulk update. If the bulk update contains a patch, then the patch level reported by the IDENTIFY command will also be different.

Note: Bulk update will not alter the content of the attack counter log (see Section 5.1.3.3 - [Read attack counter log](#)) and chip individual data, for example the UID.

7.4.4 OS configuration

An essential part of the Config Module is to allow changing of the OS settings.

7.4.4.1 Access rules

- Customer-owned configuration item are listed in this document and can be modified by a customer
- NXP-owned configuration item are not listed in this document and will be provided by NXP when required. To access a NXP-owned configuration item via the APDUs SET CONFIG ITEM or GET CONFIG ITEM, a token needs to be presented in the command which authorizes the access.

7.4.4.2 SET CONFIG ITEM command

The SET CONFIG ITEM command is in essence a STORE DATA command to the ISD. The command uses the Config Module to permanently change the configuration items for JCOP 4 P71. All configuration items have a default value in the Flash image. A SET CONFIG ITEM APDU is needed only if the default value needs to be changed.

All customer configuration items can be changed using the SET CONFIG ITEM APDU (see Section 8 - [Product configuration](#)). SET CONFIG ITEM also multiple configuration items to be changed in one APDU. The entire STORE DATA command is executed in a transaction. So, none of the changes are performed if any error occurs.

Tab. 7.10: SET CONFIG ITEM command format and parameter settings

Code	Value	Parameter settings
CLA	'80'	GlobalPlatform
INS	'E2'	STORE DATA
P1	'88'	Last block/DGI format/case 3
P2	'00'	Block number
Lc	Var	Length of data field
Data	See below	Payload
Le	00	Length of response data

With the payload formatted as follows:

Tab. 7.11: Payload of APDU data field

Field	Length	Content	Comment
Config DGI	'02'	'DF2B'	Proprietary JCOP DGI identifying the Config Module for CUSTOMER access
Length DGI DF2B payload	'01'	var	Length in bytes of the config payload (the rest of the command data). Must be equal to Lc-2.
Tag 1	var	var	First configuration tag
Length 1	'01'	var	Length of the first configuration tag value
Value 1	var	var	Configuration value
...			
Tag n	var	var	nth configuration tag
Length n	'01'	var	Length of the nth configuration tag value
Value n	var	var	Configuration value

The SET CONFIG ITEM command may return the following error status word:

Tab. 7.12: Error status words returned by SET CONFIG ITEM

SW	Meaning	Condition
9000h	NO_ERROR	Successful processing. The config item(s) have been modified.
6982h	SECURITY_STATUS_NOT_SATISFIED	Not in a Secure Channel or one of the config item is NXP-owned so the operation is not allowed.
6985h	CONDITIONS_NOT_SATISFIED	One of the config items cannot be modified in the current state
6A80h	WRONG_DATA	One of the config item tag is invalid or one of the value of a config item is invalid. Note that in general, values are not checked for validity except in specific cases.
6700h	WRONG_LENGTH	One of the config item length is invalid
6A81h	FUNC_NOT_SUPPORTED	Config Module has been deleted. Configuration is no longer possible

7.4.4.3 GET CONFIG ITEM command

The GET CONFIG ITEM command is in essence a GET DATA command to the ISD. The command uses the Config Module to retrieve the configuration items for JCOP 4 P71.

Tab. 7.13: GET CONFIG ITEM command format and parameter settings

Code	Value	Parameter settings
CLA	'80'	GlobalPlatform
INS	'CA'	GET DATA
P1	'00'	
P2	'FE'	P1-P2 encodes proprietary tag FE. This tag is used by JCOP to access JCOP proprietary DGIs
Lc	Var	Length of data field
Data	See below	Payload
Le	00	Length of response data

With the payload formatted as follows:

Tab. 7.14: Payload of APDU data field

Field	Length	Content	Remark
Config DGI	'02'	'DF2B'	Proprietary JCOP DGI identifying the Config Module for CUSTOMER access
Length DGI DF2B payload	'01'	'02'	Length of the config tag to read.
Config item tag	'02'	var	Config item tag

Tab. 7.15: GET DATA (Config) response data

Code	Value	Meaning
Tag	'FE'	
Length	3+config data length	
DGI	'DF2B'	Identifies Config Module answer
Ldata	'xx'	Length of the config data
Config data	'xx...'	The value of the config item, in clear-text

The GET CONFIG ITEM command may return the following error status word:

Tab. 7.16: Error status words returned by GET CONFIG ITEM

SW	Meaning	Condition
9000h	NO_ERROR	Successful processing. The config items value is returned.
6982h	SECURITY_STATUS_NOT_SATISFIED	Not in a Secure Channel. One of the config item is NXP-owned so the request is not allowed.
6A80h	WRONG_DATA	One of the config item tag is invalid.
6A81h	FUNC_NOT_SUPPORTED	Config Module has been deleted. Configuration is no longer possible.

7.5 OS initialization command sequence

The general command processing of JCOP 4 P71 is described in [4.5.3 - Command processing](#).

After the card production only the ISD exists in the card to receive OS initialization commands. No applet has the Default Selected privilege, this privilege is owned by the ISD.

The following sequence of commands is typical for a complete OS initialization of JCOP:

1. Power on the card.
2. (Optional.) SELECT(by name) ISD instance (see Section [7.4.2.1 - SELECT \(by name\) command](#)).
3. (Optional.) Identify the card to ensure that the intended product is used (see Section [2 - Product identification](#)).
4. (Optional.) Re-configure the used SCP protocol for the transport key authentication.
5. Authenticate to the ISD using the NXP provided transport ISD keyset to open a secure communication channel according to [\[15\]](#).
6. (Optional.) Apply bulk update (see Section [7.4.3 - Bulk Update](#)).
7. Replace the transport ISD keyset with the real ISD keyset.
8. (Optional.) Send a sequence of SET CONFIG ITEM APDUs to configure the card (see Section [7.4.4.2 - SET CONFIG ITEM command](#)).

9. (Conditional.) Delete the Load File of the Config Module (see Section [7.4.2.3 - DELETE command](#)).
10. (Optional.) Load, install and/or personalize applets.
11. Set the card life cycle state to *INITIALIZED* or *SECURED* (see Section [7.4.2.2 - SET STATUS command](#)).
12. Reset the card.

8 Product configuration

JCOP 4 P71 provides a wide range of configuration options to adapt the behavior of the card to the customer's needs. These configuration options are described in this section.

The configuration items can be changed during the OS initialization phase by using SET CONFIG ITEM APDUs, see Section 7.4.4.2 - [SET CONFIG ITEM command](#). All configuration items have a default value in NVM. Sending a SET CONFIG ITEM APDU is only needed if the default value is to be changed.

8.1 Card Production Life Cycle (CPLC)

The Card Production Life Cycle data as defined in VISA GlobalPlatform Card Specification 2.1.1 are coded in 42 bytes. The CPLC format and the default values are shown in Table 8.1.

The customer CPLC data can be written during the OS initialization, see for details.

On request, NXP offers an additional service that allows customers to set customer CPLC data fields. These data can be entered as custom-specific data in the Order Entry Form (see Section 8.4 - [Customer's CPLC data \(option\)](#) for details).

Tab. 8.1: CPLC data

Field name	Length	Default Value	Owner
IC fabricator ¹	2	'4790'	NXP
IC type ¹	2	'D321'	NXP
Operating system identifier ¹	2	'47' '0'	NXP
Operating system release date ¹	2	'00' '00'	NXP
Operating system release level ¹	2	'00' '00'	NXP
IC fabrication date ¹²³	2	tt	NXP
IC Serial number ¹²³	4	nnnb	NXP
IC Batch identifier ¹²³	2	bb	NXP
IC module fabricator	2	'00' '00'	Customer
IC module packaging date ⁴	2	'00' '00'	Customer
ICC manufacturer	2	'00' '00'	Customer
IC embedding date	2	'00' '00'	Customer
IC pre-personalizer ⁵	2	WX	Customer
IC pre-perso date ⁵	2	YN	Customer
IC pre-perso equipment ID ⁵	4	NNNN	Customer
IC personalizer	2	'00' '00'	Customer

Continued on the next page.

Tab. 8.1 – Continued from the previous page.

Field name	Length	Default Value	Owner
IC personalization date	2	'00' '00'	Customer
IC personalization equipment ID	4	'00'... '00'	Customer

¹ These data elements shall not be modified.

² The combination of IC fabrication date, IC serial number and IC batch identifier represents a unique card number for all JCOP 4 P71 releases on smart card ICs from NXP Semiconductors.

³ Elements IC fabrication date, IC serial number, IC batch identifier, IC personalizer, IC personalization date and IC personalization equipment identifier are filled with production data by default (NXP Semiconductors internal values).

⁴ IC module packaging date is set when the module is delivered by NXP (same coding as for IC fabrication date), otherwise it is set to '00'... '00'.

⁵ IC pre-personalizer, IC pre-perso date, and IC pre-perso equipment ID are only filled by NXP Semiconductors if the delivery form is module, otherwise it is '00'... '00'.

Tab. 8.2: CPLC data elements

Code	Data element	Format	Length in bytes
t	Time stamp ¹	BCD	2
N	Die number ²	ASCII	4 to 5
n	Die number ²	BCD	3
b	Batch number ³	BCD	3
W	Wafer number ⁴	BINARY	1
X	X-coordinate ⁴	BINARY	1
Y	Y-coordinate ⁴	BINARY	1
S	Revision number ⁵	BINARY	6

¹ Time stamp format consists of 4 BCD-digits in 2 bytes. Digit 1 is the last digit of the year. For example 4 for the year 2014. Digits 2, 3, and 4 hold the day of the year. For example 225 for August 13th.

² Die numbers always start with 1. In combination with batch number a die-individuality over several batches can be achieved.

³ Batch numbers bbb consist of a serial number (3...5 bytes = max. 999999 to max. 9999999999) and they include no split or download part. Instead, if a new download (= new generation of FabKey variables) for the same batch is necessary, a new serial number is used. As BCD-Batch numbers (bbbb) can only be constructed out of decimal characters, a definite serial number over all production lots of all customers is created. This serial number has a defined back-track to the original batch number.

⁴ Wafer number and X-Y-coordinates are added by test equipment.

⁵ Revision number of the software versioning system.

8.2 Supported unique identifiers

JCOP 4 P71 products support single and double UID according to ISO/IEC 14443-3A for the following configurations, which can be ordered via the Order Entry Form:

- Double UID, length 7 bytes (default)
- Single UID, length 4 bytes
 - Derived from Double UID (FNUID)
 - Single UID as set in hardware
 - Random UID

8.3 Configuration options

JCOP 4 P71 can be configured using the SET CONFIG ITEM command during the OS initialization phase, for details see Section 7.4.4.2 - [SET CONFIG ITEM command](#).

Note: The card does not perform internal consistency checks when a configuration item is changed, except when explicitly specified. It is the sole responsibility of the organization performing OS initialization to ensure that the values are consistent and valid.

The default options through the OEF are:

Tab. 8.3: ATS configurations

	Default value
EMV: 106kbps ¹	106kbps
SECID: Up to 848kbps	848kbps
¹ The EMV specifications prohibit announcements of contactless speeds higher than 106kbps.	

Tab. 8.4: ATR configurations

Protocol	ATR
	Cold ATR default
T=1 (default)	(4x) 3B FA 13 00 00 91 01 31 FE 45 00 31 C1 73 C8 40 00 00 90 00 68
T=0	(4x) 3B FA 13 00 FF 10 00 00 31 C1 73 C8 40 00 00 90 00
	Warm ATR default
T=1	(1x) 3B EA 00 00 81 31 FE 45 00 31 C1 73 C8 40 00 00 90 00 7A

Continued on the next page.

Tab. 8.4 – Continued from the previous page.

Protocol	ATR
T=0	(1x) 3B 6A 00 FF 00 31 C1 73 C8 40 00 00 90 00

8.3.1 Contactless communication

Note: The communication settings of JCOP 4 P71 are highly configurable. Some configuration settings are reflected in particular bytes of the ATS. JCOP internally performs no consistency checks between a configuration setting and the corresponding indication in the ATS. In case of mismatch, communication with the card may become impossible. It is the sole responsibility of the card manufacturer to ensure a consistent configuration.

8.3.1.1 CIU_FWT_OFFSET

Allows the configuration of the time span which is subtracted from the calculated WTX interval to give JCOP enough time to process the WTX interrupt and send the WTX request.

JCOP internally generates interrupts to prepare and send WTX requests in a constant time. The time interval of the interrupts is defined by the Frame Waiting Time (FWT), see [4.7.2 - Protocol T=CL](#). (The interval is measured in microseconds.)

The time needed by JCOP to process the interrupt and prepare the WTX request is the FWT Offset (FWTO).

The FWTO is subtracted from the calculated FWT. In case the result is below 0, the WTX generation is disabled:

Interrupt interval = $FWT - FWTO$

Interrupt interval < 0: WTX generation disabled

See Section [4.7 - WTX configurations](#)

Definition:

Tag value	Length [Bytes]	Typical default value
'10C4'	2	'0DAC'

Allowed values:

All values are allowed.

8.3.1.2 TCL_L3_ACTIVATION_CONTROL

This configuration item determines how the firmware behaves during activation of the ISO/IEC 14443-3A protocol. The L3 activation control parameter comprises the UID handling.

This configuration item contains several bits, the bits are defined as follows (further detail in Table 8.6).

Tab. 8.5: L3 Activation Control Parameter

Bit 7	6	5	4	3	2	1	0	Definition
-	-	-	-	x	x	-	-	UID selection
Unused bits are set to '0'.								

Definition:

Tag value	Length [Bytes]	Typical default value
'10A5'	1	'04'

Allowed values:

Bits 2 and 3 define how the UID is selected:

Tab. 8.6: UID selection (Bits 2 and 3)

Bit value	Meaning
'00'	A random UID, generated by the OS, is used.
'01'	The UID stored in security row of JCOP 4 P71 is used.
'1X'	Uses the UID stored in security row of JCOP 4 P71 to calculate a single size fixed non-unique ID (FNUID). Note that the security row must have been personalized with a double size UID.

8.3.1.3 TCL_ATS_IF

Defines the T=CL interface Bytes used for the Contactless Interface Unit (CIU). These bytes are the first bytes in the ATS before the historical characters. The content of this configuration item is defined as follows:

Tab. 8.7: T=CL Interface Bytes

Interface Byte	Presence	Information
Length	Mandatory	Length of the following Bytes plus the Length byte itself
T0	Mandatory	
TA1	Conditional	
TB1	Conditional	
TC1	Conditional	If present: least significant bit must be '0'

The presence of the Interface Bytes TA1, TB1, and TC1 is conditional and depends on the value of the T0 Interface Byte. For details on the coding of these bytes see ISO 14443-4 [19].

Bit '0' of TC1 indicates 'NAD' which is not supported by JCOP and shall be set to '0'.

Bit '1' of TC1 indicates CID support. CID support is also indicated in the configuration item COMM_BEHAVIOR (see 8.3.3.3 - COMM_BEHAVIOR). Both indications shall match.

Definition:

Tag value	Length [Bytes]	Typical default value
'109E'	5	'05' '78' '80' '71' '02'

Allowed values:

For the first byte (Length): '01'... '04'

For the other bytes: all values which comply to ISO 7816-3 [17].

Note: To ensure the device has enough time to initialize, a minimum value of '1' must be used for the SFGI (Start-up Frame Guard time Integer) encoded on byte TB(1) of the ATS (Answer to Select).

8.3.1.4 TCL_ATS_CURRENT_HISTLEN

Defines the actually used length of the historical characters in configuration item TCL_ATS_HISTCHARS. This configuration item can also be changed by the Java Card API SysControl.setActivationParameters

Definition:

Tag value	Length [Bytes]	Typical default value
'109F'	1	'0A'

Allowed values:

The maximum allowed length of the historical characters depends on the presence of the ATS interface bytes TA1, TB1 and TC1 and is limited by the capabilities of the hardware (for a definition of the interface bytes TA1, TB1 and TC1 in configuration item TCL_ATS_IF see Section [8.3.1.3 - TCL_ATS_IF](#)). The maximum allowed length is 20 Bytes for the combined length of TA1, TB1, TC1 and the historical characters.

8.3.1.5 TCL_ATS_HISTCHARS

Defines the Historical Characters for the ATS. The currently used length of the historical characters is defined in configuration item TCL_ATS_CURRENT_HISTLEN, see Section [8.3.1.4 - TCL_ATS_CURRENT_HISTLEN](#) for details.

Definition:

Tag value	Length [Bytes]	Typical default value
'10A0'	max. 20	'00' '31' 'C1' '73' 'C8' '40' '00' '00' '90' '00'

Allowed values:

All values are allowed. The historical characters are copied directly into the ATS.

8.3.1.6 TCL_ATQA_MSB

Defines the most significant byte of the ATQA.

Definition:

Tag value	Length [Bytes]	Typical default value
'10A1'	1	'00'

Allowed values:

All values as defined in ISO/IEC 14443-3A [\[1\]](#).

8.3.1.7 TCL_ATQA_LSB

Defines the least significant byte of the ATQA.

Definition:

Tag value	Length [Bytes]	Typical default value
'10A2'	1	'48'

Allowed values:

All values as defined in ISO/IEC 14443-3A [1].

8.3.1.8 TCL_SAK_COMPLETE

Defines the SAK in case of a complete UID.

Definition:

Tag value	Length [Bytes]	Typical default value
'10A3'	1	'20'

Allowed values:

All values as defined in ISO/IEC 14443-3A [1].

8.3.1.9 TCL_SAK_INCOMPLETE

Defines the SAK in case of an incomplete UID.

Definition:

Tag value	Length [Bytes]	Typical default value
'10A4'	1	'24'

Allowed values:

All values as defined in ISO/IEC 14443-3A [1].

8.3.2 Contact-based communication

Note: The communication settings of JCOP 4 P71 are highly flexible configurable. Some configuration settings are reflected in particular bytes of the ATR. JCOP internally performs no consistency checks between a configuration setting and the corresponding indication in the ATR. In case of mismatch, communication with the card may become impossible. It is the sole responsibility of the card manufacturer to ensure a consistent configuration.

8.3.2.1 7816_ATR_COLD_CONF

Defines the communication settings for the ISO/IEC 7816 cold ATR.

Tab. 8.8: Communication settings for the cold ATR

Bit 7	6	5	4	3	2	1	0	Definition
-	-	-	-	-	-	0	-	Direct convention
-	-	-	-	-	-	1	-	Inverse convention
-	-	-	-	-	0	-	-	Check byte TCK not enabled
-	-	-	-	-	1	-	-	Check byte TCK enabled

Unused bits are set to '0'.

The initial character TS of the ATR will be set to 0x3B (direct convention) or 0x3F (inverse convention) depending on the configuration of bit b1. The TCK byte will only be appended to the ATR if bit b2 (check byte TCK enabled) is set.

Definition:

Tag value	Length [Bytes]	Typical default value
'1090'	1	'24'

Allowed values:

Any combinations of the above defined bits.

8.3.2.2 7816_ATR_COLD_IF_LEN

Defines the used length of the Interface Bytes for the cold ATR as specified in the configuration item 7816_ATR_COLD_IF, see Section [8.3.2.3 - 7816_ATR_COLD_IF](#).

Definition:

Tag value	Length [Bytes]	Typical default value
'108C'	1	'07'

Allowed values:

All values in the range '01'... '0F'

8.3.2.3 7816_ATR_COLD_IF

Defines the Interface Bytes for the cold ATR. JCOP always calculates the TS byte of the ATR and the TCK byte if indicated in 7816_ATR_COLD_CONF (see [8.3.2.1 - 7816_ATR_COLD_CONF](#)). This configuration item contains the Format byte T0 and the Interface bytes TA, TB, TC, and TD.

For further reading see ISO 7816-3 [[17](#)].

Note: The length indication of the interface bytes may needed to be changed, see Section [8.3.2.2 - 7816_ATR_COLD_IF_LEN](#).

Definition:

Tag value	Length [Bytes]	Typical default value
'108D'	max. 15	'E0' '00' '00' '81' '31' 'FE' '45'

Allowed values:

All values as defined in [[17](#)]

8.3.2.4 7816_ATR_COLD_HIST_LEN

Defines the used length of the Historical Characters for the cold ATR as specified in the configuration item 7816_ATR_COLD_HIST, see Section [8.3.2.5 - 7816_ATR_COLD_HIST](#).

Definition:

Tag value	Length [Bytes]	Typical default value
'108E'	1	'0A'

Allowed values:

All values in the range '00'... '0F'

8.3.2.5 7816_ATR_COLD_HIST

Defines the Historical Characters for the cold ATR. The Historical Characters follow the Interface Bytes in the ATR. Note that the length indication of the historical characters may needed to be changed, see Section [8.3.2.4 - 7816_ATR_COLD_HIST_LEN](#)

Definition:

Tag value	Length [Bytes]	Typical default value
'108F'	max. 15	'00' '31' 'C1' '73' 'C8' '40' '00' '00' '90' '00'

Allowed values:

No restriction, the historical characters can freely be chosen

8.3.2.6 7816_ATR_WARM_CONF

Defines the communication settings for the ISO/IEC 7816 warm ATR.

Tab. 8.9: Communication settings for the warm ATR

Bit 7	6	5	4	3	2	1	0	Definition
-	-	-	-	-	-	0	-	Direct convention
-	-	-	-	-	-	1	-	Inverse convention
-	-	-	-	-	0	-	-	Check byte TCK not enabled
-	-	-	-	-	1	-	-	Check byte TCK enabled

Unused bits are set to '0'.

The initial character TS of the ATR will be set to 0x3B (direct convention) or 0x3F (inverse convention) depending on the configuration of bit b1. The TCK byte will only be appended to the ATR if bit b2 (check byte TCK enabled) is set.

Definition:

Tag value	Length [Bytes]	Typical default value
'1098'	1	'24'

Allowed values:

Any combinations of the above defined bits.

8.3.2.7 7816_ATR_WARM_IF_LEN

Defines the used length of the Interface Bytes for the warm ATR as specified in the configuration item 7816_ATR_WARM_IF, see Section 8.3.2.8 - [7816_ATR_WARM_IF](#).

Definition:

Tag value	Length [Bytes]	Typical default value
'1094'	1	'07'

Allowed values:

All values in the range '01'... '0F'

8.3.2.8 7816_ATR_WARM_IF

Defines the Interface Bytes for the warm ATR. JCOP always calculates the TS byte of the ATR and the TCK byte if indicated in 7816_ATR_WARM_CONF (see [8.3.2.6 - 7816_ATR_WARM_CONF](#)). This configuration item contains the Format byte T0 and the Interface bytes TA, TB, TC, and TD.

For further reading see ISO/IEC 7816-3 [17].

Note: Transport media type coding: The length indication of the interface bytes may needed to be changed, see Section [8.3.2.7 - 7816_ATR_WARM_IF_LEN](#).

Definition:

Tag value	Length [Bytes]	Typical default value
'1095'	max. 15	'E0' '00' '00' '81' '31' 'FE' '45'

Allowed values:

All values as defined in [17].

8.3.2.9 7816_ATR_WARM_HIST_LEN

Defines the used length of the Historical Characters for the warm ATR as specified in the configuration item 7816_ATR_WARM_HIST, see Section [8.3.2.10 - 7816_ATR_WARM_HIST](#).

Definition:

Tag value	Length [Bytes]	Typical default value
'1096'	1	'0A'

Allowed values:

All values in the range '00'... '0F'

8.3.2.10 7816_ATR_WARM_HIST

Defines the Historical Characters for the warm ATR. The Historical Characters follow the Interface Bytes in the ATR.

Note that the length indication of the historical characters may needed to be changed, see Section [8.3.2.9 - 7816_ATR_WARM_HIST_LEN](#)

Definition:

Tag value	Length [Bytes]	Typical default value
'1097'	20	'00' '31' 'C1' '73' 'C8' '40' '00' '00' '90' '00'

Allowed values:

No restriction, the historical characters can freely be chosen.

8.3.2.11 7816_ATR_WARM_WT_T0

Defines the waiting time integer after a warm reset for the T=0 protocol.

Definition:

Tag value	Length [Bytes]	Typical default value
'1099'	1	'0A'

Allowed values:

All values as defined in ISO/IEC 7816-3 [17] ('01'... 'FF').

8.3.2.12 7816_ATR_WARM_WT_T1

Defines the character and block waiting time integer after a warm reset for the T=1 protocol. The coding of this configuration item is identical to coding of the first TB byte for the T=1 protocol defined in ISO/IEC 7816 [17]:

Tab. 8.10: T=1 warm ATR waiting time integers

Bit 7	6	5	4	3	2	1	0	Definition
-	-	-	-	x	x	x	x	Character Waiting Time Integer
x	x	x	x	-	-	-	-	Block Waiting Time Integer

Unused bits are set to '0'.

Definition:

Tag value	Length [Bytes]	Typical default value
'109A'	1	'45'

Allowed values:

All values as defined in [17]:

Character Waiting Time: '0'... 'F',

Block Waiting Time: '0'... '9'.

8.3.2.13 WAITING_TIME_MARGIN

Additional margin to trigger WTX interrupt earlier than specified in ISO/IEC 7816-3 [17]. The unit of this item is 16*D etu.

Definition:

Tag value	Length [Bytes]	Typical default value
'10AE'	1	'00'

Allowed values:

Any value in the range '00' to 'FF'

8.3.2.14 WTXI

The BWT multiplier to transmit in the INF of a T=1 WTX request.

Definition:

Tag value	Length [Bytes]	Typical default value
'10AF'	1	'01'

Allowed values:

Any value in the range '01' to 'FF'

8.3.3 General communication

8.3.3.1 PPS_HANDLING

Configures the handling of PPS.

- Bit1 - 0=PPS handling as per ATS definition(CL interface), 1=override ATS and allow PPS.
- Bit0 - 0=PPS handling as per ATR definition(CT interface), 1=override ATR and allow PPS.

Definition:

Tag value	Length [Bytes]	Typical default value
'10AD'	1	'01'

Allowed values:

Any value in the range '00' to '03'

8.3.3.2 NR_OF_LOGICAL_CHANNELS

Defines the maximum number of logical channels. Any attempt to open more logical channels than the maximum number will result in error status word 6A81h (FUNC_NOT_SUPPORTED). The basic channel with the channel ID 0 is included, that is, the value '01' allows only the basic logical channel to open.

The number of logical channels can be set to 0 to allow GlobalPlatform 2.1.1 compatibility. When set to 0 the logical channel information on the CLA byte is ignored by the card.

Note: The number of supported logical channels cannot be decreased below the number of logical channels currently in use. Otherwise, the status word 6985h (CONDITIONS_NOT_SATISFIED) is returned.

Definition:

Tag value	Length [Bytes]	Typical default value
'2015'	1	'01'

Allowed values:

Any value in the range of '00'... '04'.

8.3.3.3 COMM_BEHAVIOR

Configures the behavior of the communication interfaces.

This configuration item contains several bits, the bits are defined as follows.

Tab. 8.11: Communication behavior

Bit 7	6	5	4	3	2	1	0	Definition
-	-	-	-	-	-	-	0	RFU
-	-	-	-	-	-	0	-	EMVCo incompatible
-	-	-	-	-	-	1	-	EMVCo compatible
-	-	-	-	-	0	-	-	I-block number check disabled
-	-	-	-	-	1	-	-	I-block number check enabled
-	-	-	-	0	-	-	-	SB114 warning support disabled
-	-	-	-	1	-	-	-	SB114 warning support enabled
-	0	-	-	-	-	-	-	Extended length APDU lock disabled (extended APDU reaches standard applets which do not implement the extended length interface).
-	1	-	-	-	-	-	-	Extended length APDU lock enabled (extended APDU will not reach standard applets which do not implement the extended length interface).
0	-	-	-	-	-	-	-	RFU

Unused bits are set to '0'.

- EMV compatibility:

1 = EMVCo_Mode, 0 = ISO/IEC 7816-3.

This has impact on the BWI, CWI, guard time and max character repetition behavior in accordance with the EMVCo EMV 4.3 Book 1 [7] and ISO/IEC 7816-3 [17] specifications.

- I-block number check:

Enable or disable block number check for a received I-block.

- SB114 warning support:

This controls how warning status words are handled.

0 = EMVCo scenario A7; 1 = EMVCo SB114 behavior.

Query whether warning conditions (62xx and 63xx) are handled according to the EMVCo EMV 4.3 Book 1 [7] specification, Annexe A, and EMVCo Specification Bulletin no. 114 [8].

- Extended length APDU

APDUs with extended length are forwarded also to applets which do not implement the ExtendedLength interface.

Definition:

Tag value	Length [Bytes]	Typical default value
'109C'	1	'42'

Allowed values:

See Table 8.11

8.3.3.4 PROTOCOL_COMPLIANCY

The Java Card, GlobalPlatform and EMVCo specifications differ in the returned error status word in some aspects related to application selection. Also the value returned by `APDU.getProtocol()` can be modified. This configuration item is used to define JCOP's behavior regarding these differences.

The following JCOP behavior can be modified:

Tab. 8.12: Adjustable JCOP behavior

Position	Feature	Allowed values	Modification	Default
0	Select file with invalid AID	'01' '02'	6999h (APPLET_SELECT_FAILED) 6A82h (FILE_NOT_FOUND)	'02'
1	Select file while already selected	'01' '02'	6999h (APPLET_SELECT_FAILED) 6A82h (FILE_NOT_FOUND)	'02'
2	Invalid CLA in MANAGE CHANNEL	'01' '02'	6882h (SECURE_MESSAGING_NOT_SUPPORTED) 6E00h (CLA_NOT_SUPPORTED)	'02'
3	Transport media type coding in <code>APDU.getProtocol()</code>	see below	see below	'02'
4...7	RFU	'00'	not used	'00'

Definition:

Tag value	Length [Bytes]	Typical default value
'4004'	8	'02' '02' '02' '02' '00' '00' '00' '00'

Allowed values:

This configuration item consists of 8 Bytes, any unused Bytes shall be set to '00'. The other bytes shall be set to a value as described below. The allowed values to modify the returned status words are coded as follows:

- '01': Java Card compliant
- '02': GlobalPlatform compliant

Other values shall not be used.

Transport media type coding

The transport media type coding modifies the return value of `APDU.getProtocol()`. A Java Card compliant setting returns the value as-is. The alternative coding returns only the transport protocol type (T=0 or T=1) in the lower nibble; the transport media in the higher nibble is set to '0'.

Please note that the coding is different to the coding of the returned status word.

The allowed values to modify the transport media type coding are as follows:

- '01': alternative coding
- '02': Java Card compliant

Other values shall not be used.

8.3.3.5 APDU_BUFFER_SIZE

Defines the size of the APDU buffer array. Any attempt to configure a value outside of the allowed range will result in error status word 6A81h (`FUNC_NOT_SUPPORTED`). Increasing the size of the APDU buffer will lead to a reduction of the available THEAP. If not enough free THEAP is available, the command will fail with status word 6985h (`CONDITIONS_NOT_SATISFIED`). It is recommended to use a buffer size as a multiple of 16 minus 2 bytes for optimal resource allocation. The remaining THEAP can be checked using the `GET_AVAILABLE_MEMORY` configuration item after modifying of the configuration item. This configuration item needs to be sent as a single item (multiple configurations in the same `STORE DATA` are not allowed).

Definition:

Tag value	Length [Bytes]	Typical default value
'F826'	2	'10F'

Allowed values:

The allowed values are any values between '10E' and '800'.

8.3.3.6 SET_GLOBAL_CVM

Allows the initial value of the Global CVM to be set. The command data comprises tag, length, format, PIN length and PIN value:

- Format: '1', '2' or '3' where 1 = ASCII, 2 = BCD and 3 = HEX (defined by GlobalPlatform).
- PIN length: length of following PIN value field.
- PIN value: variable.

For example: 'F82906020411223344' represents a global CVM 6 Bytes long in BCD format with the PIN value '11223344'.

Definition:

Tag value	Length [Bytes]	Typical default value
'F829'	Following value field	

Allowed values:

As described above.

8.3.3.7 VHBR_ENABLED

This configuration item makes it possible to enable and disable the VHBR support to the JCOP platform. By default the configuration item is enabled.

Definition:

Tag value	Length [Bytes]	Typical default value
'10C1'	1	5A

Allowed values:

- '5A' (TRUE8): VHBR is supported.
- 'A5' (FALSE8): VHBR is not supported.

8.3.4 FIPS support

FIPS support requires the FIPS module to be present and for FIPS mode to be enabled (see Section 8.3.4.1 - [FIPS_COMPLIANCE](#)). Cryptographic algorithm self-tests can be configured to execute automatically on device reset or can be configured to be on request. A request to execute the FIPS cryptographic self-tests is made by sending a GET DATA APDU to the ISD.

This configuration item contains several bits, the bits are defined as follows.

Tab. 8.13: FIPS settings

Bit 7	6	5	4	3	2	1	0	Definition
-	-	-	-	-	-	-	X	'0': FIPS mode off. '1': FIPS mode enabled. (ISD must be set to use SCP03 before FIPS mode can be enabled.)
-	-	-	-	-	-	-	-	N/A
-	-	-	-	-	X	-	-	'0': ROM integrity check at start-up off. '1': ROM integrity check on start-up active.
-	-	-	-	X	-	-	-	'0': Crypto self-tests on request (via GET DATA APDU to the ISD). '1': Crypto self-tests on start-up.
Unused bits are set to '0'.								

If FIPS mode is enabled, JCOP will block usage of non-FIPS approved algorithms.

FIPS approved supported algorithms are as follows:

- RSA 2048 and 3072 key generation (CRT and Plain supported) including a pair-wise consistency check on generation. The exponent value shall be restricted to 0x010001.
- RSA PKCS#1 sign (key sizes $\geq$ 2048 with SHA256, SHA384 or SHA512)
- RSA PKCS#1 verify (key sizes $\geq$ 2048 with SHA1, SHA224, SHA256, SHA384 or SHA512)
- RSA PSS sign (key sizes $\geq$ 2048 with SHA256, SHA384 or SHA512)
- RSA PSS verify (key sizes $\geq$ 2048 with SHA1, SHA224, SHA256, SHA384 or SHA512)
- RNG

- TDES using 3 keys (keys require a usage counter as number of blocks encrypted/decrypted is restricted to a maximum of 0x100000 blocks: NIST SP 800-67 Revision 2 section 3.4 [5])
- AES 128, 192 and 256 (ECB and CBC)
- AES CMAC and key derivation
- SHA-1, SHA224, SHA256, SHA384 and SHA512
- ECC DSA sign, verify and key generation (NIST curves ≥ 224)

In addition, only SCP03 will be supported.

Note: SCP02 and TDES using 2 keys are therefore deactivated based on FIPS rules. As a result, EMV applets may not work correctly when FIPS mode is enabled.

8.3.4.1 FIPS_COMPLIANCE

Configuration of integrity checks according to FIPS [20].

Definition:

Tag value	Length [Bytes]	Typical default value
'108A'	1	'E1'

Allowed values:

See Table 8.13.

8.3.5 GlobalPlatform

8.3.5.1 SCP_ENABLE

Defines the supported secure channel protocols. Each supported SCP protocol is coded on two Bytes: the first byte specifies the SCP protocol, the second byte the supported protocol option.

JCOP provides 5 slots for the definition of the SCPs, each slot consists of two Bytes. Example:

SCP protocol	Option ("i")	Meaning
'02'	'55'	SCP02, i='55'

Unused slots shall be set to '00 00'.

The initial keyset (KVN 0xFF) must still be present, that is a new keyset has not been stored, otherwise the command will fail with status word 6985h (CONDITIONS_NOT_SATISFIED)

Definition:

Tag value	Length [Bytes]	Typical default value
'1057'	10	'02 55' '00 00' '00 00' '00 00' '00 00'

Allowed values:

Any supported combination of the SCP protocol and corresponding protocol option.

Each SCP protocol shall only be listed once in this configuration item. The item must contain at least one SCP, otherwise the command will fail with status word 6985h (CONDITIONS_NOT_SATISFIED).

If a SCP is removed and that SCP is currently in use on some logical channel, then the secure channel is forcibly closed. This includes the logical channel in which the command to modify the config item is sent.

The new value of SCP_ENABLE cannot exclude a SCP that is currently configured as supported for an existing Supplementary Security Domain, otherwise the command will fail with status word 6985h (CONDITIONS_NOT_SATISFIED). This applies to SSDs only, not to the ISD.

8.3.5.2 IDENTIFY_CMD

JCOP 4 P71 makes it possible to restrict the availability of the IDENTIFY command (see Section [5.1.1.1 - GET DATA IDENTIFY](#)).

The identification commands GET DATA(CPLC) and GET DATA(Platform ID) can be restricted by the configuration item PROPRIETARY_GET_DATA_DISABLED, see Section [8.3.5.3 - PROPRIETARY_GET_DATA_DISABLED](#).

Definition:

Tag value	Length [Bytes]	Typical default value
'2010'	1	'5A'

Allowed values:

- '5A': IDENTIFY command available,
- 'A5': IDENTIFY command not available.

8.3.5.3 PROPRIETARY_GET_DATA_DISABLED

Used to disable GET DATA commands with tag indicating proprietary data ('00FE'). The availability of the IDENTIFY command (see Section 5.1.1.1 - [GET DATA IDENTIFY](#)) can be restricted by the configuration item IDENTIFY_CMD (see 8.3.5.2 - [IDENTIFY_CMD](#)). Reading the attack counter log (see Section 5.1.3.3 - [Read attack counter log](#)) is always allowed. For details on the GET DATA commands with the tag indicating proprietary data ('00FE') see Section 5.1 - [Additional APDUs](#).

Definition:

Tag value	Length [Bytes]	Typical default value
'2016'	1	'A5'

Allowed values:

- 'A5': No restriction on GET DATA command with tag indicating proprietary data.
- '5A': GET DATA command with tag indicating proprietary data disabled.

8.3.5.4 GET_DATA_ACCESS_CONTROL

Bitmask to enforce the authentication to ISD to access different tags with proprietary GET DATA command. Each bit of the bitmask encodes the access: 0: accessible, 1: accessible only once authenticated to the ISD. See below for P2 coding.

Definition:

Tag value	Length [Bytes]	Typical default value
'201F'	2	'0000'

Allowed values:

P2 is coded as follows:

Tab. 8.14: Coding of P2

Bit 7	6	5	4	3	2	1	0	Definition
-	-	-	-	-	-	-	X	DF20
-	-	-	-	-	-	X	-	DF25

Continued on the next page.

Tab. 8.14 – Continued from the previous page.

Bit 7	6	5	4	3	2	1	0	Definition
-	-	-	-	-	X	-	-	DF26
-	-	-	-	X	-	-	-	DF28
-	-	-	X	-	-	-	-	'0066'
-	-	X	-	-	-	-	-	'9F7F'

Unused bits are set to '0'.

8.3.6 Other configuration items

8.3.6.1 CUSTOM_CPLC_DATA

CPLC data as defined by the Customer, see Section [8.1 - Card Production Life Cycle \(CPLC\)](#) for details.

Definition:

Tag value	Length [Bytes]	Typical default value
'2018'	24	Customer CPLC data

Allowed values:

Valid CPLC data.

8.3.6.2 TRANSPORT_STATE

Enables or disables the transport state of the product. This can be used to limit the functionality when the product is shipped from one site to another one. When re-enabling the transport state, no APDU to the ISD is possible without re-authentication to the ISD.

Definition:

Tag value	Length [Bytes]	Typical default value
'201E'	1	'5A'

Allowed values:

- 0x5A: Transport state activated
- 0xA5: Transport state deactivated

8.3.6.3 MAX_SUPPORTED_RSA_KEYLEN_BIT

Defines the maximum RSA key size in bit. Any attempt to configure a value outside of the allowed range will result in error status word 6A81h (FUNC_NOT_SUPPORTED). Increasing the maximum key size will lead to a reduction of the available THEAP. If not enough free THEAP is available, the command will fail with status word 6985h (CONDITIONS_NOT_SATISFIED).

It is recommended to check the available remaining THEAP using the GET_AVAILABLE_MEMORY configuration item after modifying the configuration item.

Definition:

Tag value	Length [Bytes]	Typical default value
'108B'	2	'0800'

Allowed values:

The allowed values are:

- '0800': Keys up to 2048 bit.
- '1000': RSA keys up to 4096 bit are supported.

Note: Support for 4096 RSA key length reduces the available THEAP size. When the THEAP is already filled up, the STORE DATA command fails with status word 6985h (CONDITIONS_NOT_SATISFIED).

8.3.6.4 MIFARE_APPLET_AID

The remote interface for MIFARE DESFire EV2 or MIFARE Plus EV1 requires to configure the instance AID of the specific MIFARE applet. The first byte in the configuration item denotes the length of the AID. The following 16 Bytes contain the AID, padded with zeros when the AID is shorter than 16 Bytes.

Definition:

Tag value	Length [Bytes]	Typical default value
'10C3'	17	'0CD276000085304A434F90010100000000'

Allowed values:

- '0CD276000085304A434F90010100000000': The configuration value that should be used for the MIFARE DESFire EV2 applet

- '0CD276000085304A434F90000100000000': The configuration value that should be used for the MIFARE Plus EV1 applet
- '00000000000000000000000000000000': When the remote interface is not required.

8.4 Customer's CPLC data (option)

All agreements related to intended CPLC entries and its file transfer/format and verification modalities from the customer to NXP Semiconductors are part of the Order Entry Form. An additional CPLC (FabKey) Data Transfer Form has to be used for the specification of data elements. Default values of CPLC data are defined in the Order Entry Form and in [8.1 - Card Production Life Cycle \(CPLC\)](#). It is possible to pre-define the following customer specific CPLC data:

- IC module fabricator
- IC module packaging date
- ICC manufacturer
- IC embedding date
- IC OS initializer
- IC OS initialization date
- IC OS initialization equipment identifier
- IC personalizer
- IC personalization date
- IC personalization equipment identifier

9 Ordering and delivery

9.1 Key exchange

The Transport Key will be specified by NXP and delivered to the customer as explained below.

To get access to the Transport Key of a JCOP 4 P71 product, send a request to retrieve this key to the email address **fabkey.bli@nxp.com**. In order to properly process this request, the FabKey Help Desk needs to identify the product of your request either by your Order Entry Form number or with the complete output of the IDENTIFY command (see [5.1.1.1 - GET DATA IDENTIFY](#)).

The request will be processed by the FabKey Help Desk. Be aware that the response containing the transport key needs to be sent encrypted. Therefore the GPG key needs to be exchanged with the FabKey Help Desk before, where the Help Desk is required to verify the GPG fingerprint. This fingerprint needs to be submitted via a different communication channel from the GPG key itself.

The response from the Help Desk is confidential and needs to be handled under security and export control rules of the product and its documentation.

Note that individual Transport Keys are defined for different products.

NXP Semiconductors can only accept requests for Transport Keys after the customer has already received products of that type number.

9.2 Customer type submission using Order Entry Form

The Order Entry Form (OEF) is used to transfer commercial and technical information on each new JCOP 4 P71 “Commercial Type” from the customer to NXP Semiconductors. The Order Entry Form is provided by a web-based interface as a subsection of the NXP Customer Extranet. This information is used to produce sample pieces (Customer Qualification Samples (CQS)) for custom configuration products and, after verification at customer site, the agreed volume.

The information consists of:

1. Commercial agreements on sample delivery and volume delivery (mandatory)
2. Customer's product options (mandatory)
3. Customer's additional ROM package data (optional)
4. Customer's CPLC data (optional)

9.2.1 Commercial agreements on sample delivery and volume delivery

Refer to “Order Entry Form JCOP 4 P71” for all agreements. It is recommended to check the form entries together with customer’s local NXP Semiconductors Marketing & Sales representative. Any deviation of order type with regard to standard specification has to be agreed on. The new type will receive a commercial type name representing information on family type, Flash data, FabKey number, delivery type and export regulation.

9.3 Customer Flash data

All agreements related to new additional Flash data for JCOP 4 P71 custom configuration products and its file transfer/format and verification modalities from the customer to NXP Semiconductors are part of the Order Entry Form.

9.4 Product delivery

This section describes the measures that are needed to ensure secure delivery of a JCOP product. For details see the wafer and delivery specification of the hardware [24].

NXP offers two ways of delivery of the product:

1. The customer collects the product at the NXP site (“Collection”).
2. The product is sent by NXP to the customer. To guarantee that the product is not manipulated during the delivery, the product is delivered in parcels sealed with special tape. The tape is printed with consecutive numbers and has special adhesive features which make any manipulation visible. NXP encloses a form in the parcel which the customer is asked to return. By this NXP is informed that the customer has received the undamaged parcel (“Shipment”).

Both methods guarantee that the customer gets authentic products. Additionally the customer can use the Transport Key to authenticate the chip.

9.4.1 Delivery as wafer

When the product is delivered as wafer there reside functional and non functional dies on the wafer. The non-functional dies must be destroyed to such an extent that no analysis or misuse is possible after destruction. The non-functional dies (scrap) have to be handled secure until the destruction.

NXP offers information about the defect dies by means of online access to an electronic wafer map file within a secured web site of NXP Semiconductors.

Collection of wafers

The customer fetches the product from the warehousing and distribution department:

NXP Semiconductors Germany GmbH

Warehousing & Distribution Department (W&D Hamburg)
Tropolowitzstr. 20
22529 Hamburg
Germany

Shipment of wafers

The product is delivered directly to the customer or via one of the following Regional Distribution Centers:

NXP Semiconductors RDC Europe	NXP Semiconductors RDC APAC	NXP Semiconductors USA Inc.
c/o UPS Supply Chain Solutions Hal B, Veldenweg 3 NL-6075 Herkenbosch The Netherlands	c/o EXEL Hong Kong Ltd. 2F, Wyler Center 1 1200 Tai Lin Pai Road Kwai Chung, N.T. Hong Kong	c/o FedEx Global Supply Chain Services Raines Road, Shelby 38118 Memphis USA

9.4.2 Delivery as module

When the product is provided as module, functional and non-functional packaged dies are delivered by NXP on a tape. The non-functional modules have a hole punched into them. Non-functional modules and smart cards must be destroyed to such an extent that no analysis or misuse is possible after destruction. These non-functional items have to be securely stored until destruction.

Collection of modules

The customer fetches the product from the following locations:

NXP Semiconductors (Thailand)	NXP Semiconductors Germany GmbH
303 Chaengwattana Rd. Laksi Bangkok 10210 Thailand	Stresemannallee 101 22529 Hamburg Germany

Shipment of modules

This delivery method is identical to the shipment of wafers as described in the previous section, see Section 9.4.1 - [Delivery as wafer](#).

10 Appendix

10.1 JCOP system AIDs

JCOP 4 P71 contains some components developed in Java Card. The following table includes the AIDs used by JCOP. The AIDs are listed here for information purpose only.

Tab. 10.1: JCOP 4 P71 package AIDs

AID	Meaning
A0000001515350h	Security Domain package
A0000000620101h	javacard.framework package
A0000000620001h	java.lang package
A0000000620002h	java.io package
A0000000620102h	javacard.security package
A0000000620209h	javacardx.apdu package
A0000000620201h	javacardx.crypto package
A0000000620203h	javacardx.external package
A00000015100h	org.globalplatform package
A000000151435254h	JCOP internal
D276000085304A434F4001h	JCOP internal
D276000085304A434F4101h	JCOP internal
D276000085304A434F505Ah	JCOP internal
D276000085304A434F504D4F4406h	Configuration Module package
D276000085304A434F504D4F4401h	com.nxp.id.jcop.ecc
D276000085304A434F504D4F4407h	com.nxp.id.jcop.fips
D276000085304A434F4003h	com.nxp.id.jcop.globalplatform.auxiliary
D276000085304A434F504D4F4402h	com.nxp.id.jcop.koreanseed
D276000085304A434F504D4F4400h	com.nxp.id.jcop.rsakeygen
D276000085304A434F5063h	com.nxp.id.jcopx.accelerator
D276000085304A434F506Eh	com.nxp.id.jcopx.egovaccelerators
D276000085304A434F506Fh	com.nxp.id.jcopx.math
D276000085304A434F5078h	com.nxp.id.jcopx.memory
D276000085304A434F5079h	com.nxp.id.jcopx.mifaresupport_4
D276000085304A434F504D4F440Ch	com.nxp.id.jcopx.paceim
D276000085304A434F5076h	com.nxp.id.jcopx.piv
D276000085304A434F5072h	com.nxp.id.jcopx.puf
D276000085304A434F5071h	com.nxp.id.jcopx.rawcomm
D276000085304A434F505Fh	com.nxp.id.jcopx.securebox
D276000085304A434F5058h	com.nxp.id.jcopx.security

Continued on the next page.

Tab. 10.1 – Continued from the previous page.

AID	Meaning
D276000085304A434F5077h	com.nxp.id.jcopx.system
D276000085304A434F5059h	com.nxp.id.jcopx.util

Tab. 10.2: JCOP 4 P71 Instance AIDs

AID	Meaning
A000000151000000h	Issuer Security Domain
A000000151535041h	Security Domain module

10.2 A note on ECDH GM

Step 3 in the PACE authentication protocol includes calculation of mapped generator GMap according to the formula (generic mapping): $GMAP = G * s + H$ Where:

- G is the domain base point
- s is the random nonce generated by the card in previous step
- H is the shared secret computed by ECDH exchange using keys in standard domain.

See Section 4.3.1, Generic Mapping, in [13] “Supplemental Access Control for Machine Readable Travel Documents”. The NXP proprietary interface of PACE ECDH mapping offers an efficient implementation that combines the calculations of step 2 and step 3.

10.2.1 Proprietary implementation

The NXP proprietary implementation uses an ECC key agreement object with a proprietary algorithm value. The method that implements the PACE mapping step is `javacard.security.KeyAgreement.generateSecret()`. Note that although this method is called in PACE step 3, it also calculates the shared secret H from PACE step 2. See detailed sub-steps in [10.2.2](#).

Both `byte[]` parameters to this method, `publicData` and `secret`, are used as input/output parameters with proprietary data format.

Input data items:

- TermPubKey: terminal public key in standard domain G.
- TermMappedPubKey: terminal ephemeral public key in mapped domain GMAP.
- Nonce_s: random nonce generated in previous step.
- OID: encoding of selected algorithm and mapping type.

Output data items:

- TermMappedPubKey: terminal ephemeral public key (same as input data).
- CardMappedPubKey: card ephemeral public key in mapped domain GMAP (generated in this step).
- ShareSecret: shared secret for PACE authentication (generated in this step).

10.2.2 Sub-steps performed

The method generateSecret() with proprietary algorithm PR_ALG_EC_DH_MAPPING performs the following sub-steps:

- Initial shared secret
Compute shared secret H in standard domain from PACE step 2:
 $H = \text{TermPubKey} * \text{CardPrivKey}$
- Mapped generator
Compute mapped generator:
 $GMap = G * s + H$
- New key pair
Generate new ECDH key pair in mapped domain:
 $\text{MappedPubKey} = \text{CardMappedPrivKey} * GMap$
- New shared secret
Compute shared secret in mapped domain:
 $\text{SharedSecret} = \text{CardMappedPrivKey} * \text{TermMappedPubKey}$

10.2.3 KeyAgreement class

To use the proprietary PACE mapping and key agreement step, create an object using proprietary algorithm tag = 0x80.

Code example:

```
static final byte PR_ALG_EC_DH_MAPPING = (byte) 0x80;  
KeyAgreement PACEkeyAgree =  
KeyAgreement.getInstance(PR_ALG_EC_DH_MAPPING, false);
```

10.2.4 KeyAgreement.generateSecret() proprietary method

```

short generateSecret(byte[] publicData, short publicOffset, short publicLength, byte[] secret,
short secretOffset)
  
```

The following table summarizes the input and output data parameters for generic mapping:

Tab. 10.3: Generic mapping input and output parameters

	publicData byte array	secret byte array
Input	Mapped term. public key in GMap	Padded nonce, Term. public key in G
Output	Mapped card public key in GMap	sharedSecret, Mapped term. public key in GMap

The parameters are formatted as follows:

publicData: input data format

Terminal mapped public key (point on curve) is received as input to General Authenticate in PACE step 3, with a TLV structure as shown below. The publicData byte array input is only the value field (without the tags), so the array offset is the offset of the '04' point encoding byte:

```

<tag='7C'><length>
  
```

```

<tag='83'><length>
  
```

```

04<mapped terminal public key> // pass array offset of '04<public key>' in publicData param.
  
```

secret: input data format

The input data passed in 'secret' parameter is a concatenation of the following 3 elements:

- random nonce_s, padded with leading zeros to the key size (curve prime size),
- 4 bytes filled with zero (RFU), and
- OID and terminal public key from step 2, with TLV structure as shown below:

```

<tag='7F49'><length>
  
```

```

<tag='06' 0ID><length='0A'><Crypto. mechanism ref.> // same as data received in MSE SET
  
```

```

<tag='86' pubkey><length>
  
```

```

04< terminal public key> // public key in standard domain, received in step 2
  
```

publicData: output data format

Card ephemeral mapped public key (point on in mapped curve), to be returned as APDU output data as follows:

```

<tag='7C'><length>
  
```

```

<tag='84'><length> // applet needs to set tag='84'
  
```

```

04<mapped card public key> // array offset passed to method
  
```

Note: The mapped terminal public key value (input) is replaced by mapped card public key value. The TLV tags ('7C' and '83') and length on the same buffer are not modified by the mapping method. The applet needs to change the input tag='83' to output tag='84' before returning the data to terminal.

secret: output data format

The output data returned in the `secret` parameter is a concatenation of the following 3 elements:

- New shared secret in mapped domain (curve prime size)
- 4 bytes filled with zero (RFU)
- OID and mapped terminal public key (copied from `publicData` input), with TLV structure as shown below (unchanged from input):

```
<tag='7F49'><length>
<tag='06' 0ID><length='0A'><Crypto. mechanism ref.> // same as data received in MSE SET
<tag='86' pubkey><length>
04<mapped terminal public key> // copied from input public data
```

Note:

- The random nonce (input in `secret`) is replaced by new shared secret in mapped domain (output in `secret`)
- The mapped terminal public key (input in `publicData`) is copied to output in `secret`, where it replaces the terminal public key in standard domain input (input in `secret`). The rest of the buffer (TLV tags and length) remains unchanged.

10.3 Performance figures

In the absence of standard performance tests, typical cryptographic operations are timed. The protocol used is T=1, direct convention, Fi/Di='11' with BWI=4. The reader clock rate is 4.8 MHz. Power Class A (5 V) is used.

To avoid measuring communication and other overhead, the execution time is calculated as difference between the times measured for an APDU which executes the operation mentioned in the table and an APDU which does not execute the operation. The Java Card applet which was loaded onto the card for the performance measurements uses RAM for the output buffer.

The execution times below are typical values but cannot be guaranteed.

Tab. 10.4: AES, standard API

Operation	Cipher	Data length	Execution time
Encrypt 128 bit AES Key	ALG_AES_BLOCK_128_CBC_NOPAD	128 Byte	1.67 ms
Decrypt 128 bit AES Key	ALG_AES_BLOCK_128_CBC_NOPAD	128 Byte	1.71 ms
Encrypt 128 bit AES Key	ALG_AES_BLOCK_128_CBC_NOPAD	512 Byte	2.00 ms
Decrypt 128 bit AES Key	ALG_AES_BLOCK_128_CBC_NOPAD	512 Byte	2.00 ms

Tab. 10.5: AES, CryptoBaseX

Operation	Cipher	Data length	Execution time
Encrypt 128 bit AES_X Key (CryptoBaseX)	ALG_AES_BLOCK_128_CBC_NOPAD	128 Byte	2.00 ms
Decrypt 128 bit AES_X Key (CryptoBaseX)	ALG_AES_BLOCK_128_CBC_NOPAD	128 Byte	2.00 ms
Encrypt 128 bit AES_X Key (CryptoBaseX)	ALG_AES_BLOCK_128_CBC_NOPAD	512 Byte	3.00 ms
Decrypt 128 bit AES_X Key (CryptoBaseX)	ALG_AES_BLOCK_128_CBC_NOPAD	512 Byte	3.00 ms

Tab. 10.6: DES, standard API

Operation	Cipher	Data length	Execution time
Encrypt 3KEY 3DES	ALG_DES_CBC_NOPAD	128 Byte	1.68 ms
Decrypt 3KEY 3DES	ALG_DES_CBC_NOPAD	128 Byte	1.77 ms
Encrypt 3KEY 3DES	ALG_DES_CBC_NOPAD	512 Byte	2.20 ms
Decrypt 3KEY 3DES	ALG_DES_CBC_NOPAD	512 Byte	2.50 ms

Tab. 10.7: DES, CryptoBaseX

Operation	Cipher	Data length	Execution time
Encrypt 3KEY 3DES (CryptoBaseX)	ALG_DES_CBC_NOPAD	128 Byte	1.79 ms
Decrypt 3KEY 3DES (CryptoBaseX)	ALG_DES_CBC_NOPAD	128 Byte	1.89 ms
Encrypt 3KEY 3DES (CryptoBaseX)	ALG_DES_CBC_NOPAD	512 Byte	2.30 ms
Decrypt 3KEY 3DES (CryptoBaseX)	ALG_DES_CBC_NOPAD	512 Byte	2.60 ms

11 Supported specifications

11.1 Applet specifications

- EMV Card Personalization Specification, Version 1.1, July 2007
- EMV Common Payment Application Specification, Version 1.0, December 2005
- M/Chip Multi-application Requirements, Multiple M/Chip Instances on a Single Card, 30 June 2016
- M/Chip Advance Card Application Specification Payment & Data Storage Version 1.2.1, August 2016
- Multiple M/Chip Instances - MCADS Personalization Profiles Add-On for IAT version 1.02, November 2016
- PKI Specifications (PKCS#11, PKCS#15) — middleware for Windows, Linux and Mac also available
- Visa Integrated Circuit Card Specification (VIS) 1.6
- Visa Contactless Payment Specification (VCPS) 2.2

11.2 Operating system specifications

- EMV Integrated Circuit Card Specifications for Payment Systems, Book 1 through 4, version 4.3, EMVCo, November 2011.
- EMV Contactless Specification for Payment Systems, Book A through D, version 2.6, EMVCo, August 2016.
- GlobalPlatform 2.2.1 ID Configuration:
 - Multi-application environment, post issuance loading, delegated management and lifecycle management.
 - Secure Channel Protocol (SCP) 01, 02 and 03.
- GlobalPlatform Card Specification 2.3, GlobalPlatform, December 2016.
- EMV 4.3, EMVCo, November 2011.
- EMV Contactless 2.6, EMVCo, 2016.
- 3.0.4 Classic, Oracle Corporation, September 2011.
- 3.0.5 Classic, Oracle Corporation, June 2015. Memory management and garbage collection are supported.
- ISO 7816 (contact)
- ISO 14443 (contactless)

- PUF: interface to the physical unclonable function provided by the hardware.
- Secure Box: interface that allows native libraries to be stored and securely used on the hardware.

12 Acronyms

12.1 Alphanumeric

- 3DES – Data Encryption Standard with 3 keys

12.2 A

- AC – Attack Counter
- AES – Advanced Encryption Standard
- AID – Application Identifier
- APDU – Application Protocol Data Unit as defined in ISO/IEC 7816
- API – Application Programming Interface
- ARC – Authentication Retry Counter
- ATR – Answer to Reset ISO-7816
- ATS – Answer to Select ISO-14443
- ATQA – Answer To Request, Type A

12.3 C

- CAP – Converted Applet format
- CBC – Cipher Block Chaining
- CIU – Contactless Interface Unit
- CLA – Class byte of the command message
- CPLC – Card Production Life Cycle
- CQS – Customer Qualification Samples
- CVM – Cardholder Verification Method

12.4 D

- DAP – Data Authentication Pattern
- DES – Data Encryption Standard
- DGI – Data Grouping Identifier

12.5 E

- ECB – Electronic codebook
- ECC – Elliptic Curve Cryptography
- ELF – Executable Load File

12.6 F

- FWT – Frame Waiting Time

12.7 I

- ICV – Initial Chaining Vector
- IFSD – Information Field Size Interface Device
- ISD – Issuer Security Domain
- ISO – International Organization for Standardization

12.8 K

- KVN – Key Version Number

12.9 L

- LFDBH – Load File Data Block Hash

12.10 N

- NVM – Non-Volatile Memory

12.11 O

- OEF – Order Entry Form
- OS – Operating System

12.12 P

- PIN – Personal Identification Number

12.13 R

- ROM – Read Only Memory
- RSA – Rivest Shamir Adleman asymmetric algorithm

12.14 S

- SAK – Select Acknowledge
- SCP – Secure Channel Protocol
- SCP01 – Secure Channel Protocol '01'
- SCP02 – Secure Channel Protocol '02'
- SCP03 – Secure Channel Protocol '03'
- SD – Security Domain
- SHA – Secure Hash Algorithm
- SSD – Supplementary Security Domain
- SW – Status Word

12.15 T

- TLV – Tag Length Value

12.16 U

- UID – Unique Identifier

12.17 V

- VHBR – Very High Baud Rate

12.18 W

- WTX – Waiting Time Extension

13 Contents

1 Introduction	2		
1.1 Audience	2	4.4.2	Garbage collection 29
1.2 Roles	2	4.4.3	Transaction support 29
1.3 Reading this document	3	4.4.4	Card termination behavior 30
2 Product identification	4	4.4.5	Limitations 30
3 Product description	5	4.5	GlobalPlatform 2.3 30
3.1 Application options	5	4.5.1	Framework 31
3.2 Communication interfaces	7	4.5.2	APDUs 32
3.3 Integrated MoC	8	4.5.3	Command processing 33
3.4 Cryptographic algorithms and key sizes	8	4.5.4	API packages 33
3.5 Product customization options	9	4.5.5	Issuer Security Domain (ISD) 35
3.6 Available memory	10	4.5.6	Supplementary Security Domain (SSD) 35
3.7 Designed-in support	11	4.5.7	Secure channel protocols 35
4 Standard features	12	4.5.8	Configuration 37
4.1 JCOP 4 P71 product family features	12	4.5.9	Limitations 38
4.2 Java Card 3.0.5 Classic	12	4.6	Communications 40
4.2.1 API packages	13	4.6.1	Contactless interface 40
4.2.2 Protocols	13	4.6.2	Contact-based interface 40
4.2.3 Sending response data	14	4.7	WTX configurations 41
4.2.4 Extended length APDU support	15	4.7.1	Protocol T=0 41
4.2.5 Cryptography	15	4.7.2	Protocol T=CL 42
4.2.6 Exception handling	24	5 Proprietary features and platform-dependent behavior	43
4.2.7 ECDSA signature length	26	5.1	Additional APDUs 43
4.2.8 Limitations	27	5.1.1	Card identification 43
4.3 Java Card Virtual Machine	28	5.1.2	ISD administration 49
4.3.1 CAP file restrictions	28	5.1.3	Card information 53
4.4 Runtime Environment	29	5.1.4	Retrieving the FIPS configuration and triggering self tests 59
4.4.1 Multiple logical channels	29	5.2	OS modules 60

5.2.1	Config Module	61	5.9.1	SCP protocols	156
5.2.2	eGov accelerators	61	5.9.2	Unsuccessful or incomplete authentication	157
5.2.3	RSA key generation	61	5.10	Flash statistics	158
5.2.4	ECC	61	5.11	Proprietary APDUs	159
5.2.5	Korean Seed	62	5.12	Proprietary status words	159
5.2.6	FIPS	62	5.13	Platform-specific Java Card 3.0.5 features .	159
5.2.7	PIV (Opacity)	62	5.13.1	OwnerPINxWithPredecrement.decrementTriesRemaining	
5.2.8	Secure Box	62			
5.2.9	PACE IM	62	6 Product security		160
5.2.10	Module behavior	62	6.1	Procedural requirements	160
5.2.11	Limitations	62	6.2	Operating environment requirements	161
5.3	MIFARE	63	6.3	Applet requirements	162
5.4	JCOPX Java Card API extension	63	6.4	Cryptographic requirements	162
5.4.1	JCOPX Accelerator	64	7 Pre-personalization (OS initialization)		164
5.4.2	JCOPX eGovAccelerators	65	7.1	Transport state	165
5.4.3	JCOPX Math	78	7.1.1	PUT DATA (Configuration) commands in transport mode	165
5.4.4	JCOPX Memory	83	7.2	Masking process	166
5.4.5	JCOPX MifareSupport.4	88	7.3	Key definition	166
5.4.6	JCOPX PIV	93	7.4	Config Module	167
5.4.7	JCOPX PUF	95	7.4.1	Velocity limitation	167
5.4.8	JCOPX RawComm	97	7.4.2	OS initialization command processing . .	167
5.4.9	JCOPX SecureBox	100	7.4.3	Bulk Update	170
5.4.10	JCOPX Security	110	7.4.4	OS configuration	171
5.4.11	JCOPX System	140	7.5	OS initialization command sequence	174
5.4.12	JCOPX Util	148			
5.5	Statistical random number generator test .	155	8 Product configuration		176
5.6	Uninitialized OwnerPin	155	8.1	Card Production Life Cycle (CPLC)	176
5.7	FIPS self-test	155	8.2	Supported unique identifiers	178
5.8	Attack detection	155	8.3	Configuration options	178
5.8.1	Behavior in case of an attack	155	8.3.1	Contactless communication	179
5.8.2	Restricted mode	156	8.3.2	Contact-based communication	183
5.9	Velocity limitations	156			

8.3.3	General communication	191	12 Acronyms	215
8.3.4	FIPS support	196	12.1	Alphanumeric 215
8.3.5	GlobalPlatform	197	12.2	A 215
8.3.6	Other configuration items	200	12.3	C 215
8.4	Customer's CPLC data (option)	202	12.4	D 215
9	Ordering and delivery	203	12.5	E 216
9.1	Key exchange	203	12.6	F 216
9.2	Customer type submission using Order Entry Form	203	12.7	I 216
9.2.1	Commercial agreements on sample delivery and volume delivery	204	12.8	K 216
9.3	Customer Flash data	204	12.9	L 216
9.4	Product delivery	204	12.10	N 216
9.4.1	Delivery as wafer	204	12.11	O 216
9.4.2	Delivery as module	205	12.12	P 216
10	Appendix	206	12.13	R 217
10.1	JCOP system AIDs	206	12.14	S 217
10.2	A note on ECDH GM	207	12.15	T 217
10.2.1	Proprietary implementation	207	12.16	U 217
10.2.2	Sub-steps performed	208	12.17	V 217
10.2.3	KeyAgreement class	208	12.18	W 217
10.2.4	KeyAgreement.generateSecret() proprietary method	209	13 Contents	218
10.3	Performance figures	210	14 Bibliography	221
11	Supported specifications	213	15 Legal information	223
11.1	Applet specifications	213	15.1	Definitions 223
11.2	Operating system specifications	213	15.2	Disclaimers 223
			15.3	Licenses 223
			15.4	Patents 224
			15.5	Trademarks 224

14 Bibliography

- [1] ISO/IEC 14443 Proximity Cards - Part 3: Initialization and anti-collision - ISO/IEC 14443-3:2011.
- [2] NIST Special Publication 800-38B Recommendation for Block Cipher Modes of Operation: The CMAC Mode for Authentication. http://csrc.nist.gov/publications/nistpubs/800-38B/SP_800-38B.pdf.
- [3] ETSI TS 102 226 v10.0.0 Smart Cards; Remote APDU structure for UICC based applications, October 2010.
- [4] ISO/IEC 19790:2012 Information technology - Security techniques - Security requirements for cryptographic modules, 2012.
- [5] NIST Special Publication 800-67 Recommendation for the Triple Data Encryption Algorithm (TDEA) Block Cipher, revision 2. <https://nvlpubs.nist.gov/nistpubs/specialpublications/nist.sp.800-67r2.pdf>, November 2017.
- [6] David Cooper, Hildegard Ferraiolo, Ketan Mehta, Salvatore Francomacaro, Ramaswamy Chandramouli, NIST and Jason Mohler, Electrosoft Services Inc. SP 800-73-4: Interfaces for Personal Identity Verification - Part 1: PIV Card Application Namespace, Data Model and Representation, May 2015.
- [7] EMVCo. Book 1: Application Independent ICC to Terminal Interface Requirements, rev. 4.3, November 2011.
- [8] EMVCo. SB-114: Case 4 Command Processing with Warning Conditions, February 2013.
- [9] Bundesamt fuer Sicherheit in der Informationstechnik. Anwendungshinweise und Interpretationen zum Schema, AIS 20: Funktionalitaetsklassen und Evaluationsmethodologie fuer deterministische Zufallszahlen-generatoren, Version 2.1, December 2 2011.
- [10] Bundesamt für Sicherheit in der Informationstechnik. Technische Richtlinie - Kryptographische Verfahren: Empfehlungen und Schlüssellängen, BSI-TR02102, January 9 2013.
- [11] GlobalPlatform. GlobalPlatform Card Mapping Guidelines of Existing GP v2.1.1 Implementation on v2.2.1, January 2011.
- [12] GlobalPlatform. GlobalPlatform ID Configuration, December 2011.
- [13] ICAO. Supplement to Doc 9303, TR - Supplemental Access Control for Machine Readable Travel Documents, May 13 2014.

- [14] GlobalPlatform Inc. GlobalPlatform Card Common Implementation Configuration 2.0, December 2015.
- [15] GlobalPlatform Inc. GlobalPlatform Card Specification 2.3, December 2015.
- [16] GlobalPlatform Inc. Configuration 2, GlobalPlatform Card Financial Configuration 2.0, January 2018.
- [17] ISO. ISO 7816-3: Part 3: Cards with contacts - Electrical interface and transmission protocols, November 2006.
- [18] ISO. ISO 7816-4: Identification cards - Integrated circuit cards - Organization, security and commands for interchange, April 2013.
- [19] ISO/IEC. ISO/IEC 14443 Proximity Cards - Part 4: Transmission protocol - ISO/IEC 14443-2:2008.
- [20] National Institute of Standards and Technology. FIPS PUB 140-2: FEDERAL INFORMATION PROCESSING STANDARDS PUBLICATION, Security Requirements for Cryptographic Modules, May 25 2001.
- [21] Oracle. Java Card 3 Platform, Application Programming Interface, Classic Edition, Version 3.0.5, May 2015.
- [22] Oracle. Java Card 3 Platform, Runtime Environment Specification, Classic Edition, Version 3.0.5, May 2015.
- [23] Oracle. Java Card 3 Platform, Virtual Machine Specification, Classic Edition, Version 3.0.5, May 2015.
- [24] NXP Semiconductors. SmartMX3 family N7121 - Wafer and Delivery Specification, Rev. 1.0, August 29 2017.
- [25] NXP Semiconductors. NXP Secure Smart Card Controller Antenna Design Guide Application Note, rev. 3.0, doc. no. 497630, November 23 2018.
- [26] NXP Semiconductors. SmartMX3 Family P71D321 - Delivery forms and electrical characteristics, doc. no. 458010, January 19 2018.
- [27] NXP Semiconductors. SmartMX3 Family P71D321 Overview, Pinning and Electrical Characteristics Product Short Data Sheet, rev. 3.0, doc. no. 412530, November 23 2018.
- [28] NXP Semiconductors. JCOP 4 P71D321 User Guidance and Administrator Manual, doc. no. 496535, March 22 2019.

15 Legal information

15.1 Definitions

Draft – The document is a draft version only. The content is still under internal review and subject to formal approval, which may result in modifications or additions. NXP Semiconductors does not give any representations or warranties as to the accuracy or completeness of information included herein and shall have no liability for the consequences of use of such information.

15.2 Disclaimers

Limited warranty and liability – Information in this document is believed to be accurate and reliable. However, NXP Semiconductors does not give any representations or warranties, expressed or implied, as to the accuracy or completeness of such information and shall have no liability for the consequences of use of such information.

In no event shall NXP Semiconductors be liable for any indirect, incidental, punitive, special or consequential damages (including - without limitation - lost profits, lost savings, business interruption, costs related to the removal or replacement of any products or rework charges) whether or not such damages are based on tort (including negligence), warranty, breach of contract or any other legal theory.

Notwithstanding any damages that customer might incur for any reason whatsoever, NXP Semiconductors' aggregate and cumulative liability towards customer for the products described herein shall be limited in accordance with the Terms and conditions of commercial sale of NXP Semiconductors.

Right to make changes – NXP Semiconductors reserves the right to make changes to information published in this document, including without limitation specifications and product descriptions, at any time and without notice. This document supersedes and replaces all information supplied prior to the publication hereof.

Suitability for use – NXP Semiconductors products are not designed, authorized or warranted to be suitable for use in life support, life-critical or safety-critical systems or equipment, nor in applications where failure or malfunction of an NXP Semiconductors product can reasonably be expected to result in personal injury, death or severe property or environmental damage. NXP Semiconductors accepts no liability for inclusion and/or use of NXP Semiconductors products in such equipment or applications and therefore such inclusion and/or use is at the customer's own risk.

Applications – Applications that are described herein for any of these products are for illustrative purposes only. NXP Semiconductors makes no representation or warranty that such applications will be suitable for the specified use without further testing or modification.

Customers are responsible for the design and operation of their applications and products using NXP Semiconductors products, and NXP Semiconductors accepts no liability for any assistance with applications or customer product design. It is customer's sole responsibility to determine whether the NXP Semiconductors product is suitable and fit for the customer's applications and products planned, as well as for the planned application and use of customer's third party customer(s). Customers should provide appropriate design and operating safeguards to minimize the risks associated with their applications

and products.

NXP Semiconductors does not accept any liability related to any default, damage, costs or problem which is based on any weakness or default in the customer's applications or products, or the application or use by customer's third party customer(s). Customer is responsible for doing all necessary testing for the customer's applications and products using NXP Semiconductors products in order to avoid a default of the applications and the products or of the application or use by customer's third party customer(s). NXP does not accept any liability in this respect.

Export control – This document as well as the item(s) described herein may be subject to export control regulations. Export might require a prior authorization from competent authorities.

Evaluation products – This product is provided on an "as is" and "with all faults" basis for evaluation purposes only. NXP Semiconductors, its affiliates and their suppliers expressly disclaim all warranties, whether express, implied or statutory, including but not limited to the implied warranties of non-infringement, merchantability and fitness for a particular purpose. The entire risk as to the quality, or arising out of the use or performance, of this product remains with customer.

In no event shall NXP Semiconductors, its affiliates or their suppliers be liable to customer for any special, indirect, consequential, punitive or incidental damages (including without limitation damages for loss of business, business interruption, loss of use, loss of data or information, and the like) arising out of the use of or inability to use the product, whether or not based on tort (including negligence), strict liability, breach of contract, breach of warranty or any other theory, even if advised of the possibility of such damages.

Notwithstanding any damages that customer might incur for any reason whatsoever (including without limitation, all damages referenced above and all direct or general damages), the entire liability of NXP Semiconductors, its affiliates and their suppliers and customer's exclusive remedy for all of the foregoing shall be limited to actual damages incurred by customer based on reasonable reliance up to the greater of the amount actually paid by customer for the product or five dollars (US\$5.00). The foregoing limitations, exclusions and disclaimers shall apply to the maximum extent permitted by applicable law, even if any remedy fails of its essential purpose.

15.3 Licenses

ICs with DPA Countermeasures functionality

NXP ICs containing functionality implementing countermeasures to Differential Power Analysis and Simple Power Analysis are produced and sold under applicable license from Cryptography Research, Inc.

15.4 Patents

Notice is herewith given that the subject device uses one or more of the following patents and that each of these patents may have corresponding patents in other jurisdictions.

<Patent ID> – owned by <Company name>

15.5 Trademarks

Notice: All referenced brands, product names, service names and trademarks are property of their respective owners.

MIFARE – is a trademark of NXP B.V.

Please be aware that important notices concerning this document and the product(s) described herein, have been included in the section 'Legal information'.
